Formal Specification of the Plutus Core Language

Plutus Core Team

5th November 2026

Abstract

This is intended to be a reference guide for developers who want to utilise the Plutus Core infrastructure. We lay out the grammar and syntax of untyped Plutus Core terms, and their semantics and evaluation rules. We also describe the built-in types and functions. The Appendices include a list of supported builtins in each era and some aspects of Plutus Core which have been mechanically formalised.

This document only describes untyped Plutus Core: a subsequent version will also include the syntax and semantics of Typed Plutus Core and describe its relation to untyped Plutus Core.

Contents

Contents 1

1 Preliminaries 4
- 1.1 Introduction . . . 4
- 1.2 Some basic notation . . . 4
  - 1.2.1 Sets . . . 4
  - 1.2.2 Lists . . . 5
  - 1.2.3 Bytestrings and bitstrings . . . 6
  - 1.2.4 Miscellaneous notation . . . 6

2 Untyped Plutus Core 7
- 2.1 The grammar of Plutus Core . . . 7
  - 2.1.1 Lexical grammar . . . 7
  - 2.1.2 Grammar . . . 8
  - 2.1.3 Notes . . . 8
- 2.2 Interpretation of built-in types and functions . . . 10
  - 2.2.1 Built-in types . . . 10
    - 2.2.1.1 Type variables . . . 10
    - 2.2.1.2 Polymorphic types . . . 11
    - 2.2.1.3 Type assignments . . . 11
    - 2.2.1.4 Casing on constants of built-in types . . . 12
  - 2.2.2 Built-in functions . . . 13
    - 2.2.2.1 Inputs to built-in functions . . . 13
    - 2.2.2.2 Outputs of built-in functions . . . 13
    - 2.2.2.3 Signatures and denotations of built-in functions . . . 14
    - 2.2.2.4 Denotations of built-in functions . . . 15
    - 2.2.2.5 Results of built-in functions. . . . 16
    - 2.2.2.6 Parametricity for fully-polymorphic arguments . . . 16
  - 2.2.3 Evaluation of built-in functions . . . 16
    - 2.2.3.1 Compatibility of inputs and signature entries . . . 16
    - 2.2.3.2 Evaluation . . . 17
- 2.3 Term reduction . . . 17
  - 2.3.1 Values in Plutus Core . . . 17
  - 2.3.2 Term reduction . . . 19
- 2.4 The CEK machine . . . 21
  - 2.4.1 Converting CEK evaluation results into Plutus Core terms . . . 24
- 2.5 Cost accounting for Untyped Plutus Core . . . 25

3 Typed Plutus Core **26**

4 Plutus Core on Cardano **27**
4.1 Protocol versions . . . 27
4.2 Ledger languages . . . 28
4.2.1 Evolution of Plutus ledger languages and Plutus Core language versions . . . 28
4.3 Built-in types and functions . . . 29
4.3.1 Batch 1 . . . 30
4.3.1.1 Built-in types and type operators . . . 30
4.3.1.2 Built-in functions . . . 34
4.3.2 Batch 2 . . . 39
4.3.2.1 Built-in functions . . . 39
4.3.3 Batch 3 . . . 39
4.3.3.1 Built-in functions . . . 39
4.3.4 Batch 4 . . . 40
4.3.4.1 Miscellaneous built-in functions . . . 40
4.3.4.2 BLS12-381 built-in types . . . 42
4.3.4.3 BLS12-381 built-in functions . . . 44
4.3.5 Batch 5 . . . 49
4.3.5.1 Built-in functions . . . 49
4.3.6 Batch 6 . . . 52
4.3.6.1 The `array` type operator . . . 52
4.3.6.2 Built-in functions (1) . . . 52
4.3.6.3 The built-in `value` type . . . 53
4.3.6.4 Built-in functions operating on `values` . . . 56
4.3.7 Batch 7 . . . 60
4.3.7.1 Built-in functions . . . 60

A Formally Verified Behaviours **62**

B Serialising `data` Objects Using the CBOR Format **63**
B.1 Introduction . . . 63
B.2 Notation . . . 64
B.3 The CBOR format . . . 64
B.4 Encoding and decoding the heads of CBOR items . . . 64
B.5 Encoding and decoding bytestrings . . . 66
B.6 Encoding and decoding integers . . . 67
B.7 Encoding and decoding `data` . . . 67

C Serialising Plutus Core Terms and Programs Using the `flat` Format **70**
C.1 Encoding and decoding . . . 70
C.1.1 Padding . . . 71
C.2 Basic `flat` encodings . . . 72
C.2.1 Fixed-width natural numbers . . . 72
C.2.2 Pairs . . . 72
C.2.3 Lists . . . 72
C.2.4 Natural numbers . . . 73
C.2.5 Integers . . . 73
C.2.6 Bytestrings . . . 73
C.2.7 Strings . . . 75

C.3 Encoding and decoding Plutus Core . . . 75
C.3.1 Programs . . . 75
C.3.2 Terms . . . 75
C.3.3 Built-in types . . . 77
C.3.4 Constants . . . 78
C.3.5 Built-in functions . . . 80
C.3.6 Variable names . . . 84
C.4 Cardano-specific serialisation issues . . . 84
C.4.1 Scope checking . . . 84
C.4.2 CBOR wrapping . . . 85
C.5 Example . . . 85

Bibliography **87**

Index of Notation **91**

Chapter 1

Preliminaries

1.1 Introduction

Plutus Core (more correctly, Untyped Plutus Core) is an eagerly-evaluated version of the untyped lambda calculus extended with some "built-in" types and functions; it is intended for the implementation of validation scripts on the Cardano blockchain. This document presents the syntax and semantics of Plutus Core, a specification of an efficient evaluator, a description of the built-in types and functions available in various releases of Cardano, and a specification of the binary serialisation format used by Plutus Core.

Since Plutus Core is intended for use in an environment where computation is potentially expensive and excessively long computations can be problematic we have also developed a costing infrastructure for Plutus Core programs. A description of this will be added in a later version of this document.

We also have a typed version of Plutus Core which provides extra robustness when untyped Plutus Core is used as a compilation target, and we will eventually provide a specification of the type system and semantics of Typed Plutus Core here as well, together with its relationship to Untyped Plutus Core.

1.2 Some basic notation

We begin with some notation which will be used throughout the document.

1.2.1 Sets

- $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$.
- $\mathbb{N}_+ = \{1, 2, 3, \ldots\}$.
- The symbol $\uplus$ denotes a disjoint union of sets; for emphasis we often use this to denote the union of sets which we know to be disjoint.
- Given sets X and Y with Y containing a distinguished element 0, we let

  $$\mathsf{FinSup}(X, Y) = \{f : X \to Y : f(x) = 0 \text{ for all but finitely many } x\}$$

  be the set of *finitely supported functions* from X to Y.
- Given a function $f : X \to Y$ and $a \in X$, $b \in Y$, $f[a \mapsto b](x)$ denotes f updated to map a to b:

  $$f[a \mapsto b](x) = \begin{cases} b & \text{if } x = a \\ f(x) & \text{otherwise} \end{cases}$$

- Given a set X, X^* denotes the set of finite sequences of elements of X:

$$X^* = \biguplus \{X^n : n \in \mathbb{N}\}$$

  and X^+ denotes the set of nonempty finite sequences of elements of X:

$$X^+ = \biguplus \{X^n : n \in \mathbb{N}_+\}.$$

  Elements of X^* and X^+ will usually be written using the usual tuple notation $(x_1, \ldots, x_n)$. The empty sequence (corresponding to the case $n = 0$) will be written ().

- $\mathbb{N}_{[a,b]} = \{n \in \mathbb{N} : a \leq n \leq b\}$.
- $\mathbb{B} = \mathbb{N}_{[0,255]}$, the set of 8-bit bytes.
- $\mathbb{B}^*$ is the set of all bytestrings.
- $\mathbb{b} = \{0, 1\}$, the set of bits.
- $\mathbb{b}^*$ is the set of all bitstrings.
- $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$.
- $\mathbb{F}_q$ denotes a finite field with q elements (q a prime power).
- $\mathbb{F}_q^\times$ denotes the multiplicative group of nonzero elements of $\mathbb{F}_q$.
- $\mathbb{U}$ denotes the set of Unicode scalar values, as defined in [54, Definition D76].
- $\mathbb{U}^*$ is the set of all Unicode strings.
- We assume that there is a special symbol $\times$ which does not appear in any other set we mention. The symbol $\times$ is used to indicate that some sort of error condition has occurred, and we will often need to consider situations in which a value is either $\times$ or a member of some set S. For brevity, if S is a set then we define

$$S_\times := S \uplus \{\times\}.$$

1.2.2 Lists

- The symbol [] denotes an empty list.
- The notation $[x_m, \ldots, x_n]$ denotes a list containing the elements $x_m, \ldots, x_n$. If $m > n$ then the list is empty.
- The length of a list L is denoted by $\ell(L)$.
- Given two lists $L = [x_1, \ldots, x_m]$ and $L' = [y_1, \ldots, y_n]$, $L{\cdot}L'$ denotes their concatenation $[x_1, \ldots, x_m, y_1, \ldots, y_n]$.
- Given an object x and a list $L = [x_1, \ldots, x_n]$, we denote the list $[x, x_1, \ldots, x_n]$ by $x{\cdot}L$.
- Given a list $L = [x_1, \ldots, x_n]$ and an object x, we denote the list $[x_1, \ldots, x_n, x]$ by $L{\cdot}x$.
- Given a syntactic category V, the symbol $\overline{V}$ denotes a possibly empty list $[V_1, \ldots, V_n]$ of elements $V_i \in V$.

1.2.3 Bytestrings and bitstrings

We make frequent use of bytestrings and bitstrings and for the sake of conciseness we occasionally use special notation. We also define conversion functions between bytestrings and bitstrings

- We typically index the bytes in bytestrings starting from the *left* end but the bits in bitstrings from the *right* end.
- The bytestring $[c_0, \ldots, c_{n-1}]$ may be denoted by $c_0 \cdots c_{n-1}$ ($n \geq 0$, $c_i \in \mathbb{B}$); the empty bytestring may be denoted by ϵ.
- The bitstring $[b_{n-1}, \ldots, b_0]$ may be denoted by $b_{n-1} \cdots b_0$ ($n \geq 0$, $b_i \in \mathbb{b}$); the empty bitstring may be denoted by ϵ: we also use this symbol for the empty bytestring, but this should not cause any confusion.
- In the special case of bitstrings we sometimes use notation such as `101110` to denote the list $[1, 0, 1, 1, 1, 0]$; we use a teletype font to avoid confusion with decimal numbers.
- A bytestring can naturally be viewed as a bitstring whose length is a multiple of 8 simply by concatenating the bits of the individual bytes, and vice-versa (by breaking the bitstring into groups of 8 bits). To make this precise we define two conversion functions $\mathsf{bits} : \mathbb{B}^* \to \mathbb{b}^*$ and $\mathsf{bytes} : \{s \in \mathbb{b}^* : 8 \mid \ell(s)\} \to \mathbb{B}^*$. These depend on the fact that any $c \in \mathbb{B}$ can be written uniquely in the form $\Sigma_{i=0}^{7} 2^i b_i$ with $b_0, \ldots, b_7 \in \mathbb{b}$.
  - $\mathsf{bits}([c_0, \ldots, c_{n-1}]) = [b_{8n-1}, \ldots, b_0]$ where $c_j = \Sigma_{i=0}^{7} 2^i b_{8(n-j-1)+i}$
  - $\mathsf{bytes}([b_{8n-1}, \ldots, b_0]) = [c_0, \ldots, c_{n-1}]$ where $c_j = \Sigma_{i=0}^{7} 2^i b_{8(n-j-1)+i}$.

1.2.4 Miscellaneous notation

- Given integers $k \in \mathbb{Z}$ and $n \geq 1$ we write $k \bmod n = \min\{r \in \mathbb{Z} : r \geq 0 \text{ and } n | k - r\}$

Chapter 2

Untyped Plutus Core

2.1 The grammar of Plutus Core

This section presents the grammar of Plutus Core in a Lisp-like form. This is intended as a specification of the abstract syntax of the language; it may also by used by tools as a concrete syntax for working with Plutus Core programs, but this is a secondary use and we do not make any guarantees of its completeness when used in this way. The primary concrete form of Plutus Core programs is the binary format described in Appendix C.

2.1.1 Lexical grammar

Name	n	::=	`[a-zA-Z][a-zA-Z0-9_']*(-[0-9]+)?`	name
Var	x	::=	n	term variable
BuiltinName	bn	::=	n	built-in function name
Version	v	::=	`[0-9]+.[0-9]+.[0-9]+`	version
Natural	k	::=	`[0-9]+`	a natural number
Constant	c	::=	⟨literal constant⟩	

Figure 2.1: Lexical grammar of Plutus Core

2.1.2 Grammar

Term	L, M, N	::=	x	variable
			`(con` T c`)`	constant
			`(builtin` b`)`	builtin
			`(lam` x M`)`	λ abstraction
			`[`M N`]`	function application
			`(delay` M`)`	delay execution of a term
			`(force` M`)`	force execution of a term
			`(constr` k $M_1 \ldots M_m$`)`	constructor with tag k and m arguments ($m \geq 0$)
			`(case` M $N_1 \ldots N_n$`)`	case analysis with n alternatives ($n \geq 0$)
			`(error)`	error
Program	P	::=	`(program` v M`)`	versioned program

Figure 2.2: Grammar of untyped Plutus Core

2.1.3 Notes

Version numbers. The version number at the start of a program specifies the Plutus Core language version used in the program.

A *Plutus Core language version* describes a version of the basic language with a particular set of features. A language version consists of three non-negative integers separated by decimal points, for example `1.4.2`. Language versions are ordered lexicographically.

The grammar above describes Plutus Core version 1.1.0. Version 1.0.0 is identical, except that `constr` and `case` are not included. Version 1.0.0 is fully forward-compatible with version 1.1.0, so any valid version 1.0.0 program is also a valid version 1.1.0 program. The semantics, evaluator and serialisation formats described later in this document all apply to both versions, except that it is an error to use `constr` or `case` in any program with a version prior to 1.1.0: a parser, deserialiser, or evaluator should fail immediately if `constr` or `case` is encountered when processing such a program.

Scoping. For simplicity, **we assume throughout that the body of a Plutus Core program is a closed term**, ie, that it contains no free variables. Thus `(program 1.0.0 (lam x x))` is a valid program but `(program 1.0.0 (lam x y))` is not, since the variable `y` is free. This condition should be checked before execution of any program commences, and the program should be rejected if its body is not closed. The assumption implies that any variable x occurring in the body of a program must be bound by an occurrence of `lam` in some enclosing term; in this case, we always assume that x refers to the *most recent* (ie, innermost) such binding.

Iterated applications. An application of a term M to a term N is represented by `[`M N`]`. We may occasionally write `[`M $N_1 \ldots N_k$`]` or `[`M $\overline{N}$`]` as an abbreviation for an iterated application $[\ldots[[M\ N_1]\ N_2]\ldots N_k]$, and tools may also use this as concrete syntax. If the sequence $N_1 \ldots N_k$ is empty (which will happen when $k = 0$, for instance) then `[`M $\overline{N}$`]` is just M.

Constructors and case analysis. In version 1.1.0 and later, Plutus Core supports creating structured data using `constr` and deconstructing it using `case`. Both of these terms are unusual in that they have (possibly empty) lists of children: `constr` has the (0-based) *tag* and then a list of arguments; `case` has a

scrutinee and then a list of case branches. The behaviour of `constr` and `case` is mostly straightforward: `constr` evaluates its arguments and forms a value; `case` evaluates the scrutinee into a `constr` value, selects the branch corresponding to the tag on the value (if the tag is k and the branches are $N_1, \ldots, N_n$ then it selects N_{k+1}, with an error occurring if k does not lie between 0 and $n-1$), and then applies that to the arguments in the value. Note that `case` does *not* strictly evaluate the case branches, only applying (and hence evaluating) the one that is eventually selected. The list of branches in `case` is allowed to be empty, but in that case there will be an error if it is ever applied to a scrutinee.

Constructor tags. Constructor tags can in principle be any natural number. In practice, since they cannot be dynamically constructed, we can limit them to a fixed size without having to worry about overflow. So we limit them to 64 bits, although this is currently only enforced in the binary format (see Section C.3.2).

Built-in types and functions. The language is parameterised by a set $\mathcal{U}$ of *built-in types* (we sometimes refer to $\mathcal{U}$ as the *universe*) and a set $\mathcal{B}$ of *built-in functions* (*builtins* for short), both of which are sets of Names. Briefly, the built-in types represent sets of constants such as integers or strings; constant expressions `(con` T c`)` represent values of the built-in types (the integer 123 or the string `"string"`, for example), and built-in functions are functions operating on these values, and possibly also general Plutus Core terms. Precise details are given in Section 2.2.

See Section 4.3 for a description of the types and functions which have already been deployed on the Cardano blockchain (or will be in the near future).

Casing on built-in types. Some built-in types support branching using `case`. For example, with Cardano's default universe, if i is a variable bound to a value of the `integer` type then `(case` i $M_1 \ldots M_5$`)` will cause the term M_{i+1} to be evaluated if i lies between 0 and 4, with `(error)` occurring if $i < 0$ or $i > 4$. See Section 2.2.1.4 for details and Section 4.3 (Table 4.8) for the types which can currently be used with `case` on the Cardano blockchain and a specification of how `case` works for them.

De Bruijn indices. The grammar defines names to be textual strings, but occasionally (specifically in Appendix C) we want to use de Bruijn indices ([32], [4, C.3]), and for this we redefine names to be natural numbers. In de Bruijn terms, λ-expressions do not need to bind a variable, but in order to re-use our existing syntax we arbitrarily use 0 for the bound variable, so that all λ-expressions are of the form `(lam 0` M`)`; other variables (ie, those not appearing immediately after a `lam` binder) are represented by natural number greater than zero.

Name suffixes. Names may include an optional numeric suffix consisting of a dash followed by one or more digits (for example, `x-0`, `name-42`, or `variable-12345`). This suffix provides a mechanism for disambiguating variables that share the same base name, allowing multiple distinct variables with identical textual representations to coexist within a program.

Lists in constructor and case terms. The grammar defines constructor and case terms to have a variable number of subterms written in sequence with no delimiters. This corresponds to the concrete syntax, e.g. we write `(constr 0` t_1 t_2 t_3`)`. However, in the rest of the specification we will abuse notation and treat these terms as having *lists* of subterms.

2.2 Interpretation of built-in types and functions

As mentioned above, Plutus Core is generic over a universe $\mathcal{U}$ of types and a set $\mathcal{B}$ of built-in functions. As the terminology suggests, built-in functions are interpreted as functions over terms and elements of the built-in types: in this section we make this interpretation precise by giving a specification of built-in types and functions in a set-theoretic denotational style. We require a considerable amount of extra notation in order to do this, and we emphasise that nothing in this section is part of the syntax of Plutus Core: it is meta-notation introduced purely for specification purposes.

2.2.1 Built-in types

We require some extra syntactic notation for built-in types: see Figure 2.3.

$$\begin{array}{rcll} at & ::= & n & \text{Atomic type} \\ op & ::= & n & \text{Type operator} \\ T & ::= & at \mid op(T, T, ..., T) & \text{Built-in type} \end{array}$$

Figure 2.3: Type names and operators

We assume that we have a set $\mathcal{U}_0$ of *atomic type names* and a set $\mathcal{O}$ of *type operator names*. Each type operator name $op \in \mathcal{O}$ has an *argument count* $|op| \in \mathbb{N}_+$, and a type name $op(T_1, \ldots, T_n)$ is well-formed if and only if $n = |op|$. We define the *universe* $\mathcal{U}$ to be the closure of $\mathcal{U}_0$ under repeated applications of operators in $\mathcal{O}$:

$$\mathcal{U}_{i+1} = \mathcal{U}_i \cup \{op(T_1, \ldots, T_{|op|}) : op \in \mathcal{O}, T_1, \ldots, T_{|op|} \in \mathcal{U}_i\}$$

$$\mathcal{U} = \bigcup \{\mathcal{U}_i : i \in \mathbb{N}_+\}$$

The universe $\mathcal{U}$ consists entirely of *names*, and the semantics of these names are given by *denotations*. Each built-in type $T \in \mathcal{U}$ is associated with some mathematical set $[\![T]\!]$, the *denotation* of T. For example, we might have $[\![\texttt{bool}]\!] = \{\mathsf{true}, \mathsf{false}\}$ and $[\![\texttt{integer}]\!] = \mathbb{Z}$ and $[\![\texttt{pair}\,(a, b)]\!] = [\![a]\!] \times [\![b]\!]$. We assume that if $T, T' \in \mathcal{U}$ and $T \neq T'$ then $[\![T]\!]$ and $[\![T']\!]$ are disjoint, and we put

$$[\![\mathcal{U}]\!] = \biguplus \{[\![T]\!] : T \in \mathcal{U}\}.$$

See Section 4.3 for a description of the types and functions which have already been deployed on the Cardano blockchain (or will be in the near future).

For non-atomic type names $T = op(T_1, \ldots, T_r)$ we would generally expect the denotation of T to be obtained in some uniform way (ie, parametrically) from the denotations of $T_1, \ldots, T_r$; we do not insist on this though.

2.2.1.1 Type variables

Built-in functions can be polymorphic, and to deal with this we need *type variables*. An argument of a polymorphic function can be either restricted to built-in types or can be an arbitrary term, and we define two different kinds of type variables to cover these two situations. See Figure 2.4.

$$\begin{array}{llll} \text{TypeVariable} & tv & ::= & n_{\#} \quad \text{built-in-polymorphic type variable} \\ & & & * \qquad \text{fully-polymorphic type variable} \end{array}$$

Figure 2.4: Type variables

We denote the set of all possible type variables by $\mathcal{V}$ and the set of all built-in-polymorphic type variables $v_{\#}$ by $\mathcal{V}_{\#}$. Note that $\mathcal{V} \cap \mathcal{U} = \varnothing$ since the symbols $*$ and $_{\#}$ do not occur in names in $\mathcal{U}$.

The two kinds of type variable are required because we have two different kinds of polymorphism. Later on we will see that built-in functions can take arguments which can be of a type which is unknown but must be in $\mathcal{U}$, whereas other arguments can range over a larger set of values such as the set of all Plutus Core terms. Type variables in $\mathcal{V}_{\#}$ are used in the former situation and $*$ is used in the latter.

2.2.1.2 Polymorphic types

We also need to talk about polymorphic types, and to do this we define an extended universe of polymorphic types $\mathcal{U}_{\#}$ by adjoining $\mathcal{V}_{\#}$ to $\mathcal{U}_0$ and closing under type operators as before:

$$\begin{gathered} \mathcal{U}_{\#,0} = \mathcal{U}_0 \cup \mathcal{V}_{\#} \\ \mathcal{U}_{\#,i+1} = \mathcal{U}_{\#,i} \cup \{op(T_1, \dots, T_{|op|}) : op \in \mathcal{O}, T_1, \dots, T_{|op|} \in \mathcal{U}_{\#,i}\} \\ \mathcal{U}_{\#} = \bigcup \{\mathcal{U}_{\#,i} : i \in \mathbb{N}_+\}. \end{gathered}$$

We will denote a typical element of $\mathcal{U}_{\#}$ by the symbol P (possibly subscripted).
We define the set of *free #-variables* of an element of $\mathcal{U}_{\#}$ by

$$\begin{gathered} \mathsf{FV}_{\#}(P) = \varnothing \text{ if } P \in \mathcal{U}_0 \\ \mathsf{FV}_{\#}(v_{\#}) = \{v_{\#}\} \\ \mathsf{FV}_{\#}(op(P_1, \dots, P_k)) = \mathsf{FV}_{\#}(P_1) \cup \mathsf{FV}_{\#}(P_2) \cup \cdots \cup \mathsf{FV}_{\#}(P_r). \end{gathered}$$

Thus $\mathsf{FV}_{\#}(P) \subseteq \mathcal{V}_{\#}$ for all $P \in \mathcal{U}$. We say that a type name $P \in \mathcal{U}_{\#}$ is *monomorphic* if $\mathsf{FV}_{\#}(P) = \varnothing$ (in which case we actually have $P \in \mathcal{U}$); otherwise P is *polymorphic*. The fact that type variables in $\mathcal{U}_{\#}$ are only allowed to come from $\mathcal{V}_{\#}$ will ensure that values of polymorphic types such as lists and pairs can only contain values of built-in types: in particular, we will not be able to construct types representing things such as lists of Plutus Core terms.

2.2.1.3 Type assignments

A *type assignment* is a function $S : D \to \mathcal{U}$ where D is some subset of $\mathcal{V}_{\#}$. As usual we say that D is the *domain* of S and denote it by $\operatorname{dom} S$.

We can extend a type assignment S to a map $\hat{S} : \mathcal{U}_{\#} \uplus \{*\} \to \mathcal{U}_{\#} \uplus \{*\}$ by defining

$$\begin{gathered} \hat{S}(v_{\#}) = S(v_{\#}) \quad \text{if } v_{\#} \in \operatorname{dom} S \\ \hat{S}(v_{\#}) = v_{\#} \quad \text{if } v_{\#} \in \mathcal{V}_{\#} \setminus \operatorname{dom} S \\ \hat{S}(T) = T \quad \text{if } T \in \mathcal{U}_0 \\ \hat{S}(op(P_1, \dots, P_n)) = op(\hat{S}(P_1), \dots, \hat{S}(P_n)) \\ \hat{S}(*) = *. \end{gathered}$$

If $P \in \mathcal{U}_\#$ and S is a type assignment with $\mathsf{FV}_\#(P) \subseteq \operatorname{dom} S$ then in fact $\hat{S}(P) \in \mathcal{U}$; in this case we say that $\hat{S}(P)$ is an *instance* or a *monomorphisation* of P (*via* S). If T is an instance of P then there is a unique smallest S (with $\mathsf{FV}_\#(P) = \operatorname{dom} S$) such that $T = \hat{S}(P)$: we write $T \preceq_S P$ to indicate that T is an instance of P via S and S is minimal.

Constructing type assignments. We say that a collection $\{S_i : 1 \leq i \leq n\}$ of type assignments is *consistent* if $S_i|_{D_{ij}} = S_j|_{D_{ij}}$ for all i and j, where $|$ denotes function restriction and $D_{ij} = \operatorname{dom} S_i \cap \operatorname{dom} S_j$. If this is the case then (viewing functions as sets of pairs in the usual way) $S_1 \cup \cdots \cup S_n$ is also a well-formed type assignment (each variable in its domain is associated with exactly one type).

Given $T \in \mathcal{U}$ and $P \in \mathcal{U}_\#$ it can be shown that $T \preceq_S P$ if and only if one of the following holds:

- $T = P$ and $S = \varnothing$.
- $P \in \mathcal{V}_\#$ and $S = \{(v_\#, T)\}$.
- – $T = op(T_1, \ldots, T_n)$ with each $T_i \in \mathcal{U}$.
  – $P = op(P_1, \ldots, P_n)$ with each $P_i \in \mathcal{U}_\#$.
  – $T_i \preceq_{S_i} P_i$ for $1 \leq i \leq n$.
  – $\{S_1, \ldots, S_n\}$ is consistent.
  – $S = S_1 \cup \cdots \cup S_n$.

This allows us to decide whether $T \in \mathcal{U}$ is an instance of $P \in \mathcal{U}_\#$ and, if so, to construct an S with $T \preceq_S P$.

2.2.1.4 Casing on constants of built-in types

In concrete implementations, some built-in types may admit *casing*, which allows constants of that type to be used as scrutinees of the `case` construct. One can write `(case` M $M_1 \ldots M_n$`)` where M evaluates to a constant x of some built-in type T, and one of the terms M_i will be evaluated, perhaps applied to some built-in values which will generally be obtained from x. For example, `(case (con integer` k`)` $M_1 \ldots M_n$`)` will evaluate M_{k+1} if $0 \leq k \leq n-1$ and will evaluate to `(error)` otherwise. If x is `(con (pair boolean integer) (False, 5))` then `(case` x M_1`)` will cause the application `[[`M_1 `(con boolean False)] (con integer 5)]` to be evaluated (possibly failing if M_1 is of the wrong type). For pairs, exactly one branch is required, with `(error)` occurring otherwise. In general the number of branches for a particular type is variable, with an error occurring if `case` would cause a non-existent branch to be selected.

To specify this behaviour in general we require some more notation. For each built-in type $T \in \mathcal{U}$ we assume that there is a *branch bound* $\mathsf{maxBranch}(T) \in \mathbb{N} \uplus \{\infty\}$ (we extend the usual ordering on $\mathbb{N}$ to this set by assuming that $\infty > n$ for all $n \in \mathbb{N}$) and a *matching function*

$$\mathsf{match}_T : [\![T]\!] \to \mathbb{N}_+ \times [\![\mathcal{U}]\!]^* \uplus \{\times\}.$$

Given a built-in type T and $x \in [\![T]\!]$, $\mathsf{match}_T(x)$ returns either $\times$ (indicating an error) or a pair $(i, (v_1, \ldots, v_k))$ with $i \geq 1$ a natural number saying which branch of a `case` expression should be selected and $v_1, \ldots, v_k$ built-in values in $[\![\mathcal{U}]\!]$ which the selected branch expression should be applied to.

More precisely, if M is a term which evaluates to the constant `(con` T x`)` then the expression

`(case` M $M_1 \ldots M_n$`)`

will evaluate to `(error)` if any of the following holds:

- $n > \mathsf{maxBranch}(T)$ (too many branches)
- $\mathsf{match}_T(x) = \times$ (casing is not supported on the value x)
- $\mathsf{match}_T(x) = (i, (v_1, \ldots, v_k))$ and $i > n$ (the branch selected for x is missing).

Otherwise, the `case` expression will cause M_i to be evaluated with the arguments $\{\!\{v_1\}\!\}, \ldots, \{\!\{v_k\}\!\}$ (where $\{\!\{\cdot\}\!\}$ is a *reification* function which converts elements of $[\![\mathcal{U}]\!]$ back into constants in the term language: see Section 2.2.2.5 below). The case $k = 0$ is allowed, in which case the expression M_i is evaluated as normal.

Definitions of match_T and $\mathsf{maxBranch}(T)$ for Cardano's built-in types are given in Chapter 4. Not all built-in types permit casing, and `case` will always fail with `(error)` for such types: this can be ensured by specifying $\mathsf{maxBranch}(T) = 0$.

2.2.2 Built-in functions

2.2.2.1 Inputs to built-in functions

To treat the typed and untyped versions of Plutus Core uniformly it is necessary to make the machinery of built-in functions generic over a set $\mathcal{I}$ of *inputs* which are taken as arguments by built-in functions. In practice $\mathcal{I}$ will be the set of Plutus Core values or something very closely related.

We require $\mathcal{I}$ to have the following two properties:

- $\mathcal{I}$ is disjoint from $[\![T]\!]$ for all $T \in \mathcal{U}$
- There should be disjoint subsets $\mathcal{C}_T \subseteq \mathcal{I}$ (where $T \in \mathcal{U}$) of *constants of type* T and maps $[\![\cdot]\!]_T : \mathcal{C}_T \to [\![T]\!]$ (*denotation*) and $\{\!\{\cdot\}\!\}_T : [\![T]\!] \to \mathcal{C}_T$ (*reification*) such that $\{\!\{[\![c]\!]_T\}\!\}_T = c$ for all $c \in \mathcal{C}_T$. We do not require these maps to be bijective (for example, there may be multiple inputs with the same denotation), but the condition implies that $[\![\cdot]\!]_T$ is surjective and $\{\!\{\cdot\}\!\}_T$ is injective.

It is also convenient to let $[\![\mathcal{I}]\!] = \mathcal{I}$ and define both $[\![\cdot]\!]_\mathcal{I}$ and $\{\!\{\cdot\}\!\}_\mathcal{I}$ to be the identity function, and we write

$$[\![\mathcal{U}]\!]_\mathcal{I} = [\![\mathcal{U}]\!] \uplus \mathcal{I}.$$

For example, we could take $\mathcal{I}$ to be the set of all Plutus Core values (see Section 2.3.1), $\mathcal{C}_T$ to be the set of all terms of the form `(con` T c`)`, $[\![\cdot]\!]_T$ to be the function `(con` T c`)` $\mapsto c$, and $\{\!\{\cdot\}\!\}_T$ to be the function $c \mapsto$ `(con` T c`)`. For simplicity we are assuming that mathematical entities occurring as members of type denotations $[\![T]\!]$ are embedded directly as values c in Plutus Core constant terms. In reality, tools which work with Plutus Core will need some concrete syntactic representation of constants; we do not specify this here, but see Section 4.3 for suggested syntax for the built-in types currently in use on the Cardano blockchain.

2.2.2.2 Outputs of built-in functions

All built-in functions either fail (in which case we will regard them as returning the symbol $\times$) or return an object which lies in either the denotation of some built-in type T or in the set of inputs $\mathcal{I}$, ie, builtins return elements of the set

$$\mathcal{R}_\times := \biguplus \{[\![T]\!] : T \in \mathcal{U}\} \uplus \mathcal{I} \uplus \{\times\}.$$

2.2.2.3 Signatures and denotations of built-in functions

We will consistently use the symbol τ and subscripted versions of it to denote members of $\mathcal{U}_\# \uplus \{*\}$ in the rest of the document; these indicate the types of values consumed by built-in functions.

We also define a class of *quantifications* which are used to introduce type variables: a quantification is a symbol of the form $\forall v$ with $v \in \mathcal{V}$; the set of of all possible quantifications is denoted by $\mathcal{Q}$.

Signatures. Every built-in function $b \in \mathcal{B}$ has a *signature* $\sigma(b)$ which describes the types of its arguments and its return value: a signature is of the form

$$[\iota_1, \ldots, \iota_n] \to \omega$$

with

- $\iota_j \in \mathcal{U}_\# \uplus \{*\} \uplus \mathcal{Q}$ for all j
- $\omega \in \mathcal{U}_\# \uplus \{*\}$
- $|\{j : \iota_j \notin \mathcal{Q}\}| \geq 1$ (so $n \geq 1$)
- If ι_j involves $v \in \mathcal{V}$ then $\iota_k = \forall v$ for some $k < j$, and similarly for ω; in other words, any type variable v must be introduced by a quantification before it is used. (Here ι *involves* v if either $\iota = T \in \mathcal{U}_\#$ and $v \in \mathsf{FV}_\#(T)$ or $\iota = v = *$.)
- If ω involves $v \in \mathcal{V}$ then some ι_j must involve v; this implies that $\mathsf{FV}_\#(\omega) \subseteq \bigcup\{\mathsf{FV}_\#(\iota_j) : \iota_j \in \mathcal{U}_\#\}$.
- If $j \neq k$ and $\iota_j, \iota_k \in \mathcal{Q}$ then $\iota_j \neq \iota_k$; ie, no quantification appears more than once.
- If $\iota_i = \forall v \in \mathcal{Q}$ then some $i_j \notin \mathcal{Q}$ with $j > i$ must involve v (signatures are not allowed to contain phantom type variables).

For example, in our default set of built-in functions we have the functions `mkCons` with signature $[\forall a_\#, a_\#, \texttt{list}\,(a_\#)] \to \texttt{list}\,(a_\#)$ and `ifThenElse` with signature $[\forall *, \texttt{bool}, *, *] \to *$. When we use `mkCons` its arguments must be of built-in types, but the two final arguments of `ifThenElse` can be any Plutus Core values.

If b has signature $[\iota_1, \ldots, \iota_n] \to \omega$ then we define the *arity* of b to be

$$\alpha(b) = [\iota_1, \ldots, \iota_n].$$

We also define

$$\chi(b) = n.$$

We may abuse notation slightly by using the symbol σ to denote a specific signature as well as the function which maps built-in function names to signatures, and similarly with the symbol α.

Given a signature $\sigma = [\iota_1, \ldots, \iota_n] \to \omega$, we define the *reduced signature* $\bar{\sigma}$ to be

$$\bar{\sigma} = [\iota_j : \iota_j \notin \mathcal{Q}] \to \omega$$

Here we have extended the usual set comprehension notation to lists in the obvious way, so $\bar{\sigma}$ just denotes the signature σ with all quantifications omitted. We will often write a reduced signature in the form $[\tau_1, \ldots, \tau_m] \to \omega$ to emphasise that the entries are *types*, and $\forall$ does not appear.

Also, given an arity $= [\iota_1, \ldots, \iota_n]$, the *reduced arity* is

$$\bar{\alpha} = [\iota_j \,:\, \iota_j \notin \mathcal{Q}].$$

Commentary. What is the intended meaning of the notation introduced above? In Typed Plutus Core we have to instantiate polymorphic functions (both built-in functions and polymorphic lambda terms) at concrete types before they can be applied, and in Untyped Plutus Core instantiation is replaced by an application of `force`. When we are applying a built-in function we supply its arguments one by one, and we can also apply `force` (or perform type instantiation in the typed case) to a partially-applied builtin "between" arguments (and also after the final argument); no computation occurs until all arguments have been supplied and all `force`s have been applied. The arity (read from left to right) specifies what types of arguments are expected and how they should be interleaved with applications of `force`, and $\chi(b)$ tells you the total number of arguments and applications of `force` that a built-in function b requires. The fully-polymorphic type variable $*$ indicates that an arbitrary value from $\mathcal{I}$ can be provided, whereas a type from $\mathcal{U}_\#$ indicates that a value of the specified built-in type is expected. Occurrences of quantifications indicate that `force` is to be applied to a partially-applied builtin; we allow this purely so that partially-applied builtins can be treated in the same way as delayed lambda-abstractions: `force` has no effect unless it is the very last item in the signature. In Plutus Core, partially-applied builtins are values which can be treated like any others (for example, by being passed as an argument to a `lam`-expression): see Section 2.3.1.

2.2.2.4 Denotations of built-in functions

The basic idea is that a built-in function b should represent some mathematical function on the denotations of the types of its inputs. However, this is complicated by the presence of polymorphism and we have to require that there is such a function for every possible monomorphisation of b.

More precisely, suppose that we have a builtin b with reduced signature $[\tau_1, \ldots \tau_n] \to \omega$. For every type assignment S with $\mathrm{dom}\, S = \mathsf{FV}_\#(\tau_1) \cup \cdots \cup \mathsf{FV}_\#(\tau_n)$ (which contains $\mathsf{FV}_\#(\omega)$ by the conditions on signatures in Section 2.2.2.3) we require a *denotation of b at S*, a function

$$[\![b]\!]_S \,:\, [\![\hat{S}(\tau_1)]\!] \times \cdots \times [\![\hat{S}(\tau_n)]\!] \to [\![\hat{S}(\omega)]\!]_\times$$

where

$$[\![*]\!] = \mathcal{I}.$$

This makes sense because $\hat{S}(\tau_i) \in \mathcal{U} \uplus \mathcal{I}$ for all i, so $[\![\hat{S}(\tau_i)]\!]$ is always defined, and similarly for ω.

If $\mathsf{FV}_\#(\bar{\sigma}(b)) = \emptyset$ (in which case we say that b is *monomorphic*) then the only relevant type assignment will be the empty one; in this case we have a single denotation

$$[\![b]\!]_\emptyset \,:\, [\![\tau_1]\!] \times \cdots \times [\![\tau_n]\!] \to [\![\omega]\!]_\times.$$

Denotations of builtins are mathematical functions which terminate on every possible input; the symbol $\times$ can be returned by a function to indicate that something has gone wrong, for example if an argument is out of range.

In practice we expect most builtins to be *parametrically polymorphic* [56, 49], so that the denotation $[\![b]\!]_S$ will be the "same" for all type assignments S; we do not insist on this though.

2.2.2.5 Results of built-in functions.

Recall from Section 2.2.2.2 that the result of the evaluation of a built-in function lies in the set

$$\mathcal{R}_{\mathsf{x}} = \biguplus \{[\![T]\!] : T \in \mathcal{U}\} \uplus \mathcal{I} \uplus \{\mathsf{x}\}.$$

Since we have assumed that all denotations $[\![T]\!]$ with $T \in \mathcal{U}$ are disjoint from each other and from $\mathcal{I}$ (Section 2.2.2.1) we can define a function

$$\{\!|\cdot|\!\} : \mathcal{R}_{\mathsf{x}} \to \mathcal{I}_{\mathsf{x}}$$

which converts elements $r \in \mathcal{R}_{\mathsf{x}}$ back into inputs (or the x symbol) by

$$\{\!|r|\!\} = \begin{cases} \{\!|r|\!\}_T \in \mathcal{C}_T \subseteq \mathcal{I} & \text{if } r \in [\![T]\!] \\ r & \text{if } r \in \mathcal{I} \\ \mathsf{x} & \text{if } r = \mathsf{x}. \end{cases}$$

See Section 2.2.2.1 for the definition of $\{\!|\cdot|\!\}_T$.

2.2.2.6 Parametricity for fully-polymorphic arguments

A built-in function b can only inspect arguments which are values of built-in types; other arguments (occurring as $*$ in $\bar{\sigma}(b)$) are treated opaquely, and can be discarded or returned as (part of) a result, but cannot be altered or examined (in particular, they cannot be compared for equality): b is *parametrically polymorphic* in such arguments. This implies that if the sequence returned by a builtin contains a value $v \in \mathcal{I}$, then v must have been an argument of the builtin.

2.2.3 Evaluation of built-in functions

2.2.3.1 Compatibility of inputs and signature entries

The previous section describes how a built-in function is interpreted as a mathematical function. When a Plutus Core built-in function b is applied to a sequence of arguments, the arguments must have types which are compatible with the signature of b; for example, if b has signature $[\forall *, \forall a_{\#}, \forall b_{\#}, a_{\#}, b_{\#}, a_{\#}, *, *] \to *$ and b is applied to a sequence of inputs V_1, V_2, V_3, V_4, V_5 then V_1, V_2, and V_3 must all be constants of some monomorphic built-in types and the types of V_1 and V_3 must be the same; V_4 and V_5 can be arbitrary inputs. This section describes the conditions for type compatibility.

In detail, given a reduced arity $\bar{\alpha} = [\tau_1, \ldots, \tau_n]$, a sequence $\bar{V} = [V_1, \ldots, V_m]$, and a type assignment S we say that $\bar{V}$ is *compatible with* $\bar{\alpha}$ (*via* S) if and only if $n = m$ and, letting $I = \{i : 1 \leq i \leq n, \tau_i \in \mathcal{U}_{\#}\}$ (so $\tau_j = *$ if $j \notin I$), there exist type assignments S_i ($1 \leq i \leq n$) such that all of the following are satisfied

- For all $i \in I$ there exists $T_i \in \mathcal{U}$ such that $V_i \in \mathcal{C}_{T_i}$ and $T_i \preceq_{S_i} \tau_i$.
- $\{S_i : i \in I\}$ is consistent (see Section 2.2.1.3).
- $S = \bigcup\{S_i : i \in I\}$.

If these conditions are all satisfied then we can find suitable S_i using the procedure described in Section 2.2.1.3 and this allows us to construct S explicitly since the S_i are consistent. Note that in this case $\mathrm{dom}\, S = \mathrm{dom}\, S_1 \cup \ldots \cup \mathrm{dom}\, S_n = \mathsf{FV}_\#(\tau_1) \cup \cdots \cup \mathsf{FV}_\#(\tau_n) = \mathsf{FV}_\#(\alpha)$, so S is minimal in the sense that no S' with $\mathrm{dom}\, S'$ strictly smaller than $\mathrm{dom}\, S$ is sufficient to monomorphise all of the τ_i simultaneously. We write

$$[V_1, \ldots, V_m] \approx_S [\tau_1, \ldots, \tau_n]$$

in this case. If $\bar{V}$ is not compatible with $\bar{\alpha}$ then we write $\bar{V} \not\approx \bar{\alpha}$.

2.2.3.2 Evaluation

For later use we define a function $\mathsf{EvalBuiltin}$ which attempts to evaluate an application of a built-in function b to a sequence of inputs $[V_1, \ldots, V_m]$. This fails if the number of inputs is incorrect or if the inputs are not compatible with $\bar{\alpha}(b)$:

$$\mathsf{EvalBuiltin}(b, [V_1, \ldots, V_n]) = \mathsf{X} \quad \text{if } [V_1, \ldots, V_n] \not\approx \bar{\alpha}(b).$$

Otherwise, the conditions for the existence of a denotation of b are met and we can apply that denotation to the denotations of the inputs and then reify the result. If $[V_1, \ldots, V_n] \approx_S \bar{\alpha}(b) = [\tau_1, \ldots, \tau_n]$, let $T_i = \hat{S}(\tau_i)$ for $1 \leq i \leq n$; then we define

$$\mathsf{EvalBuiltin}(b, [V_1, \ldots, V_n]) = \{\!|\, [\![b]\!]_S([\![V_1]\!]_{T_1}, \ldots, [\![V_n]\!]_{T_n}) \,|\!\}.$$

It can be checked that the compatibility condition guarantees that this makes sense according to the definition of $[\![b]\!]_S$ in Section 2.2.2.4.

Notes.

- All of the machinery which we have defined for built-in functions is parametric over the set $\mathcal{I}$ of inputs and the sets $\mathcal{C}_T \subseteq \mathcal{I}$ of constants. This also applies to the $\mathsf{EvalBuiltin}$ function, so its meaning is not fully defined until we have given concrete definitions of the sets of inputs and constants.
- The error value X can occur in two different ways: either because the arguments are not compatible with the signature, or because the builtin itself returns X to signal some error condition.
- The symbol X is not part of Plutus Core; when we define reduction rules and evaluators for Plutus Core later some extra translation will be required to convert the result of $\mathsf{EvalBuiltin}$ into something appropriate to the context.

2.3 Term reduction

This section defines the semantics of (untyped) Plutus Core.

2.3.1 Values in Plutus Core

The semantics of built-in functions in Plutus Core are obtained by instantiating the sets $\mathcal{C}_T$ of constants of type T (see Section 2.2.2.1) to be the expressions of the form `(con` T c`)` and the set $\mathcal{I}$ to be the set of Plutus Core *values*, terms which cannot immediately undergo any further reduction, such as lambda terms and delayed terms. Values also include partial applications of built-in functions such as `[(builtin`

modInteger) (con integer 5)], which cannot perform any computation until a second integer argument is supplied. However, partial applications must also be *well-formed*, in the sense that applications of `force` must be correctly interleaved with genuine arguments, and the arguments must themselves be values.

We define syntactic classes V of Plutus Core values and A of partial builtin applications simultaneously:

$$\begin{array}{lrll} \text{Value} & V & ::= & (\texttt{con}\ T\ c) \\ & & & (\texttt{delay}\ M) \\ & & & (\texttt{lam}\ x\ M) \\ & & & (\texttt{constr}\ i\ \overline{V}) \\ & & & A \end{array}$$

Figure 2.5: Values in Plutus Core

Here A is the class of well-formed partial applications, and to define this we first define a class of possibly ill-formed iterated applications B for each built-in function $b \in \mathcal{B}$:

$$\begin{array}{lrl} B & ::= & (\texttt{builtin}\ b) \\ & & [B\ V] \\ & & (\texttt{force}\ B) \end{array}$$

Figure 2.6: Partial built-in function application

We let B denote the set of terms generated by the grammar in Figure 2.6 and we define a function β which extracts the name of the built-in function occurring in a term in B:

$$\begin{array}{ll} \beta((\texttt{builtin}\ b)) & = b \\ \beta([B\ V]) & = \beta(B) \\ \beta((\texttt{force}\ B)) & = \beta(B) \end{array}$$

We also define a function $\|\cdot\|$ which measures the size of a term $B \in \mathsf{B}$:

$$\begin{array}{ll} \|(\texttt{builtin}\ b)\| & = 0 \\ \|[B\ V]\| & = 1 + \|B\| \\ \|(\texttt{force}\ B)\| & = 1 + \|B\| \end{array}$$

Well-formed partial applications. A term $B \in \mathsf{B}$ is an application of $b = \beta(B)$ to a number of values in S, interleaved with applications of `force`. We now define what it means for B to be a *well-formed partial application*. Suppose that $\alpha(b) = [\iota_1, \ldots, \iota_n]$. Firstly we require that $\|B\| < n$, so that b is not fully applied; in this case we put $\iota = \iota_{\|B\|}$, the element of b's signature which describes what kind of "argument" b currently expects. The definition is completed by induction on the structure of B:

1. $B = (\texttt{builtin}\ b)$ is always well formed.
2. $B = [B'\ V]$ is well formed if B' is well formed and $\iota \in \mathcal{U}_\#$ or $\iota = *$ (equivalently, $\iota \notin \mathcal{Q}$).
3. $B = (\texttt{force}\ B')$ is well formed if B' is well formed and $\iota \in \mathcal{Q}$.

The definition of values in Figure 2.5 is now completed by defining A to be the syntactic class of well-formed *partial* built-in function applications:

$$A = \{B \in \mathsf{B} : B \text{ is a well-formed partial application}\}.$$

Note that this definition does not impose any requirements of type correctness. For example, with the types and functions defined in Section 4.3.1 the term $X = \texttt{[(builtin modInteger) (con string "blue")]}$ is a valid value which could be treated like any other, for instance by being passed as an argument to a `lam` expression. However, the evaluation rules described in the next section require that when a built-in function b becomes *fully* applied the types of the arguments are checked against the signature of b using the relation $\approx$ and the function $\mathsf{EvalBuiltin}$ defined in Sections 2.2.3.1 and 2.2.3.2, so an error would arise if the term X were ever applied to another argument.

More notation. Suppose that A is a well-formed partial application with $\alpha(\beta(A)) = [\iota_1, \ldots, \iota_n]$. We define a function next which extracts the next argument (or `force`) expected by A:

$$\mathsf{next}(A) = \iota_{\|A\|+1}.$$

This makes sense because in a well-formed partial application A we have $\|A\| < n$.

We also define a function args which extracts the arguments which b has received so far in A:

$$\begin{array}{ll} \mathsf{args}(\texttt{(builtin } b\texttt{)}) & = [] \\ \mathsf{args}(\texttt{[}A\ V\texttt{]}) & = \mathsf{args}(A)\cdot V \\ \mathsf{args}(\texttt{(force } A\texttt{)}) & = \mathsf{args}(A). \end{array}$$

2.3.2 Term reduction

We define the semantics of Plutus Core using contextual semantics (or reduction semantics): see [39] or [37] or [40, 5.3], for example. We use A to denote a partial application of a built-in function as in Section 2.3.1 above. For builtin evaluation, we instantiate the set $\mathcal{I}$ of Section 2.2.2.1 to be the set of Plutus Core values. Thus all builtins take values as arguments and return a value or X. Since values are terms here, we can take $\{\!|V|\!\} = V$.

The notation $[V/x]M$ below denotes substitution of the value V for the variable x in M. This is *capture-avoiding* in that substitution is not performed on occurrences of x inside subterms of M of the form $\texttt{(lam } x\ N\texttt{)}$.

Recall also that $\texttt{[}M\ N_1 \ldots N_k\texttt{]}$ is an abbreviation for iterated application $[\ldots[[M\ N_1]\ N_2]\ldots N_k]$ and that $\texttt{[}M\ N_1 \ldots N_k\texttt{]} = M$ if $k = 0$.

$$
\begin{array}{llll}
\text{Frame} & f & ::= & \texttt{[_}\ M\texttt{]} & \text{left application} \\
 & & & \texttt{[}V\ \texttt{_]} & \text{right application} \\
 & & & \texttt{(force _)} & \text{force} \\
 & & & \texttt{(constr}\ i\ \overline{V}\ \texttt{_}\ \overline{M}\texttt{)} & \text{constructor argument} \\
 & & & \texttt{(case _}\ \overline{M}\texttt{)} & \text{case scrutinee}
\end{array}
$$

(a) Grammar of reduction frames for Plutus Core

$\boxed{M \ \to \ M'}$

Term M reduces in one step to term M'.

$$\frac{}{\texttt{[(lam}\ x\ M\texttt{)}\ V\texttt{]} \ \to \ [V/x]M}$$

$$\frac{\ell(A) = \chi(\beta(A)) - 1 \qquad \mathsf{next}(A) \in \mathcal{U}_{\#} \cup \{*\}}{\texttt{[}A\ V\texttt{]} \ \to \ \mathsf{EvalBuiltin}'(\beta(A), \mathsf{args}(A) \cdot V)}$$

$$\frac{\ell(A) < \chi(\beta(A)) - 1 \qquad \mathsf{next}(A) \in \mathcal{U}_{\#} \cup \{*\}}{\texttt{[}A\ V\texttt{]} \ \to \ \texttt{[}A\ V\texttt{]}}$$

$$\frac{}{\texttt{(force (delay}\ M\texttt{))} \ \to \ M}$$

$$\frac{0 \leq i \leq n-1}{\texttt{(case (constr}\ i\ \overline{V}\texttt{)}\ M_1 \ldots M_n\texttt{)} \ \to \ \texttt{[}M_{i+1}\ \overline{V}\texttt{]}}$$

$$\frac{i < 0 \ \text{ or } \ i > n-1}{\texttt{(case (constr}\ i\ \overline{V}\texttt{)}\ M_1 \ldots M_n\texttt{)} \ \to \ \texttt{(error)}}$$

$$\frac{n \leq \mathsf{maxBranch}(T) \qquad \mathsf{match}_T(c) = (i, (v_1, \ldots, v_k)) \qquad i \leq n}{\texttt{(case (con}\ T\ c\texttt{)}\ M_1 \ldots M_n\texttt{)} \ \to \ \texttt{[}M_i\ \{\!|v_1|\!\} \ldots \{\!|v_k|\!\}\texttt{]}}$$

$$\frac{n > \mathsf{maxBranch}(T)}{\texttt{(case (con}\ T\ c\texttt{)}\ M_1 \ldots M_n\texttt{)} \ \to \ \texttt{(error)}}$$

$$\frac{n \leq \mathsf{maxBranch}(T) \qquad \mathsf{match}_T(c) = \times}{\texttt{(case (con}\ T\ c\texttt{)}\ M_1 \ldots M_n\texttt{)} \ \to \ \texttt{(error)}}$$

$$\frac{n \leq \mathsf{maxBranch}(T) \qquad \mathsf{match}_T(c) = (i, (v_1, \ldots, v_k)) \qquad i > n}{\texttt{(case (con}\ T\ c\texttt{)}\ M_1 \ldots M_n\texttt{)} \ \to \ \texttt{(error)}}$$

(b) Reduction via contextual semantics

Figure 2.7: Term reduction for Plutus Core

$$\frac{\ell(A) = \chi(\beta(A)) - 1 \qquad \mathsf{next}(A) \in \mathcal{Q}}{\texttt{(force}\ A\texttt{)} \ \to\ \mathsf{EvalBuiltin}'(\beta(A), \mathsf{args}(A))}$$

$$\frac{\ell(A) < \chi(\beta(A)) - 1 \qquad \mathsf{next}(A) \in \mathcal{Q}}{\texttt{(force}\ A\texttt{)} \ \to\ A}$$

$$\frac{}{f\{\texttt{(error)}\} \ \to\ \texttt{(error)}}$$

$$\frac{M \ \to\ M'}{f\{M\} \ \to\ f\{M'\}}$$

(c) Reduction via contextual semantics (continued)

$$\mathsf{EvalBuiltin}'(b, [V_1, \ldots, V_n]) = \begin{cases} \texttt{(error)} & \text{if } \mathsf{EvalBuiltin}(b, [V_1, \ldots, V_n]) = \times \\ \mathsf{EvalBuiltin}(b, [V_1, \ldots, V_n]) & \text{otherwise.} \end{cases}$$

(d) Built-in function application

Figure 2.7: Term reduction for Plutus Core (continued)

It can be shown that any closed Plutus Core term whose evaluation terminates yields either `(error)` or a value. Recall from Section 2.1.3 that we require the body of every Plutus Core program to be closed.

2.4 The CEK machine

This section contains a description of an abstract machine for efficiently executing Plutus Core. This is based on the CEK machine of Felleisen and Friedman [38].

The machine alternates between two main phases: the *compute* phase ($\triangleright$), where it recurses down the AST looking for values, saving surrounding contexts as frames (or *reduction contexts*) on a stack as it goes; and the *return* phase ($\triangleleft$), where it has obtained a value and pops a frame off the stack to tell it how to proceed next. In addition there is an error state $\blacklozenge$ which halts execution with an error, and a halting state $\square$ which halts execution and returns a value to the outside world.

To evaluate a program `(program` v M`)`, we first check that the version number v is valid, then start the machine in the state $[]; [] \triangleright M$. It can be proved that the transitions in Figure 2.10 always preserve validity of states, so that the machine can never enter a state such as $[] \triangleleft M$ or s, `(force _)` $\triangleleft$ `(lam` x A M`)` which isn't covered by the rules. If such a situation were to occur in an implementation then it would indicate that the machine was incorrectly implemented or that it was attempting to evaluate an ill-formed program (for example, one which attempts to apply a variable to some other term).

State	Σ	$::=$	$s; \rho \rhd M \mid s \lhd V \mid \blacklozenge \mid \square V$
Stack	s	$::=$	f^*
CEK value	V	$::=$	$\langle \mathtt{con}\ T\ c \rangle \mid \langle \mathtt{delay}\ M\ \rho \rangle \mid \langle \mathtt{lam}\ x\ M\ \rho \rangle$
			$\mid \langle \mathtt{constr}\ i\ \overline{V} \rangle \mid \langle \mathtt{builtin}\ b\ \overline{V}\ \eta \rangle$
Environment	ρ	$::=$	$[] \mid \rho[x \mapsto V]$
Expected builtin arguments	η	$::=$	$[\iota] \mid \iota \cdot \eta$

Figure 2.8: Grammar of CEK machine states for Plutus Core

Frame	f	$::=$	$(\mathtt{force}\ _)$	force
			$[_\ (M, \rho)]$	left application to term
			$[_\ V]$	left application to value
			$[V\ _]$	right application of value
			$(\mathtt{constr}\ i\ \overline{V}\ _\ (\overline{M}, \rho))$	constructor argument
			$(\mathtt{case}\ _\ (\overline{M}, \rho))$	case scrutinee

Figure 2.9: Grammar of CEK stack frames

Figures 2.8 and 2.9 define some notation for *states* of the CEK machine: these involve a modified type of value adapted to the CEK machine, environments which bind names to values, and a stack which stores partially evaluated terms whose evaluation cannot proceed until some more computation has been performed (for example, since Plutus Core is a strict language function arguments have to be reduced to values before application takes place, and because of this a lambda term may have to be stored on the stack while its argument is being reduced to a value). Environments are lists of the form $\rho = [x_1 \mapsto V_1, \dots, x_n \mapsto V_n]$ which grow by having new entries appended on the right; we say that x *is bound in the environment* ρ if ρ contains an entry of the form $x \mapsto V$, and in that case we denote by $\rho[x]$ the value V in the rightmost (ie, most recent) such entry.*

To make the CEK machine fit into the built-in evaluation mechanism defined in Section 2.2 we define $\mathcal{I} = V$ and $\mathcal{C}_T = \{\langle \mathtt{con}\ T\ c \rangle : T \in \mathcal{U}, c \in [\![T]\!]\}$.

The rules in Figure 2.10 show the transitions of the machine; if any situation arises which is not included in these transitions (for example, if a frame $[\langle \mathtt{con}\ T\ c \rangle\ _]$ is encountered or if an attempt is made to apply `force` to a partial builtin application which is expecting a term argument), then the machine stops immediately in an error state.

*The description of environments we use here is more general than necessary in that it permits a given variable to have multiple bindings; however, in what follows we never actually retrieve bindings other than the most recent one and we never remove bindings to expose earlier ones. The list-based definition has the merit of simplicity and suffices for specification purposes but in an implementation it would be safe to use some data structure where existing bindings of a given variable are discarded when a new binding is added.

$\boxed{\Sigma \mapsto \Sigma'}$

Machine takes one step from state Σ to state Σ'

$$
\begin{array}{rl}
s;\rho \triangleright x & \mapsto s \triangleleft \rho[x] \text{ if } x \text{ is bound in } \rho \\
s;\rho \triangleright (\texttt{con}\ T\ c) & \mapsto s \triangleleft \langle \texttt{con}\ T\ c \rangle \\
s;\rho \triangleright (\texttt{lam}\ x\ M) & \mapsto s \triangleleft \langle \texttt{lam}\ x\ M\ \rho \rangle \\
s;\rho \triangleright (\texttt{delay}\ M) & \mapsto s \triangleleft \langle \texttt{delay}\ M\ \rho \rangle \\
s;\rho \triangleright (\texttt{force}\ M) & \mapsto (\texttt{force}\ _) \cdot s;\rho \triangleright M \\
s;\rho \triangleright [M\ N] & \mapsto [_\ (N,\rho)] \cdot s;\rho \triangleright M \\
s;\rho \triangleright (\texttt{constr}\ i\ M \cdot \overline{M}) & \mapsto (\texttt{constr}\ i\ [\ ]\ _\ (\overline{M},\rho)) \cdot s;\rho \triangleright M \\
s;\rho \triangleright (\texttt{constr}\ i\ [\ ]) & \mapsto s \triangleleft \langle \texttt{constr}\ i\ [\ ] \rangle \\
s;\rho \triangleright (\texttt{case}\ N\ \overline{M}) & \mapsto (\texttt{case}\ _\ (\overline{M},\rho)) \cdot s;\rho \triangleright N \\
s;\rho \triangleright (\texttt{builtin}\ b) & \mapsto s \triangleleft \langle \texttt{builtin}\ b\ [\ ]\ \alpha(b) \rangle \\
s;\rho \triangleright (\texttt{error}) & \mapsto \blacklozenge \\
[\ ] \triangleleft V & \mapsto \square V \\
[_\ (M,\rho)] \cdot s \triangleleft V & \mapsto [V\ _] \cdot s;\rho \triangleright M \\
[\langle \texttt{lam}\ x\ M\ \rho \rangle\ _] \cdot s \triangleleft V & \mapsto s;\rho[x \mapsto V] \triangleright M \\
[_\ V] \cdot s \triangleleft \langle \texttt{lam}\ x\ M\ \rho \rangle & \mapsto s;\rho[x \mapsto V] \triangleright M \\
[\langle \texttt{builtin}\ b\ \overline{V}\ (\iota \cdot \eta) \rangle\ _] \cdot s \triangleleft V & \mapsto s \triangleleft \langle \texttt{builtin}\ b\ (\overline{V} \cdot V)\ \eta \rangle \text{ if } \iota \in \mathcal{U}_{\#} \cup \{*\} \\
[_\ V] \cdot s \triangleleft \langle \texttt{builtin}\ b\ \overline{V}\ (\iota \cdot \eta) \rangle & \mapsto s \triangleleft \langle \texttt{builtin}\ b\ (\overline{V} \cdot V)\ \eta \rangle \text{ if } \iota \in \mathcal{U}_{\#} \cup \{*\} \\
[\langle \texttt{builtin}\ b\ \overline{V}\ [\iota] \rangle\ _] \cdot s \triangleleft V & \mapsto \mathsf{EvalBuiltin}_{\mathsf{CEK}}(s, b, \overline{V} \cdot V) \text{ if } \iota \in \mathcal{U}_{\#} \cup \{*\} \\
[_\ V] \cdot s \triangleleft \langle \texttt{builtin}\ b\ \overline{V}\ [\iota] \rangle & \mapsto \mathsf{EvalBuiltin}_{\mathsf{CEK}}(s, b, \overline{V} \cdot V) \text{ if } \iota \in \mathcal{U}_{\#} \cup \{*\} \\
(\texttt{force}\ _) \cdot s \triangleleft \langle \texttt{delay}\ M\ \rho \rangle & \mapsto s;\rho \triangleright M \\
(\texttt{force}\ _) \cdot s \triangleleft \langle \texttt{builtin}\ b\ \overline{V}\ (\iota \cdot \eta) \rangle & \mapsto s \triangleleft \langle \texttt{builtin}\ b\ \overline{V}\ \eta \rangle \text{ if } \iota \in \mathcal{Q} \\
(\texttt{force}\ _) \cdot s \triangleleft \langle \texttt{builtin}\ b\ \overline{V}\ [\iota] \rangle & \mapsto \mathsf{EvalBuiltin}_{\mathsf{CEK}}(s, b, \overline{V}) \text{ if } \iota \in \mathcal{Q} \\
(\texttt{constr}\ i\ \overline{V}\ _\ (M \cdot \overline{M}, \rho)) \cdot s \triangleleft V & \mapsto (\texttt{constr}\ i\ \overline{V} \cdot V\ _\ (\overline{M},\rho)) \cdot s;\rho \triangleright M \\
(\texttt{constr}\ i\ \overline{V}\ _\ ([\,], \rho)) \cdot s \triangleleft V & \mapsto s \triangleleft \langle \texttt{constr}\ i\ \overline{V} \cdot V \rangle \\
(\texttt{case}\ _\ (M_1 \dots M_n, \rho)) \cdot s \triangleleft \langle \texttt{constr}\ i\ V_1 \dots V_m \rangle & \mapsto [_\ V_1] \cdot \cdots \cdot [_\ V_m] \cdot s;\rho \triangleright M_{i+1} \text{ if } 0 \le i \le n-1 \\
(\texttt{case}\ _\ (M_1 \dots M_n, \rho)) \cdot s \triangleleft \langle \texttt{constr}\ i\ V_1 \dots V_m \rangle & \mapsto \blacklozenge \text{ if } i < 0 \text{ or } i > n-1 \\
(\texttt{case}\ _\ (M_1 \dots M_n, \rho)) \cdot s \triangleleft \langle \texttt{con}\ T\ c \rangle & \mapsto \mathsf{CaseBuiltin}_{\mathsf{CEK}}(s, \rho, T, c, (M_1, \dots, M_n))
\end{array}
$$

(a) CEK machine transitions for Plutus Core

$$
\mathsf{EvalBuiltin}_{\mathsf{CEK}}(s, b, [V_1, \dots, V_n]) = \begin{cases} \blacklozenge & \text{if } \mathsf{EvalBuiltin}\,(b, [V_1, \dots, V_n]) = \times \\ s \triangleleft \mathsf{EvalBuiltin}\,(b, [V_1, \dots, V_n]) & \text{otherwise} \end{cases}
$$

(b) Evaluation of built-in functions

Figure 2.10: A CEK machine for Plutus Core

$$\mathsf{CaseBuiltin}_{\mathsf{CEK}}(s,\rho,T,c,(M_1,\dots,M_n)) = \begin{cases} [_\ \{\!|v_1|\!\}] \cdots\cdots [_\ \{\!|v_m|\!\}] \cdot s;\rho \rhd M_i & \text{if } \begin{cases} n \le \mathsf{maxBranch}(T) \\ \mathsf{match}_T(c) = (i,(v_1,\dots,v_m)) \\ i \le n \end{cases} \\ \blacklozenge & \text{otherwise.} \end{cases}$$

(c) Casing on built-in constants

Figure 2.10: A CEK machine for Plutus Core (continued)

Optimisations. The machine presented above is suboptimal in a number of ways. For example, when casing on a constant of a built-in type one frame is pushed onto the stack for each of the values $\{\!|V_1|\!\},\dots,\{\!|V_m|\!\}$ and these then have to be popped off and processed one by one. It would be more efficient to have a single frame which contains all of the values, and in fact this optimisation is implemented in Cardano's production CEK machine.[†] However we have chosen to keep the evaluator in the specification minimal because optimisations are typically implementation-dependent and may or may not be sensible depending on the implementation language and implementation strategy and we do not wish to impose unsuitable optimisations on other implementers. Using an un-optimised machine also simplifies specification and formalisation; optimisations can then be proved to be correct in the sense that it should be possible to translate constructs in an optimised machine into constructs in the basic machine and show that the translation respects the semantics of the language.

2.4.1 Converting CEK evaluation results into Plutus Core terms

The purpose of the CEK machine is to evaluate Plutus Core terms, but in the definition in Figure 2.10 it does not return a Plutus Core term; instead the machine can halt in two different ways:

- The machine can halt in the state $\square\, V$ for some CEK value V.
- The machine can halt in the state $\blacklozenge$.

To get a complete evaluation strategy for Plutus Core we must convert these states into Plutus Core terms. The term corresponding to $\blacklozenge$ is `(error)`, and to obtain a term from $\square\, V$ we perform a process which we refer to as *discharging* the CEK value V (also known as *unloading*: see [48, pp. 129–130], [36, pp. 71ff]). This process substitutes bindings in environments for variables occurring in the value V to obtain a term $\mathcal{U}(V)$: see Figure 2.11a. Since environments contain bindings $x \mapsto W$ of variables to further CEK values, we have to recursively discharge those bindings first before substituting: see Figure 2.11b, which defines an operation $@\rho$ which does this. As before $[N/x]M$ denotes the usual (capture-avoiding) process of substituting the term N for all unbound occurrences of the variable x in the term M. Note that in Figure 2.11b we substitute the rightmost (ie, the most recent) bindings in the environment first.

[†]This is implemented in Haskell: see https://github.com/IntersectMBO/plutus, and specifically https://github.com/IntersectMBO/plutus/tree/master/plutus-core/untyped-plutus-core/src/UntypedPlutusCore/Evaluation/Machine.

$$
\begin{aligned}
\mathcal{U}(\langle \texttt{con}\ T\ c \rangle) &= (\texttt{con}\ T\ c) \\
\mathcal{U}(\langle \texttt{delay}\ M\ \rho \rangle) &= (\texttt{delay}\ M)\,@\,\rho \\
\mathcal{U}(\langle \texttt{lam}\ x\ M\ \rho \rangle) &= (\texttt{lam}\ x\ M)\,@\,\rho \\
\mathcal{U}(\langle \texttt{constr}\ i\ \overline{V} \rangle) &= (\texttt{constr}\ i\ \overline{\mathcal{U}(V)}) \\
\mathcal{U}(\langle \texttt{builtin}\ b\ V_1 V_2 \dots V_k\ \eta \rangle) &= [\dots\ [[(\texttt{builtin}\ b)\ (\mathcal{U}(V_1))]\ (\mathcal{U}(V_2))] \dots (\mathcal{U}(V_k))]
\end{aligned}
$$

(a) Discharging CEK values

$$
M\,@\,\rho = [(\mathcal{U}(V_1))/x_1] \cdots [(\mathcal{U}(V_n))/x_n]M \quad \text{if } \rho = [x_1 \mapsto V_1, \dots, x_n \mapsto V_n]
$$

(b) Iterated substitution/discharging

Figure 2.11: Discharging CEK values to obtain Plutus Core terms

We can prove that if we evaluate a closed Plutus Core term in the CEK machine and then convert the result back to a term using the above procedure then we get the result that we should get according to the semantics in Figure 2.7.

2.5 Cost accounting for Untyped Plutus Core

To follow.

Chapter 3

Typed Plutus Core

To follow.

Chapter 4

Plutus Core on Cardano

4.1 Protocol versions

The Cardano blockchain controls the introduction of features through the use of *protocol versions*, a field in the protocol parameters. The major protocol version is used to indicate when forwards-incompatible changes (i.e. those that allow blocks that were not previously allowed) are made to the rules of the chain. This is a hard fork of the chain.

The protocol version is part of the history of the chain, as are all protocol parameters. That means that all blocks are associated with the protocol version from when they were created, so that they can be interpreted correctly.

In summary, conditioning on the protocol version is the main way in which we can introduce changes in behaviour.

Table 4.1 lists the protocol versions that are relevant to the use of Plutus Core on Cardano. This table is based on the `features-table.md` document attached to CIP-0059 [16]. For brevity, in the remainder of this document we will sometimes abbreviate "Protocol Version" to PV and omit the minor version number, so for example PV9 means Protocol Version 9.0.

The table includes an entry for Protocol Version 12.0, which will initiate Cardano's Dijkstra era, and this document uses the name "Dijkstra HF" for the corresponding hard fork. The date of this hard fork is unknown at the time of writing, and its actual name will probably be different.

Protocol version	Hard fork	Epoch	Slot	Date
5.0	Alonzo	290	39916975	September 2021
7.0	Vasil	365	72316896	September 2022
8.0	Valentine	394	84844885	February 2023
9.0	Chang	507	133660855	September 2024
10.0	Plomin	537	146620809	January 2025
11.0	van Rossem	644	192845302	July 2026
12.0	Dijkstra HF			TBD

Table 4.1: Protocol versions

4.2 Ledger languages

The Cardano ledger uses Plutus Core as the programming language for *scripts*. The ledger in fact supports multiple different interpretations for scripts, and so each script is tagged with a *ledger language* that tells the ledger how to interpret it. Since the ledger must always be able to evaluate old scripts and get the same answer, the ledger language, in combination with the protocol version and the Plutus Core language version, must pin down everything about how the script is evaluated, including:

1. How to interpret the script itself (e.g. as a Plutus Core program, what versions of the Plutus Core language are allowable)
2. Other configuration the script may need in order to run (e.g. the set of builtin types and functions and their interpretations, cost model parameters)
3. How the script is invoked (e.g. after having certain arguments passed to it)

There are currently three "Plutus" ledger languages (ie, ledger languages whose underlying programming language is Plutus Core) in use on Cardano:*,

1. `PlutusV1`
2. `PlutusV2`
3. `PlutusV3`

Table 4.2 shows when each Plutus ledger language was introduced. Ledger languages remain available permanently after they have been introduced. A fourth ledger language, `PlutusV4`, will become available at Protocol Version 12.0; this will mostly differ from `PlutusV3` in the API for interacting with the Cardano ledger, which is beyond the scope of this specification (but see [47] for more information).

Protocol version	Ledger language introduced
5.0	`PlutusV1`
7.0	`PlutusV2`
9.0	`PlutusV3`
12.0	`PlutusV4`

Table 4.2: Introduction of Plutus ledger languages

4.2.1 Evolution of Plutus ledger languages and Plutus Core language versions

Plutus Core scripts first became available on Cardano at the Alonzo hard fork (PV5). Later hard forks have introduced a number of "batches" of built-in type and functions (described in Section 4.3 below) and new Plutus Core language versions (see Section 2.1.3). Initially the workings of the Cardano ledger meant that it was not possible to retroactively add new built-in functions or Plutus Core language versions to existing ledger languages after a new ledger language was introduced: for example Batch 3 was added to PlutusV2 at the Valentine hard fork (introducing PV8), but it was not possible to add Batch 3 to PlutusV1 at that time. This restriction was subsequently lifted and at the van Rossem hard fork (PV11) it became possible to use all built-in types and functions and all Plutus Core language versions in all ledger languages. Table 4.3 below shows how the situation has evolved as time progressed (the row for PV12 is provisional at the time

*Note that ledger languages are completely distinct from the point of view of the ledger, the "V1"/"V2" naming is suggestive of the fact that these two ledger languages are related, but in the implementation they are completely independent.

of writing). This is of some relevance for nodes replaying the history of the chain because they must treat scripts which use builtins or Plutus Core versions which were not available at the time when the scripts were originally executed as failing.[†]

Protocol version	`PlutusV1`	`PlutusV2`	`PlutusV3`	`PlutusV4`
5.0 (Alonzo)	1.0.0 Batch 1	—	—	—
7.0 (Vasil)	1.0.0 Batch 1	1.0.0 Batches 1–2	—	—
8.0 (Valentine)	1.0.0 Batch 1	1.0.0 Batches 1–3	—	—
9.0 (Chang)	1.0.0 Batch 1	1.0.0 Batches 1–3	1.0.0, 1.1.0 Batches 1–4	—
10.0 (Plomin)	1.0.0 Batch 1	1.0.0 Batches 1–3, 4b[†]	1.0.0, 1.1.0 Batches 1–5	—
11.0 (van Rossem)	1.0.0, 1.1.0 Batches 1–6	1.0.0, 1.1.0 Batches 1–6	1.0.0, 1.1.0 Batches 1–6	—
12.0 (Dijkstra HF)	1.0.0–1.2.0 Batches 1–7	1.0.0–1.2.0 Batches 1–7	1.0.0–1.2.0 Batches 1–7	1.0.0–1.2.0 Batches 1–7

Table 4.3: Built-in functions and Plutus Core language versions available in each ledger language at each major protocol version

Each entry in the table shows the Plutus Core language versions which are allowed and the built-in batches which are available (see Section 4.3). An entry of '—' means that the ledger language does not exist at that protocol version. The entry for PV12 is provisional at the time of writing. Only protocol versions at which something changes are shown: for example, no relevant changes happened at PV6. At PV10, the Batch 4b built-ins (`integerToByteString` and `byteStringToInteger`) were added to `PlutusV2` as a proof of concept; the remainder of Batch 4 became available in `PlutusV2` at PV11.

4.3 Built-in types and functions

Built-in batches. The built-in types and functions are defined in batches corresponding to how they were added to ledger languages. These batches are described in this section, after some preliminaries.

Built-in semantics variants. In rare cases we can make a mistake or need to change the actual behaviour of a built-in function. To handle this we define a series of built-in semantics variants, which indicate which behaviour should be used. A fix will typically be deployed by defining a new semantics variant, and then using that variant for future ledger languages (but not existing ones, since this is usually a backwards-incompatible change). Table 4.4 shows which semantics variants are used for which ledger languages.

[†]In fact, in PV7 and later, scripts which use an unavailable ledger language or unavailable built-in types or functions should be rejected during deserialisation and should never be recorded on the chain. These are *Phase 1 failures*, as opposed to *Phase 2 failures*, which involve run-time errors such as division by zero or budget overflows, and also attempts to use an unavailable Plutus Core version. Before PV7 scripts were only deserialised when they were executed, so scripts using unavailable features would have caused Phase 2 failures instead. Script evaluations with Phase 2 failures will be recorded in the history of the chain, although examination of the history shows that very few Phase 2 failures have actually occurred (the deterministic nature of script evaluation means that they become apparent if a script is evaluated off-chain prior to submission, so it is unlikely that scripts with Phase 2 failures will be submitted to the chain).

Ledger language	Built-in semantics variant used
`PlutusV1`, `PlutusV2`	Built-in semantics 1
`PlutusV3`, `PlutusV4`	Built-in semantics 2

Table 4.4: Selection of built-in semantics variant

Table 4.5 shows the semantic changes for the different variants. Changes are listed alongside the original definition of the built-in function in its original batch.

Built-in semantics variant	Changes from previous semantics
Built-in semantics 1	None
Built-in semantics 2	`consByteString` (see Note 2 in Section 4.3.1.2)

Table 4.5: Built-in semantics variants

Concrete syntax for built-in types. Recall that in the abstract notation for built-in types introduced in Section 2.2.1, a built-in type is either an *atomic type* such as `integer` or `string` or an application $op(T_1, \ldots, T_n)$ of a *type operator* to a sequence of built-in types. The concrete syntax of built-in types used in textual Plutus Core programs is slightly different in that we use a curried form of application for type operators: a type is given by

$$\begin{array}{lcll} \mathbf{T} & ::= & \textit{atomic-type} & \text{Atomic type} \\ & & (\textit{op}\ \mathbf{T}_1 \ldots \mathbf{T}_{|op|}) & \text{Type application} \end{array}$$

Note that we again require that all type operators are fully applied. We refer to the syntactic objects $\mathbf{T}$ above as *concrete built-in types*. There is an obvious bijection between these and the abstract built-in types used elsewhere in this document, and given an abstract built-in type T we will denote the corresponding concrete built-in type by $\bar{T}$.

Concrete syntax for built-in constants. We provide concrete syntax for constants of most (but not all) built-in types. For a built-in type T which has a concrete syntax we specify a set $\mathbf{C}_T$ of strings (using either regular expressions or a BNF-style grammar), and a constant of type T is then represented in the concrete syntax by an expression of the form `(con` $\bar{T}$ c_T`)` with $c_T \in \mathbf{C}_T$. Each string c_T will have an interpretation as a value of type T (ie, an element of $[\![T]\!]$) and since this will generally be the obvious interpretation we will not always spell out the details.

4.3.1 Batch 1

4.3.1.1 Built-in types and type operators

The first batch of built-in types and type operators is defined in Tables 4.6 and 4.7. We also include concrete syntax for these; the concrete syntax is not strictly part of the language, but may be useful for tools working with Plutus Core.

Type	Denotation	Concrete Syntax
`integer`	$\mathbb{Z}$	`-?[0-9]+`
`bytestring`	$\mathbb{B}^*$, the set of sequences of bytes or 8-bit characters.	`#([0-9A-Fa-f][0-9A-Fa-f])*`
`string`	$\mathbb{U}^*$, the set of sequences of Unicode characters.	See note below
`bool`	{true, false}	`True | False`
`unit`	{()}	`()`
`data`	See below	See below

Table 4.6: Atomic types, Batch 1

Operator *op*	$\|op\|$	Denotation	Concrete Syntax
`list`	1	$[\![\texttt{list}\,(t)]\!] = [\![t]\!]^*$	See below
`pair`	2	$[\![\texttt{pair}\,(t_1, t_2)]\!] = [\![t_1]\!] \times [\![t_2]\!]$	See below

Table 4.7: Type operators, Batch 1

Casing on built-in constants. From Protocol Version 11 onwards, Plutus Core supports casing on some of the built-in types described in Tables 4.6 and 4.7. See Table 4.8 and Section 2.2.1.4 for details. Casing is only available for the types mentioned in Table 4.8, and for all other types T one should assume that $\mathsf{maxBranch}(T) = 0$ and $\mathsf{match}_T(x) = \times$ for all x: the semantics of Plutus Core as defined in Figure 2.7 and implemented in the CEK machine (Figure 2.10) will ensure that this causes `(error)` if an attempt is made to case on a value of such a type during evaluation of a Plutus Core script.

Casing on `data` is only available from Protocol Version 12 onwards, and only in scripts with Plutus Core language version 1.2.0 or later.[‡] In all other circumstances one should assume that $\mathsf{maxBranch}(\texttt{data}) = \infty$ but that $\mathsf{match}_{\texttt{data}}(x) = \times$ for all x, so any attempt to case on a `data` value will cause `(error)`, regardless of the number of branches. Only values of the form $\mathrm{inj}_C(k, l)$ (corresponding to `Constr` k l: see the description of `data` below) can be cased on; in this case the selected branch is applied to the list l, a constant of type `list data`.

[‡]There's some inconsistency with casing on other types such as `integer` since that's permissible in Plutus Core 1.1.0. This is due to an oversight.

Type T	match_T	$\mathsf{maxBranch}(T)$
`unit`	$() \mapsto (1, ())$	1
`boolean`	`false` $\mapsto (1, ())$ `true` $\mapsto (2, ())$	2
`integer`	$k \mapsto (k+1, ())$ $(k \geq 0)$ $k \mapsto \times$ $(k < 0)$	∞
`list` T_1	$[x_1, x_2, \ldots, x_n] \mapsto (1, (x_1, [x_2, \ldots, x_n]))$ $(n \geq 1)$ $[] \mapsto (2, ())$	2
`pair` $T_1\, T_2$	$(a, b) \mapsto (1, (a, b))$	1
`data`	$\mathrm{inj}_C(k, l) \mapsto (k+1, (l))$ $(k \geq 0)$ $\mathrm{inj}_C(k, l) \mapsto \times$ $(k < 0)$ $d \mapsto \times$ (otherwise)	∞

Table 4.8: Matching functions for built-in types

Concrete syntax for strings. Strings are represented as sequences of Unicode characters enclosed in double quotes, and may include standard escape sequences. Surrogate characters in the range `U+D800`–`U+DFFF` are replaced with the Unicode replacement character `U+FFFD`.

Concrete syntax for lists and pairs. A list of type `list`(t) is written as a syntactic list $[c_1, \ldots, c_n]$ where each c_i lies in $\mathbf{C}_t$; a pair of type `pair`(t_1, t_2) is written as a syntactic pair (c_1, c_2) with $c_1 \in \mathbf{C}_{t_1}$ and $c_2 \in \mathbf{C}_{t_2}$. Some valid constant expressions are thus

```
(con (list integer) [11, 22, 33])
(con (pair bool string) (True, "Plutus")).
(con (list (pair bool (list bytestring)))
   [(True, []), (False, [#,#1F]), (True, [#123456, #AB, #ef2804])])
```

The `data` type. We provide a built-in type `data` which permits the encoding of simple data structures for use as arguments to Plutus Core scripts. This type is defined in Haskell as

```
data Data =
   Constr Integer [Data]
   | Map [(Data, Data)]
   | List [Data]
   | I Integer
   | B ByteString
```

In set-theoretic terms the denotation of `data` is defined to be the least fixed point of the endofunctor F on the category of sets given by $F(X) = (\llbracket\texttt{integer}\rrbracket \times X^*) \uplus (X \times X)^* \uplus X^* \uplus \llbracket\texttt{integer}\rrbracket \uplus \llbracket\texttt{bytestring}\rrbracket$, so that

$$\llbracket\texttt{data}\rrbracket = (\llbracket\texttt{integer}\rrbracket \times \llbracket\texttt{data}\rrbracket^*) \uplus (\llbracket\texttt{data}\rrbracket \times \llbracket\texttt{data}\rrbracket)^* \uplus \llbracket\texttt{data}\rrbracket^* \uplus \llbracket\texttt{integer}\rrbracket \uplus \llbracket\texttt{bytestring}\rrbracket.$$

We have injections

$$\begin{aligned} \mathrm{inj}_C &: [\![\texttt{integer}]\!] \times [\![\texttt{data}]\!]^* \to [\![\texttt{data}]\!] \\ \mathrm{inj}_M &: ([\![\texttt{data}]\!] \times [\![\texttt{data}]\!])^* \to [\![\texttt{data}]\!] \\ \mathrm{inj}_L &: [\![\texttt{data}]\!]^* \to [\![\texttt{data}]\!] \\ \mathrm{inj}_I &: [\![\texttt{integer}]\!] \to [\![\texttt{data}]\!] \\ inj_B &: [\![\texttt{bytestring}]\!] \to [\![\texttt{data}]\!] \end{aligned}$$

and projections

$$\begin{aligned} \mathrm{proj}_C &: [\![\texttt{data}]\!] \to ([\![\texttt{integer}]\!] \times [\![\texttt{data}]\!]^*)_\times \\ \mathrm{proj}_M &: [\![\texttt{data}]\!] \to ([\![\texttt{data}]\!] \times [\![\texttt{data}]\!])^*_\times \\ \mathrm{proj}_L &: [\![\texttt{data}]\!] \to [\![\texttt{data}]\!]^*_\times \\ \mathrm{proj}_I &: [\![\texttt{data}]\!] \to [\![\texttt{integer}]\!]_\times \\ \mathrm{proj}_B &: [\![\texttt{data}]\!] \to [\![\texttt{bytestring}]\!]_\times \end{aligned}$$

which extract an object of the relevant type from a `data` object D, returning $\times$ if D does not lie in the expected component of the disjoint union; also there are functions

$$\mathrm{is}_C, \mathrm{is}_M, \mathrm{is}_L, \mathrm{is}_I, \mathrm{is}_B : [\![\texttt{data}]\!] \to [\![\texttt{bool}]\!]$$

which determine whether a `data` value lies in the relevant component.

Note: `Constr` **tag values.** The `Constr` constructor of the `data` type is intended to represent values from algebraic data types (also known as sum types and discriminated unions, among other things; `data` itself is an example of such a type), where `Constr` $i\,[d_1, \ldots, d_n]$ represents a tuple of data items together with a tag i indicating which of a number of alternatives the data belongs to. The definition above allows tags to be any integer value, but because of restrictions in the serialisation format for `data` (see Section B.7) we recommend that in practice **only tags** i **with** $0 \leq i \leq 2^{64} - 1$ **should be used**: deserialisation will fail for `data` items (and programs which include such items) involving tags outside this range.

Note also that `Constr` is unrelated to the `constr` term in Plutus Core itself. Both provide ways of representing structured data, but the former is part of a built-in type whereas the latter is part of the language itself.

Concrete syntax for `data`**.** The concrete syntax for `data` is given by

$$\begin{aligned} c_{\texttt{data}} \quad ::= \quad & (\texttt{Constr}\ c_{\texttt{integer}}\ c_{\texttt{list(data)}}) \\ & (\texttt{Map}\ c_{\texttt{list(pair(data,data))}}) \\ & (\texttt{List}\ c_{\texttt{list(data)}}) \\ & (\texttt{I}\ c_{\texttt{integer}}) \\ & (\texttt{B}\ c_{\texttt{bytestring}}). \end{aligned}$$

We interpret these syntactic constants as elements of $[\![\texttt{data}]\!]$ using the various 'inj' functions defined earlier. Some valid `data` constants are

```
(con data (Constr 1 [(I 2), (B #), (Map [])])
(con data (Map [((I 0), (B #00)), ((I 1), (B #0F))]))
(con data (List [(I 0), (I 1), (B #7FFF), (List []]))
(con data (I -22))
(con data (B #001A)).
```

Note. At the time of writing the syntax accepted by IOG's parser for textual Plutus Core differs slightly from that above in that subobjects of `Constr`, `Map` and `List` objects must *not* be parenthesised: for example one must write `(con data (Constr 1 [I 2, B #,Map []])`. This discrepancy will be resolved in the near future.

4.3.1.2 Built-in functions

The first batch of built-in functions is shown in Table 4.9. The table indicates which functions can fail during execution, and conditions causing failure are specified either in the denotation given in the table or in a relevant note. Recall also that a built-in function will fail if it is given an argument of the wrong type: this is checked in conditions involving the $\sim$ relation and the EvalBuiltin function in Figures 2.7 and 2.10. Note also that some of the functions are #-polymorphic. According to Section 2.2.2.4 we require a denotation for every possible monomorphisation of these; however all of these functions are parametrically polymorphic so to simplify notation we have given a single denotation for each of them with an implicit assumption that it applies at each possible monomorphisation in an obvious way.

Function	Signature	Denotation	Can fail?	Note
`addInteger`	`[integer, integer]` → `integer`	$+$	No	
`subtractInteger`	`[integer, integer]` → `integer`	$-$	No	
`multiplyInteger`	`[integer, integer]` → `integer`	$\times$	No	
`divideInteger`	`[integer, integer]` → `integer`	div	Yes	1
`modInteger`	`[integer, integer]` → `integer`	mod	Yes	1
`quotientInteger`	`[integer, integer]` → `integer`	quot	Yes	1
`remainderInteger`	`[integer, integer]` → `integer`	rem	Yes	1
`equalsInteger`	`[integer, integer]` → `bool`	$=$	No	
`lessThanInteger`	`[integer, integer]` → `bool`	$<$	No	
`lessThanEqualsInteger`	`[integer, integer]` → `bool`	$\leq$	No	
`appendByteString`	`[bytestring, bytestring]` → `bytestring`	$([c_1, \ldots, c_m], [d_1, \ldots, d_n]) \mapsto [c_1, \ldots, c_m, d_1, \ldots, d_n]$	No	
`consByteString` (Variant 1)	`[integer, bytestring]` → `bytestring`	$(c, [c_1, \ldots, c_n]) \mapsto [\mathrm{mod}(c, 256), c_1, \ldots, c_n]$	No	2
`consByteString` (Variant 2)	`[integer, bytestring]` → `bytestring`	$(c, [c_1, \ldots, c_n]) \mapsto \begin{cases} [c, c_1, \ldots, c_n] & \text{if } 0 \leq c \leq 255 \\ \times & \text{otherwise} \end{cases}$	Yes	2

Table 4.9: Built-in functions, Batch 1

Function	Type	Denotation	Can fail?	Note
`sliceByteString`	`[integer, integer, bytestring] → bytestring`	$(s, k, [c_1, \ldots, c_n]) \mapsto [c_{\max(s+1,1)}, \ldots, c_{\min(s+k,n)}]$	No	3
`lengthOfByteString`	`[bytestring] → integer`	$[] \mapsto 0, [c_1, \ldots, c_n] \mapsto n\ (n \geq 1)$	No	
`indexByteString`	`[bytestring, integer] → integer`	$([c_1, \ldots, c_n], i) \mapsto \begin{cases} c_{i+1} & \text{if } 0 \leq i \leq n-1 \\ \times & \text{otherwise} \end{cases}$	Yes	
`equalsByteString`	`[bytestring, bytestring] → bool`	$=$	No	4
`lessThanByteString`	`[bytestring, bytestring] → bool`	$<$	No	4
`lessThanEqualsByteString`	`[bytestring, bytestring] → bool`	$\leq$	No	4
`appendString`	`[string, string] → string`	$([u_1, \ldots, u_m], [v_1, \ldots, v_n]) \mapsto [u_1, \ldots, u_m, v_1, \ldots, v_n]$	No	
`equalsString`	`[string, string] → bool`	$=$	No	
`encodeUtf8`	`[string] → bytestring`	utf8		5
`decodeUtf8`	`[bytestring] → string`	utf8^{-1}	Yes	5
`sha2_256`	`[bytestring] → bytestring`	Hash a `bytestring` using `SHA-256` [31].	No	
`sha3_256`	`[bytestring] → bytestring`	Hash a `bytestring` using `SHA3-256` [34].	No	
`blake2b_256`	`[bytestring] → bytestring`	Hash a `bytestring` using `Blake2b-256` [50].	No	
`verifyEd25519Signature`	`[bytestring, bytestring, bytestring] → bool`	Verify an Ed25519 digital signature.	Yes	6, 7
`ifThenElse`	$[\forall *, \texttt{bool}, *, *] \to *$	$(\text{true}, t_1, t_2) \mapsto t_1\ (\text{false}, t_1, t_2) \mapsto t_2$	No	
`chooseUnit`	$[\forall *, \texttt{unit}, *] \to *$	$((), t) \mapsto t$	No	
`trace`	$[\forall *, \texttt{string}, *] \to *$	$(s, t) \mapsto t$	No	8
`fstPair`	$[\forall a_\#, \forall b_\#, \texttt{pair}\,(a_\#, b_\#)] \to a_\#$	$(x, y) \mapsto x$	No	
`sndPair`	$[\forall a_\#, \forall b_\#, \texttt{pair}\,(a_\#, b_\#)] \to b_\#$	$(x, y) \mapsto y$	No	
`chooseList`	$[\forall a_\#, \forall *, \texttt{list}\,(a_\#), *, *] \to *$	$([], t_1, t_2) \mapsto t_1$, $([x_1, \ldots, x_n], t_1, t_2) \mapsto t_2\ (n \geq 1)$.	No	9
`mkCons`	$[\forall a_\#, a_\#, \texttt{list}\,(a_\#)] \to \texttt{list}\,(a_\#)$	$(x, [x_1, \ldots, x_n]) \mapsto [x, x_1, \ldots, x_n]$	No	9
`headList`	$[\forall a_\#, \texttt{list}\,(a_\#)] \to a_\#$	$[] \mapsto \times, [x_1, x_2, \ldots, x_n] \mapsto x_1$	Yes	9
`tailList`	$[\forall a_\#, \texttt{list}\,(a_\#)] \to \texttt{list}\,(a_\#)$	$[] \mapsto \times, [x_1, x_2, \ldots, x_n] \mapsto [x_2, \ldots, x_n]$	Yes	9
`nullList`	$[\forall a_\#, \texttt{list}\,(a_\#)] \to \texttt{bool}$	$[] \mapsto \text{true}, [x_1, \ldots, x_n] \mapsto \text{false}$	No	9

Table 4.9: Built-in functions, Batch 1

Function	Type	Denotation	Can fail?	Note
`chooseData`	$[\forall *, \texttt{data}, *, *, *, *, *] \to *$	$(d, t_C, t_M, t_L, t_I, t_B) \mapsto \begin{cases} t_C & \text{if } \text{is}_C(d) \\ t_M & \text{if } \text{is}_M(d) \\ t_L & \text{if } \text{is}_L(d) \\ t_I & \text{if } \text{is}_I(d) \\ t_B & \text{if } \text{is}_B(d) \end{cases}$	No	
`constrData`	$[\texttt{integer}, \texttt{list}\,(\texttt{data})] \to \texttt{data}$	inj_C	No	
`mapData`	$[\texttt{list}\,(\texttt{pair}\,(\texttt{data}, \texttt{data}))] \to \texttt{data}$	inj_M	No	
`listData`	$[\texttt{list}\,(\texttt{data})] \to \texttt{data}$	inj_L	No	
`iData`	$[\texttt{integer}] \to \texttt{data}$	inj_I	No	
`bData`	$[\texttt{bytestring}] \to \texttt{data}$	inj_B	No	
`unConstrData`	$[\texttt{data}] \to \texttt{pair}\,(\texttt{integer}, \texttt{list}\,(\texttt{data}))$	proj_C	Yes	
`unMapData`	$[\texttt{data}] \to \texttt{list}\,(\texttt{pair}\,(\texttt{data}, \texttt{data}))$	proj_M	Yes	
`unListData`	$[\texttt{data}] \to \texttt{list}\,(\texttt{data})$	proj_L	Yes	
`unIData`	$[\texttt{data}] \to \texttt{integer}$	proj_I	Yes	
`unBData`	$[\texttt{data}] \to \texttt{bytestring}$	proj_B	Yes	
`equalsData`	$[\texttt{data}, \texttt{data}] \to \texttt{bool}$	$=$	No	
`mkPairData`	$[\texttt{data}, \texttt{data}] \to \texttt{pair}\,(\texttt{data}, \texttt{data})$	$(x, y) \mapsto (x, y)$	No	
`mkNilData`	$[\texttt{unit}] \to \texttt{list}\,(\texttt{data})$	$() \mapsto []$	No	
`mkNilPairData`	$[\texttt{unit}] \to \texttt{list}\,(\texttt{pair}\,(\texttt{data}, \texttt{data}))$	$() \mapsto []$	No	

Table 4.9: Built-in functions, Batch 1 (continued)

Note 1. Integer division functions. We provide four integer division functions: `divideInteger`, `modInteger`, `quotientInteger`, and `remainderInteger`, whose denotations are mathematical functions div, mod, quot, and rem which are modelled on the corresponding Haskell operations. Each of these takes two arguments and will fail (returning $\times$) if the second one is zero. For all $a, b \in \mathbb{Z}$ with $b \neq 0$ we have

$$\text{div}(a, b) \times b + \text{mod}(a, b) = a$$

$$|\,\text{mod}(a, b)| < |b|$$

and

$$\text{quot}(a, b) \times b + \text{rem}(a, b) = a$$

$$|\,\text{rem}(a, b)| < |b|.$$

The div and mod functions form a pair, as do quot and rem; div should not be used in combination with rem, nor should quot be used with mod.

For positive divisors b, div truncates downwards and mod always returns a non-negative result ($0 \leq \mathsf{mod}(a, b) \leq b - 1$). The quot function truncates towards zero. Table 4.10 shows how the signs of the outputs of the division functions depend on the signs of the inputs; + means ≥ 0 and − means ≤ 0, but recall that for $b = 0$ all of these functions return the error value ×.

a	b	div	mod	quot	rem
+	+	+	+	+	+
−	+	−	+	−	−
+	−	−	−	−	+
−	−	+	−	+	−

Table 4.10: Behaviour of integer division functions

Note 2. The `consByteString` function. In built-in semantics variant 1, the first argument of `consByteString` is an arbitrary integer which will be reduced modulo 256 before being prepended to the second argument. In built-in semantics variant 2 we require that the first argument lies between 0 and 255 (inclusive): in any other case an error will occur.

Note 3. The `sliceByteString` function. The application `[[(builtin sliceByteString) (con integer s)] (con integer k)] (con bytestring b)]` returns the substring of b of length k starting at position s; indexing is zero-based, so a call with $s = 0$ returns a substring starting with the first element of b, $s = 1$ returns a substring starting with the second, and so on. This function always succeeds, even if the arguments are out of range: if $b = [c_1, \ldots, c_n]$ then the application above returns the substring $[c_i, \ldots, c_j]$ where $i = \max(s + 1, 1)$ and $j = \min(s + k, n)$; if $j < i$ then the empty string is returned.

Note 4. Comparisons of bytestrings. Bytestrings are ordered lexicographically in the usual way. If we have $a = [a_1, \ldots, a_m]$ and $b = [b_1, \ldots, b_n]$ then (recalling that if $m = 0$ then $a = []$, and similarly for b),

- $a = b$ if and only if $m = n$ and $a_i = b_i$ for $1 \leq i \leq m$.
- $a \leq b$ if and only if one of the following holds:
  - $a = []$
  - $m, n > 0$ and $a_1 < b_1$
  - $m, n > 0$ and $a_1 = b_1$ and $[a_2, \ldots, a_m] \leq [b_2, \ldots, b_n]$.
- $a < b$ if and only if $a \leq b$ and $a \neq b$.

For example, `#23456789` < `#24` and `#2345` < `#234500`. The empty bytestring is equal only to itself and is strictly less than all other bytestrings.

Note 5. Encoding and decoding bytestrings. The `encodeUtf8` and `decodeUtf8` functions convert between the `string` type and the `bytestring` type. We have defined $[\![\texttt{string}]\!]$ to consist of sequences of Unicode characters without specifying any particular character representation, whereas $[\![\texttt{bytestring}]\!]$ consists of sequences of 8-bit bytes. We define the denotation of `encodeUtf8` to be the function

$$\mathsf{utf8} : \mathbb{U}^* \to \mathbb{B}^*$$

which converts sequences of Unicode characters to sequences of bytes using the well-known UTF-8 character encoding [54, Definition D92]. The denotation of `decodeUtf8` is the partial inverse function

$$\mathsf{utf8}^{-1} : \mathbb{B}^* \to \mathbb{U}^*_{\mathsf{x}}.$$

UTF-8 encodes Unicode characters encoded using between one and four bytes: thus in general neither function will preserve the length of an object. Moreover, not all sequences of bytes are valid representations of Unicode characters, and `decodeUtf8` will fail if it receives an invalid input (but `encodeUtf8` will always succeed).

Note 6. Digital signature verification functions. We use a uniform interface for digital signature verification algorithms. A digital signature verification function takes three bytestring arguments (in the given order):

- a public key *vk* (in this context *vk* is also known as a *verification key*)
- a message *m*
- a signature *s*.

A signature verification function may require one or more arguments to be well-formed in some sense (in particular an argument may need to be of a specified length), and in this case the function will fail (returning ×) if any argument is malformed. If all of the arguments are well-formed then the verification function returns `true` if the private key corresponding to *vk* was used to sign the message *m* to produce *s*, otherwise it returns `false`.

Note 7. Ed25519 signature verification. The `verifyEd25519Signature` function performs cryptographic signature verification using the Ed25519 scheme [5, 42], and conforms to the interface described in Note 6. The arguments must have the following sizes:

- *vk*: 32 bytes
- *m*: unrestricted
- *s*: 64 bytes.

Note 8. The `trace` function. An application `[(builtin trace) s v]` (*s* a `string`, *v* any Plutus Core value) returns *v*. We do not specify the semantics any further. An implementation may choose to discard *s* or to perform some side-effect such as writing it to a terminal or log file.

Note 9. Semantics of lists. The mathematical denotation of lists is as finite sequences of elements. We expect that in practice they will be implemented in such a way (for example as linked lists) that `chooseList`, `mkCons`, `headList`, `tailList`, and `nullList` are all constant-time operations.

4.3.2 Batch 2

4.3.2.1 Built-in functions

The second batch of built-in functions is defined in Table 4.11. See [14].

Function	Signature	Denotation	Can fail?	Note
`serialiseData`	`[data]` → `bytestring`	$\mathcal{E}_{\texttt{data}}$	No	1

Table 4.11: Built-in functions, Batch 2

Note 1. Serialising `data` objects. The `serialiseData` function takes a `data` object and converts it into a bytestring using a CBOR encoding. A full specification of the encoding (including the definition of $\mathcal{E}_{\texttt{data}}$) is provided in Appendix B.

4.3.3 Batch 3

4.3.3.1 Built-in functions

The third batch of built-in functions is defined in Table 4.12. See [15].

Function	Signature	Denotation	Can fail?	Note
`verifyEcdsaSecp256k1Signature`	`[bytestring, bytestring, bytestring]` → `bool`	Verify an SECP-256k1 ECDSA signature	Yes	1
`verifySchnorrSecp256k1Signature`	`[bytestring, bytestring, bytestring]` → `bool`	Verify an SECP-256k1 Schnorr signature	Yes	2

Table 4.12: Built-in functions, Batch 3

Note 1. Secp256k1 ECDSA Signature verification. The `verifyEcdsaSecp256k1Signature` function performs elliptic curve digital signature verification [1, 2, 41] over the `secp256k1` curve [12, §2.4.1] and conforms to the interface described in Note 6 of Section 4.3.1.2. The arguments must have the following sizes:

- vk: 33 bytes
- m: 32 bytes
- s: 64 bytes.

The public key vk is expected to be in the 33-byte compressed form described in [7]. It is expected that the inputs to the function are well formed, and if they are not then an error will occur. For example, the signature s encodes a pair (R, S) of 32-byte bytestrings which are big-endian encodings of integers, and both of these integers must lie between 0 and $|G| - 1$ (inclusive), where G is the relevant subgroup of the `secp256k1` curve; it is also required that the last 32 bytes of the verification key are the big-endian encoding of the x-coordinate of a point in G.

Moreover, the ECDSA scheme admits two distinct valid signatures for a given message and private key, and we follow the restriction imposed by Bitcoin (see [45], `LOW_S`) and **only accept the smaller signature**; `verifyEcdsaSecp256k1Signature` will return `false` if the larger one is supplied.

Note 2. Secp256k1 Schnorr Signature verification. The `verifySchnorrSecp256k1Signature` function performs verification of Schnorr signatures [52, 44] over the `secp256k1` curve and conforms to the interface described in Note 6 of Section 4.3.1.2. The arguments are expected to be of the forms specified in BIP-340 [44] and thus should have the following sizes:

- *vk*: 32 bytes
- *m*: unrestricted
- *s*: 64 bytes.

Any deviation from the conditions of BIP-340 [44] will cause an error.

4.3.4 Batch 4

The fourth batch of built-in types and functions adds support for

- The `Blake2b-224` and `Keccak-256` hash functions (see [17]).
- Conversion functions from integers to bytestrings and vice-versa (see [19]).
- BLS12-381 elliptic curve pairing operations (see [28], [11], [51, 4.2.1], [46]). For clarity these are described separately in Sections 4.3.4.2 and 4.3.4.3.

4.3.4.1 Miscellaneous built-in functions

Function	Signature	Denotation	Can fail?	Note
`blake2b_224`	`[bytestring]` → `bytestring`	Hash a `bytestring` using `Blake2b-224` [50]	No	
`keccak_256`	`[bytestring]` → `bytestring`	Hash a `bytestring` using `Keccak-256` [6]	No	
`integerToByteString`	`[bool, integer, integer]` → `bytestring`	$(e, w, n) \mapsto \begin{cases} \mathsf{itobs}_{\mathsf{LE}}(w,n) & \text{if } e = \mathsf{false} \\ \mathsf{itobs}_{\mathsf{BE}}(w,n) & \text{if } e = \mathsf{true} \end{cases}$	Yes	1
`byteStringToInteger`	`[bool, bytestring]` → `integer`	$(e, [c_0, \ldots, c_{N-1}]) \mapsto \begin{cases} \sum_{i=0}^{N-1} c_i 256^i & \text{if } e = \mathsf{false} \\ \sum_{i=0}^{N-1} c_i 256^{N-1-i} & \text{if } e = \mathsf{true} \end{cases}$	No	2

Table 4.13: Batch 4: miscellaneous built-in functions

Note 1. Integer to bytestring conversion. The `integerToByteString` function converts non-negative integers to bytestrings. It takes three arguments:

- A boolean endianness flag *e*.
- An integer width argument *w*: it is required that $0 \leq w \leq 8192$.
- The integer *n* to be converted: it is required that $0 \leq n < 256^{8192} = 2^{65536}$.

The conversion is little-endian (`LE`) if e is `(con bool False)` and big-endian (`BE`) if e is `(con bool True)`. If the width w is zero then the output is a bytestring which is just large enough to hold the converted integer. If $w > 0$ then the output is exactly w bytes long, and it is an error if n does not fit into a bytestring of that size; if necessary, the output is padded with `0x00` bytes (on the right in the little-endian case and the left in the big-endian case) to make it the correct length. For example, the five-byte little-endian representation of the integer `0x123456` is the bytestring `[0x56, 0x34, 0x12, 0x00, 0x00]` and the five-byte big-endian representation is `[0x00, 0x00, 0x12, 0x34, 0x56]`. In all cases an error occurs if w or n lies outside the expected range, and in particular if n is negative.

The precise semantics of `integerToByteString` are given by the functions $\mathsf{itobs}_{\mathsf{LE}} : \mathbb{Z} \times \mathbb{Z} \to \mathbb{B}^*_\times$ and $\mathsf{itobs}_{\mathsf{BE}} : \mathbb{Z} \times \mathbb{Z} \to \mathbb{B}^*_\times$. Firstly we deal with out-of-range cases and the case $n = 0$:

$$\mathsf{itobs}_{\mathsf{LE}}(w, n) = \mathsf{itobs}_{\mathsf{BE}}(w, n) = \begin{cases} \times & \text{if } n < 0 \text{ or } n \geq 2^{65536} \\ \times & \text{if } w < 0 \text{ or } w > 8192 \\ [] & \text{if } n = 0 \text{ and } 0 \leq w \leq 8192 \end{cases}$$

Now assume that none of the conditions above hold, so $0 < n < 2^{65536}$ and $0 \leq w \leq 8192$. Since $n > 0$ it has a unique base-256 expansion of the form $n = \sum_{i=0}^{N-1} a_i 256^i$ with $N \geq 1$, $a_i \in \mathbb{B}$ for $0 \leq i \leq N-1$ and $a_{N-1} \neq 0$. We then have

$$\mathsf{itobs}_{\mathsf{LE}}(w, n) = \begin{cases} [a_0, \ldots, a_{N-1}] & \text{if } w = 0 \\ [b_0, \ldots, b_{w-1}] & \text{if } w > 0 \text{ and } N \leq w, \text{ where } b_i = \begin{cases} a_i & \text{if } i \leq N-1 \\ 0 & \text{if } i \geq N \end{cases} \\ \times & \text{if } w > 0 \text{ and } N > w \end{cases}$$

and

$$\mathsf{itobs}_{\mathsf{BE}}(w, n) = \begin{cases} [a_{N-1}, \ldots, a_0] & \text{if } w = 0 \\ [b_0, \ldots, b_{w-1}] & \text{if } w > 0 \text{ and } N \leq w, \text{ where } b_i = \begin{cases} 0 & \text{if } i \leq w-1-N \\ a_{w-1-i} & \text{if } i \geq w-N \end{cases} \\ \times & \text{if } w > 0 \text{ and } N > w. \end{cases}$$

Note 2. Bytestring to integer conversion. The `byteStringToInteger` function converts bytestrings to non-negative integers. It takes two arguments:

- A boolean endianness flag e.
- The bytestring s to be converted.

The conversion is little-endian if e is `(con bool False)` and big-endian if e is `(con bool True)`. In both cases the empty bytestring is converted to the integer 0. All bytestrings are legal inputs and there is no limitation on the size of s.

4.3.4.2 BLS12-381 built-in types

Supporting the BLS12-381 operations involves adding three new types and seventeen new built-in functions. The description of the semantics of these types and functions is quite complex and requires a considerable amount of notation, most of which is used only in Sections 4.3.4.2 and 4.3.4.3.

Table 4.14 describes three new built-in types.

Type	Denotation	Concrete Syntax
`bls12_381_G1_element`	G_1	`0x[0-9A-Fa-f]{96}` (see Note 6)
`bls12_381_G2_element`	G_2	`0x[0-9A-Fa-f]{192}` (see Note 6)
`bls12_381_mlresult`	H	None (see Note 6)

Table 4.14: Atomic types, Batch 4

Here G_1 and G_2 are both additive cyclic groups of prime order r, where

$$r = \texttt{0x73eda753299d7d483339d80809a1d80553bda402fffe5bfeffffffff00000001}.$$

The fields $\mathbb{F}_q$ and $\mathbb{F}_{q^2}$. To define the groups G_1 and G_2 we need the finite field $\mathbb{F}_q$ where

$$q = \texttt{0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf}\\ \texttt{6730d2a0f6b0f6241eabfffeb153ffffb9feffffffffaaab}$$

which is a 381-bit prime. The field $\mathbb{F}_q$ is isomorphic to $\mathbb{Z}_q$, the ring of integers modulo q, and hence there is a natural epimorphism from $\mathbb{Z}$ to $\mathbb{F}_q$ which we denote by $n \mapsto \bar{n}$. Given $x \in \mathbb{F}_q$, we denote by $\tilde{x}$ the smallest non-negative integer n with $\bar{n} = x$. We sometimes use literal integers to represent elements of $\mathbb{F}_q$ in the obvious way.

We also make use of the field $\mathbb{F}_{q^2} = \mathbb{F}_q[X]/(X^2+1)$; we may regard $\mathbb{F}_{q^2}$ as the set $\{a+bu : a, b \in \mathbb{F}_q\}$ where $u^2 = -1$.

It is convenient to say that an element a of $\mathbb{F}_q$ is *larger* than another element b (and b is *smaller* than a) if $\tilde{a} > \tilde{b}$ in $\mathbb{Z}$. We extend this terminology to $\mathbb{F}_{q^2}$ by saying that $a + bu$ is larger than $c + du$ if either b is larger than d or $b = d$ and a is larger than c.

The groups G_1 and G_2. There are elliptic curves E_1 defined over $\mathbb{F}_q$:

$$E_1 : Y^2 = X^3 + 4$$

and E_2 defined over $\mathbb{F}_{q^2}$:

$$E_2 : Y^2 = X^3 + 4(u+1).$$

$E_1(\mathbb{F}_q)$ and $E_2(\mathbb{F}_{q^2})$ are abelian groups under the usual elliptic curve addition operations as described in [53, III.2] or [29, 2.1]. G_1 is a subgroup of $E_1(\mathbb{F}_q)$ and G_2 is a subgroup of $E_2(\mathbb{F}_{q^2})$; explicit generators for G_1 and G_2 are given in [51, 4.2.1]. We denote the identity element (the point at infinity) in G_1 by $\mathcal{O}_{G_1}$ and that in G_2 by $\mathcal{O}_{G_2}$. Given an integer n and a group element a in G_1 or G_2, the scalar multiple na is defined as usual to be $a + \cdots + a$ (n times) if $n > 0$ and $-a + \cdots + -a$ ($-n$ times) if $n < 0$; $0a$ is the identity element of the group.

The `bls12_381_MlResult` type. Values of the `bls12_381_MlResult` type are completely opaque and can only be obtained as a result of `bls12_381_millerLoop` or by multiplying two existing elements of type `bls12_381_MlResult`. We provide neither a serialisation format nor a concrete syntax for values of this type: they exist only ephemerally during computation. We do not specify H, the denotation of `bls12_381_mlresult`, precisely, but it must be a multiplicative abelian group. See Note 7 for more on this.

4.3.4.3 BLS12-381 built-in functions

Function	Signature	Denotation	Can fail?	Note
`bls12_381_G1_add`	[`bls12_381_G1_element`, `bls12_381_G1_element`] → `bls12_381_G1_element`	$(a, b) \mapsto a + b$	No	
`bls12_381_G1_neg`	[`bls12_381_G1_element`] → `bls12_381_G1_element`	$a \mapsto -a$	No	
`bls12_381_G1_scalarMul`	[`integer`, `bls12_381_G1_element`] → `bls12_381_G1_element`	$(n, a) \mapsto na$	No	
`bls12_381_G1_equal`	[`bls12_381_G1_element`, `bls12_381_G1_element`] → `bool`	$=$	No	
`bls12_381_G1_hashToGroup`	[`bytestring`, `bytestring`] → `bls12_381_G1_element`	hash_{G_1}	Yes	3
`bls12_381_G1_compress`	[`bls12_381_G1_element`] → `bytestring`	$\mathsf{compress}_{G_1}$	No	4
`bls12_381_G1_uncompress`	[`bytestring`] → `bls12_381_G1_element`	$\mathsf{uncompress}_{G_1}$	Yes	5
`bls12_381_G2_add`	[`bls12_381_G2_element`, `bls12_381_G2_element`] → `bls12_381_G2_element`	$(a, b) \mapsto a + b$	No	
`bls12_381_G2_neg`	[`bls12_381_G2_element`] → `bls12_381_G2_element`	$a \mapsto -a$	No	
`bls12_381_G2_scalarMul`	[`integer`, `bls12_381_G2_element`] → `bls12_381_G2_element`	$(n, a) \mapsto na$	No	
`bls12_381_G2_equal`	[`bls12_381_G2_element`, `bls12_381_G2_element`] → `bool`	$=$	No	
`bls12_381_G2_hashToGroup`	[`bytestring`, `bytestring`] → `bls12_381_G2_element`	hash_{G_2}	Yes	3
`bls12_381_G2_compress`	[`bls12_381_G2_element`] → `bytestring`	$\mathsf{compress}_{G_2}$	No	4
`bls12_381_G2_uncompress`	[`bytestring`] → `bls12_381_G2_element`	$\mathsf{uncompress}_{G_2}$	Yes	5
`bls12_381_millerLoop`	[`bls12_381_G1_element`, `bls12_381_G2_element`] → `bls12_381_mlresult`	e	No	7
`bls12_381_mulMlResult`	[`bls12_381_mlresult`, `bls12_381_mlresult`] → `bls12_381_mlresult`	$(a, b) \mapsto ab$	No	7
`bls12_381_finalVerify`	[`bls12_381_mlresult`, `bls12_381_mlresult`] → `bool`	ϕ	No	7

Table 4.15: Batch 4: BLS12-381 built-in functions (continued)

Note 3. Hashing into G_1 and G_2. The denotations hash_{G_1} and hash_{G_2} of `bls12_381_G1_hashToGroup` and `bls12_381_G2_hashToGroup` both take an arbitrary bytestring b (the *message*) and a (possibly empty) bytestring of length at most 255 known as a *domain separation tag* (DST) [35, 2.2.5] and hash them to obtain a point in G_1 or G_2 respectively. The details of the hashing process are described in [35] (see specifically Section 8.8), except that **we do not support DSTs of length greater than 255**: an attempt to use a longer DST directly will cause an error. If a longer DST is required then it should be hashed to obtain a short DST as described in [35, 5.3.3], and then this should be supplied as the second argument to the appropriate `hashToGroup` function.

Note 4. Compression for elements of G_1 and G_2. Points in G_1 and G_2 are encoded as bytestrings in a "compressed" format where only the x-coordinate of a point is encoded and some metadata is used to indicate which of two possible y-coordinates the point has. The encoding format is based on the Zcash encoding for BLS12-381 points: see [57] or [46, "Serialization"] or [51, Appendices C and D]. In detail,

- Given an element x of $\mathbb{F}_q$, $\tilde{x}$ can be written as a 381-bit binary number: $\tilde{x} = \sum_{i=0}^{380} b_i 2^i$ with $b_i \in \{0, 1\}$. We define $\mathsf{bits}(x)$ to be the bitstring $b_{380} \cdots b_0$.
- A non-identity element of G_1 can be written in the form (x, y) with $x, y \in \mathbb{F}_q$. Not every element x of $\mathbb{F}_q$ is the x-coordinate of a point in G_1, but those which are in fact occur as the x-coordinate of two distinct points in G_1 whose y-coordinates are the negatives of each other. A similar statement is true for $\mathbb{F}_{q^2}$ and G_2. In both cases we denote the smaller of the possible y-coordinates by $y_{\min}(x)$ and the larger by $y_{\max}(x)$.
- For $(x, y) \in G_1 \backslash \mathcal{O}_{G_1}$ we define

$$\mathsf{compress}_{G_1}(x, y) = \begin{cases} 100 \cdot \mathsf{bits}(x) & \text{if } y = y_{\min}(x) \\ 101 \cdot \mathsf{bits}(x) & \text{if } y = y_{\max}(x) \end{cases}$$

- We encode the identity element of G_1 using

$$\mathsf{compress}_{G_1}(\mathcal{O}_{G_1}) = 110 \cdot 0^{381},$$

  where 0^{381} denotes a string of 381 0 bits.

Thus in all cases the encoding of an element of G_1 requires exactly 384 bits, or 48 bytes.

- Similarly, every non-identity element of G_2 can be written in the form (x, y) with $x, y \in \mathbb{F}_{q^2}$. We define

$$\mathsf{compress}_{G_2}(a + bu, y) = \begin{cases} 100 \cdot \mathsf{bits}(b) \cdot 000 \cdot \mathsf{bits}(a) & \text{if } y = y_{\min}(a + bu) \\ 101 \cdot \mathsf{bits}(b) \cdot 000 \cdot \mathsf{bits}(a) & \text{if } y = y_{\max}(a + bu) \end{cases}$$

- The identity element of G_2 is encoded using

$$\mathsf{compress}_{G_2}(\mathcal{O}_{G_2}) = 110 \cdot 0^{765}.$$

The encoding of an element of G_2 requires exactly 768 bits, or 96 bytes.

Note that in all cases the most significant bit of a compressed point is 1. In the Zcash serialisation scheme this indicates that the point is compressed; Zcash also supports a serialisation format where both

the x- and y-coordinates of a point are encoded, and in that case the leading bit of the encoded point is 0. We do not support this format.

Note 5. Uncompression for elements of G_1 and G_2. There are two (partial) "uncompression" functions $\mathsf{uncompress}_{G_1}$ and $\mathsf{uncompress}_{G_2}$ which convert bytestrings into group elements; these are obtained by inverting the process described in Note 4.

Uncompression for G_1 elements. Given a bytestring b, it is checked that b contains exactly 48 bytes. If not, then $\mathsf{uncompress}_{G_1}(b) = \times$ (ie, uncompression fails). If the length is equal to 48 bytes, write b as a sequence of bits: $b = b_{383} \cdots b_0$.

- If $b_{383} \neq 1$, then $\mathsf{uncompress}_{G_1}(b) = \times$.
- If $b_{383} = b_{382} = 1$ then $\mathsf{uncompress}_{G_1}(b) = \begin{cases} \mathcal{O}_{G_1} & \text{if } b_{381} = b_{380} = \cdots = b_0 = 0 \\ \times & \text{otherwise.} \end{cases}$
- If $b_{383} = 1$ and $b_{382} = 0$, let $c = \sum_{i=0}^{380} b_i 2^i \in \mathbb{N}$.
  - If $c \geq q$, $\mathsf{uncompress}_{G_1}(b) = \times$.
  - Otherwise, let $x = \bar{c} \in \mathbb{F}_q$ and let $z = x^3 + 4$. If z is not a square in $\mathbb{F}_q$, then $\mathsf{uncompress}_{G_1}(b) = \times$.
  - If z is a square then let $y = \begin{cases} y_{\min}(x) & \text{if } b_{381} = 0 \\ y_{\max}(x) & \text{if } b_{381} = 1. \end{cases}$
  - Then $\mathsf{uncompress}_{G_1}(b) = \begin{cases} (x, y) & \text{if } (x, y) \in G_1 \\ \times & \text{otherwise.} \end{cases}$

Uncompression for G_2 elements. Given a bytestring b, it is checked that b contains exactly 96 bytes. If not, then $\mathsf{uncompress}_{G_2}(b) = \times$ (ie, uncompression fails). If the length is equal to 96 bytes, write b as a sequence of bits: $b = b_{767} \cdots b_0$.

- If $b_{767} \neq 1$, then $\mathsf{uncompress}_{G_2}(b) = \times$.
- If $b_{767} = b_{766} = 1$ then $\mathsf{uncompress}_{G_2}(b) = \begin{cases} \mathcal{O}_{G_2} & \text{if } b_{765} = b_{764} = \cdots = b_0 = 0 \\ \times & \text{otherwise.} \end{cases}$
- If $b_{767} = 1$ and $b_{766} = 0$, let $c = \sum_{i=0}^{383} b_i 2^i$ and $d = \sum_{i=384}^{764} b_i 2^{i-384} \in \mathbb{N}$.
  - If $c \geq q$ or $d \geq q$, $\mathsf{uncompress}_{G_2}(b) = \times$.
  - Otherwise, let $x = \bar{c} + \bar{d}u \in \mathbb{F}_{q^2}$ and let $z = x^3 + 4(u + 1)$. If z is not a square in $\mathbb{F}_{q^2}$, then $\mathsf{uncompress}_{G_2}(b) = \times$.
  - If z is a square then let $y = \begin{cases} y_{\min}(x) & \text{if } b_{765} = 0 \\ y_{\max}(x) & \text{if } b_{765} = 1. \end{cases}$
  - Then $\mathsf{uncompress}_{G_2}(b) = \begin{cases} (x, y) & \text{if } (x, y) \in G_2 \\ \times & \text{otherwise.} \end{cases}$

Note 6. Concrete syntax for BLS12-381 types. Concrete syntax for the `bls12_381_G1_element` and `bls12_381_G2_element` types is provided via the compression and decompression functions defined in Notes 4 and 5. Specifically, a value of type `bls12_381_G1_element` is denoted by a term of the form `(con bls12_381_G1_element 0x...)` where `...` consists of 96 hexadecimal digits representing the 48-byte compressed form of the relevant point. Similarly, a value of type `bls12_381_G2_element` is denoted by a term of the form `(con bls12_381_G2_element 0x...)` where `...` consists of 192 hexadecimal digits representing the 96-byte compressed form of the relevant point. **This syntax is provided only for use in textual Plutus Core programs**, for example for experimentation and testing. We do not support constants of any of the BLS12-381 types in serialised programs on the Cardano blockchain: see Section C.3.4. However, for `bls12_381_G1_element` and `bls12_381_G2_element` one can use the appropriate uncompression function on a bytestring constant at runtime: for example, instead of

```
(con bls12_381_G1_element 0xa1e9a0...)
```

write

```
[(builtin bls12_381_G1_uncompress) (con bytestring #a1e9a0...)].
```

No concrete syntax is provided for values of type `bls12_381_mlresult`. It is not possible to parse such values, and they will appear as `(con bls12_381_mlresult <opaque>)` if output by a program.

Note 7. Pairing operations. For efficiency reasons we split the pairing process into two parts: the `bls12_381_millerLoop` and `bls12_381_finalVerify` functions. We assume that we have

- An intermediate multiplicative abelian group H.
- A function (not necessarily itself a pairing) $e : G_1 \times G_2 \to H$.
- A cyclic group μ_r of order r.
- An epimorphism $\psi : H \to \mu_r$ of groups such that $\psi \circ e : G_1 \times G_2 \to \mu_r$ is a (nondegenerate, bilinear) pairing.

Given these ingredients, we define

- $[\![\texttt{bls12_381_mlresult}]\!] = H$.
- $[\![\texttt{bls12_381_mulMlResult}]\!]$ = the group multiplication operation in H.
- $[\![\texttt{bls12_381_millerLoop}]\!] = e$.
- $[\![\texttt{bls12_381_finalVerify}]\!] = \phi$, where

$$\phi(a,b) = \begin{cases} \mathsf{true} & \text{if } \psi(ab^{-1}) = 1_{\mu_r} \\ \mathsf{false} & \text{otherwise.} \end{cases}$$

We do not mandate specific choices for H, μ_r, e, and ϕ, but a plausible choice would be

- $H = \mathbb{F}^{\times}_{q^{12}}$.
- e is the Miller loop associated with the optimal Ate pairing for E_1 and E_2 [55].
- $\mu_r = \{x \in \mathbb{F}^{\times}_{q^{12}} : x^r = 1\}$, the group of rth roots of unity in $\mathbb{F}_{q^{12}}$. (There are r distinct rth roots of unity in $\mathbb{F}_{q^{12}}$ because the embedding degree of E_1 and E_2 with respect to r is 12 (see [29, 4.1]).)

- $\psi(x) = x^{\frac{q^{12}-1}{r}}$.

The functions `bls12_381_millerLoop` and (especially) `bls12_381_finalVerify` are expected to be expensive, so their use should be kept to a minimum. Fortunately most current use cases do not require many uses of these functions.

4.3.5 Batch 5

4.3.5.1 Built-in functions

The fifth batch of built-in functions adds support for

- Logical and bitwise operations on bytestrings (see [20] and [21]).
- The `RIPEMD-160` hash function.

In the table below most of the functions involve operating on individual bits. We will often view bytestrings as bitstrings $b_{n-1}\cdots b_0$ with $b_i \in \{0,1\}$ (and n necessarily a multiple of 8). Strictly we should use the functions bits and bytes of Section 1.2.3 to convert back and forth between bytestrings and bitstrings, but we elide this for conciseness and reduce ambiguity by using lower-case names like b,c and d for bits and upper-case names like B for bytes. We denote the complement of a bit $b \in \{0,1\}$ by $\bar{b}$, so $\bar{0} = 1$ and $\bar{1} = 0$.

Function	Signature	Denotation	Can fail?	Note
`andByteString`	`[bool, bytestring, bytestring] → bytestring`	and	No	1
`orByteString`	`[bool, bytestring, bytestring] → bytestring`	or	No	1
`xorByteString`	`[bool, bytestring, bytestring] → bytestring`	xor	No	1
`complementByteString`	`[bytestring] → bytestring`	$b_{n-1}\cdots b_0 \mapsto \bar{b}_{n-1}\cdots\bar{b}_0$	No	
`shiftByteString`	`[bytestring, integer] → bytestring`	shift	No	2
`rotateByteString`	`[bytestring, integer] → bytestring`	rotate	No	3
`countSetBits`	`[bytestring] → integer`	$b_{n-1}\cdots b_0 \mapsto \lvert\{i : b_i = 1\}\rvert$	No	
`findFirstSetBit`	`[bytestring] → integer`	ffs	No	4
`readBit`	`[bytestring, integer] → bool`	$(b_{n-1}\cdots b_0, i) \mapsto \begin{cases} b_i & \text{if } 0 \le i \le n-1 \\ \times & \text{otherwise} \end{cases}$	Yes	
`writeBits`	`[bytestring, list (integer), bool] → bytestring`	writeBits	Yes	5
`replicateByte`	`[integer, integer] → bytestring`	replicateByte	Yes	6
`ripemd_160`	`[bytestring] → bytestring`	Compute a RIPEMD-160 hash [10, 33]	No	

Table 4.16: Built-in functions, Batch 5

Note 1. Bitwise logical operations. The bitwise logical operations and, or, and xor are defined by extending the usual single-bit logical operations $\wedge$, $\vee$, and $\oplus$ (respectively) to bytestrings. However, there is a complication if the bytestrings have different lengths.

- If the first argument of one of the bitwise logical operations is `false` then the longer bytestring is (conceptually) truncated on the right to have the same length as the shorter one.
- If the first argument is `true` then the shorter bytestring is (conceptually) extended on the right to have the same length as the longer one. In the case of `and` the shorter bytestring is extended with `1` bits and in the case of `or` and `xor` it is extended with `0` bits.

Formally the truncation operation mentioned above is defined as a function which removes the rightmost k bits from a bitstring:

$$\mathsf{trunc}\,(b_{n-1}\cdots b_0, k) = b_{n-1}\cdots b_k \quad \text{if } 0 \le k \le n$$

and the extension operation is defined as a function which appends k bits to a bitstring:

$$\mathsf{ext}\,(b_{n-1}\cdots b_0, k, x) = b_{n-1}\cdots b_0 \cdot x_{k-1}\cdots x_0 \quad \text{where } x \in \{0,1\},\ k \ge 0, \text{ and } x_0 = x_1 = \cdots = x_{k-1} = x.$$

In both cases, the output is the same as the input when $k = 0$.

The denotations of the bitwise logical functions are now defined on bitstring representations of bytestrings as follows, where b is a bitstring of length m and c is a bitstring of length n:

$$\begin{aligned}
\mathsf{and}\,(\mathsf{false}, b, c) &= d_{l-1}\cdots d_0 \quad \text{where } l = \min\{m,n\} \text{ and } d_i = \mathsf{trunc}\,(b, m-l)_i \wedge \mathsf{trunc}\,(c, n-l)_i \\
\mathsf{and}\,(\mathsf{true}, b, c) &= d_{l-1}\cdots d_0 \quad \text{where } l = \max\{m,n\} \text{ and } d_i = \mathsf{ext}\,(b, l-m, 1)_i \wedge \mathsf{ext}\,(c, l-n, 1)_i \\
\\
\mathsf{or}\,(\mathsf{false}, b, c) &= d_{l-1}\cdots d_0 \quad \text{where } l = \min\{m,n\} \text{ and } d_i = \mathsf{trunc}\,(b, m-l)_i \vee \mathsf{trunc}\,(c, n-l)_i \\
\mathsf{or}\,(\mathsf{true}, b, c) &= d_{l-1}\cdots d_0 \quad \text{where } l = \max\{m,n\} \text{ and } d_i = \mathsf{ext}\,(b, l-m, 0)_i \vee \mathsf{ext}\,(c, l-n, 0)_i \\
\\
\mathsf{xor}\,(\mathsf{false}, b, c) &= d_{l-1}\cdots d_0 \quad \text{where } l = \min\{m,n\} \text{ and } d_i = \mathsf{trunc}\,(b, m-l)_i \oplus \mathsf{trunc}\,(c, n-l)_i \\
\mathsf{xor}\,(\mathsf{true}, b, c) &= d_{l-1}\cdots d_0 \quad \text{where } l = \max\{m,n\} \text{ and } d_i = \mathsf{ext}\,(b, l-m, 0)_i \oplus \mathsf{ext}\,(c, l-n, 0)_i.
\end{aligned}$$

Note that although `trunc` is applied to both arguments in the above definitions, if the arguments have the same lengths then in fact neither will be truncated and if the arguments have different lengths then only the longer one will be truncated. Similarly extension will only occur if the arguments are of different lengths, in which case the shorter one will be extended.

Note 2. Shifting bits. The `shiftByteString` builtin takes a bytestring s and an integer k and shifts the bits of the bytestring k places to the left if $k \ge 0$ and k places to the right if $k < 0$, replacing vacated bits with `0`. The length of the output bytestring is the same as that of the input. The denotation (defined on the bitstring representation of s) is

$$\mathsf{shift}\,(b_{n-1}\cdots b_0, k) = d_{n-1}\cdots d_0 \quad \text{where } d_i = \begin{cases} b_{i-k} & \text{if } 0 \le i-k \le n-1 \\ 0 & \text{otherwise.} \end{cases}$$

Note 3. Rotating bits. The `rotateByteString` builtin takes a bytestring s and an integer k and rotates the bits of s k places to the left if $k \ge 0$ and k places to the right if $k < 0$. The length of the output bytestring is the same as that of the input. The denotation of `rotateByteString` (defined on the bitstring representation of s) is

$$\mathsf{rotate}\,(b_{n-1}\cdots b_0, k) = d_{n-1}\cdots d_0 \quad \text{where } d_i = b_{(i-k) \bmod n}.$$

Note 4. Finding the first set bit in a bytestring. The `findFirstSetBit` builtin returns the index of the first nonzero bit in a bytestring, counting from the *right*. If the bytestring consists entirely of zeros then it returns -1. The denotation $\mathtt{ffs} : \mathbb{b}^* \to \mathbb{Z}$ is

$$\mathtt{ffs}\,(b_{n-1} \cdots b_0) = \begin{cases} -1 & \text{if } b_i = 0 \text{ for } 0 \le i \le n-1 \\ \min\{i \,:\, b_i \neq 0\} & \text{otherwise.} \end{cases}$$

Note 5. Writing bits. The denotation writeBits of the `writeBits` builtin takes a bytestring s, a list J of integer indices, and a boolean value u. An error occurs if any of the indices in J is not a valid bit index for s. If all of the indices are within bounds then for each index j in J the j-th bit of s is updated according to the value of u (0 for false, 1 for true). The list J is allowed to contain repetitions.

Formally,

$$\mathsf{writeBits}(b_{n-1}\cdots b_0, [j_0, \ldots, j_{l-1}], u) = \begin{cases} \times & \text{if } \exists k \in \{0, \ldots, l-1\} \text{ such that } j_k < 0 \text{ or } j_k \ge n \\ d_{n-1}\cdots d_0 & \text{otherwise} \end{cases}$$

where

$$d_i = \begin{cases} b_i & \text{if } i \notin \{j_0, \ldots, j_{l-1}\} \\ 0 & \text{if } i \in \{j_0, \ldots, j_{l-1}\} \text{ and } u = \mathsf{false} \\ 1 & \text{if } i \in \{j_0, \ldots, j_{l-1}\} \text{ and } u = \mathsf{true}. \end{cases}$$

Note 6. Replicating bytes. The `replicateByte` builtin takes a length l between 0 and 8192 and an integer B between 0 and 255 and produces a bytestring of length l. It fails if either argument is outside the required bounds. The denotation is

$$\mathsf{replicateByte}\,(l, B) = \begin{cases} B_0\cdots B_{l-1} \quad (B_i = B \text{ for all } i) & \text{if } 0 \le l \le 8192 \text{ and } B \in \mathbb{B} \\ \times & \text{otherwise.} \end{cases}$$

Note that unlike the other denotations in this section we are viewing the output as a bytestring here, not a bitstring.

4.3.6 Batch 6

The sixth batch of built-in types and functions provides

- A modular exponentiation function (see CIP-0109 [18]).
- A function to drop a specified number of items from the front of a built-in list (see CIP-0132 [22]).
- A built-in array type and functions for working with arrays (see CIP-0138) [23])
- Two functions to provide efficient multi-scalar multiplication for the BLS12-381 elliptic curves (see CIP-0133 [24]).
- A built-in `value` type to represent quantities of currencies on the Cardano blockchain, together with a number of operations on values. These are described separately from the rest of the functions, in Sections 4.3.6.3 and 4.3.6.4. See CIP-0153 [25].

4.3.6.1 The `array` type operator

The `array` type operator is described in Table 4.17.

Operator *op*	$\|op\|$	Denotation	Concrete Syntax
`array`	1	$[\![\texttt{array}\,(t)]\!] = [\![t]\!]^*$	See below

Table 4.17: Array type operator, Batch 6

Arrays. Arrays are finite sequences of built-in values of a particular type with the expectation that looking up the nth element takes constant time (unlike lists).

An array of type `array`(t) is written as a syntactic list $[c_1, \ldots, c_n]$ where each c_i lies in $\mathbf{C}_t$. Some valid constant expressions are thus:

```
(con (array bool) [True, False, True])
(con (array integer) [])
```

4.3.6.2 Built-in functions (1)

The non-`value`-related built-in functions in Batch 6 are defined in Table 4.18.

Function	Signature	Denotation	Can fail?	Note
`expModInteger`	$[\texttt{integer}, \texttt{integer}, \texttt{integer}] \to \texttt{integer}$	expMod	Yes	1
`dropList`	$[\forall a_\#, \texttt{integer}, \texttt{list}\,(a_\#)] \to \texttt{list}\,(a_\#)$	$(k, [x_1, \ldots, x_n]) \mapsto \begin{cases} [x_1, \ldots, x_n] & \text{if } k \le 0 \\ [x_{k+1}, \ldots, x_n] & \text{if } 1 \le k \le n-1 \\ [] & \text{if } k \ge n \end{cases}$	No	
`lengthOfArray`	$[\forall a_\#, \texttt{array}\,(a_\#)] \to \texttt{integer}$	$[] \mapsto 0$ $[x_1, \ldots, x_n] \mapsto n \; (n \ge 1)$	No	2

Table 4.18: Built-in functions, Batch 6 (1)

Function	Type	Denotation	Can fail?	Note
`listToArray`	$[\forall a_{\#}, \texttt{list}\,(a_{\#})] \to \texttt{array}\,(a_{\#})$	$[] \mapsto []$ $[x_1, \ldots, x_n] \mapsto [x_1, \ldots, x_n]$ $(n \geq 1)$	No	2
`indexArray`	$[\forall a_{\#}, \texttt{array}\,(a_{\#}), \texttt{integer}] \to \texttt{a}_{\#}$	$([x_1, \ldots, x_n], k)$ $\mapsto \begin{cases} \times & \text{if } k < 0 \\ x_{k+1} & \text{if } 0 \leq k \leq n-1 \\ \times & \text{if } k > n-1 \end{cases}$	Yes	2
`bls12_381_G1_multiScalarMul`	[`list (integer)`, `list (bls12_381_G1_element)`] $\to$ `bls12_381_G1_element`	$([n_1, \ldots, n_k], [a_1, \ldots, a_l])$ $\mapsto n_1 a_1 + \cdots + n_r a_r$ where $r = \min(k, l)$	No	3
`bls12_381_G2_multiScalarMul`	[`list (integer)`, `list (bls12_381_G2_element)`] $\to$ `bls12_381_G2_element`	$([n_1, \ldots, n_k], [a_1, \ldots, a_l])$ $\mapsto n_1 a_1 + \cdots + n_r a_r$ where $r = \min(k, l)$	No	3

Table 4.18: Built-in functions, Batch 6 (1)

Note 1. Modular Exponentiation. The `expModInteger` function performs modular exponentiation. Its denotation $\mathsf{expMod} : \mathbb{Z} \times \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}_{\times}$ is given by

$$\mathsf{expMod}(a, e, m) = \begin{cases} \mathsf{mod}(a^e, m) & \text{if } m > 1 \text{ and } e \geq 0 \\ \min\{r \in \mathbb{N} : ra^{-e} \equiv 1 \pmod{m}\} & \text{if } m > 1, e < 0 \text{ and } \gcd(a, m) = 1 \\ \times & \text{if } m > 1, e < 0, \text{ and } \gcd(a, m) \neq 1 \\ 0 & \text{if } m = 1 \\ \times & \text{if } m \leq 0. \end{cases}$$

The fact that this is well defined for the case $e < 0$ and $\gcd(a, m) = 1$ follows, for example, from Proposition I.3.1 of [43]. See Note 1 of Section 4.3.1.2 for the definition of mod.

Note 2. Semantics of arrays. The denotation of the array type is identical to that of lists, but we expect that in practice arrays will be implemented in such a way that the `lengthOfArray` and `indexArray` functions will be constant time and `listToArray` may be nontrivial, requiring linear time.

Note 3. BLS12-381 multi-scalar multiplication. We have added two multi-scalar-multiplication operations for BLS12-381 types. Given lists of scalars $[n_1, \ldots, n_k]$ and group elements $[a_1, \ldots, a_l]$ these compute the group sum of the scalar products $n_i a_i$ with $1 \leq i \leq \min(k, l)$. The input lists are allowed to be of different lengths but the longer one is effectively truncated to make them the same length. If either list is empty then the result of the function is the zero element of the relevant group. Multi-scalar multiplication can already be performed using the standard group addition and scalar multiplication functions from Batch 4, but having dedicated functions allows for significantly more efficient implementation.

4.3.6.3 The built-in `value` type

Built-in Values. The built-in `value` type represents the Value type used to represent quantities of currencies in Cardano's EUTXO model: see [13]; we provide a built-in type to represent these and a number of operations for working with them, as proposed in CIP-0153 [25].

A `value` involves a finite set of *currency identifiers*, each of which has a finite number of associated *token identifiers*, each of which has an associated integral *quantity*. Various other names are used for these: for example the Cardano ledger and the Aiken language call currency identifiers *policy IDs* and token identifiers *asset names*, and the Plinth and Plutarch languages call currency identifiers *currency symbols* and token identifiers *token names*.

Currency and token identifiers are bytestrings, and in a particular `value` a given token identifier may be associated with multiple currency identifiers. We model `values` as finitely supported maps from (currency identifier, token identifier) pairs to integers: see Table 4.19 (and recall that $\mathbb{B}^*$ denotes the set of all bytestrings, ie finite sequences of bytes). For brevity we will call such finitely supported maps *values* in this section. In practice values will be implemented as some finite data structure in which only a finite number of currency and token identifiers appear, but for specification purposes it is convenient to allow a value to contain *all* identifiers, but with only a finite number of quantities nonzero. In practice we only allow values v such that for $c, t \in \mathbb{B}^*$ with $v(c,t) \neq 0$, it is the case that $\ell(c) \leq 32$, $\ell(t) \leq 32$ and $-2^{127} \leq q \leq 2^{127} - 1$. We will call such values *small values*; none of the built-in functions we provide allow a non-small value to be created, and non-small values will also cause a failure during deserialisation from the `flat` format (see Appendix C) and during parsing of textual UPLC.

Type	Denotation	Concrete Syntax
`value`	$\mathsf{FinSup}(\mathbb{B}^* \times \mathbb{B}^*, \mathbb{Z})$	See below

Table 4.19: Atomic types, Batch 6

To obtain a syntactic representation of the abstract value type we define a concrete "flattened" form which represents a value as a nested association list. Given $v \in \mathsf{FinSup}(\mathbb{B}^* \times \mathbb{B}^*, \mathbb{Z})$, let

$$C_v = \{c \in \mathbb{B}^* \,:\, \text{there exists } t \in \mathbb{B}^* \text{ with } v(c,t) \neq 0\}.$$

Since v is finitely supported this set is finite, and since the set $\mathbb{B}^*$ of all bytestrings is totally ordered (lexicographically), we can write

$$C_v = \{c_1, \ldots, c_n\} \text{ with } c_1 < c_2 < \cdots < c_n$$

with $n \geq 0$, the case $n = 0$ corresponding to the zero map. Similarly, for $1 \leq i \leq n$ we let

$$T_v(i) = \{t \in \mathbb{B}^* \,:\, v(c_i, t) \neq 0\};$$

each of these sets is nonempty and we can write

$$T_v(i) = \{t_{i1}, t_{i2}, \ldots, t_{ik_i}\} \text{ with } t_{i1} < t_{i2} < \cdots < t_{ik_i}$$

with $k_i \geq 1$ for $1 \leq i \leq n$. We now define $v{\downarrow} \in (\mathbb{B}^* \times (\mathbb{B}^* \times \mathbb{Z})^*)^*$ by

$$\begin{aligned} v{\downarrow} = [&(c_1, [(t_{11}, v(c_1, t_{11})), (t_{12}, v(c_1, t_{12})), \ldots, (t_{1k_1}, v(c_1, t_{1k_1}))],\\ &(c_2, [(t_{21}, v(c_2, t_{21})), (t_{22}, v(c_2, t_{22})), \ldots, (t_{2k_2}, v(c_2, t_{2k_2}))],\\ &\ldots,\\ &(c_n, [(t_{n1}, v(c_n, t_{n1})), (t_{n2}, v(c_n, t_{n2})), \ldots, (t_{nk_n}, v(c_n, t_{nk_n}))]] \end{aligned}$$

Conversely, given a list

$$\begin{aligned} l = [&(c_1, [(t_{11}, q_{11}), (t_{12}, q_{12}), \ldots, (t_{1k_1}, q_{1k_1})]),\\ &(c_2, [(t_{21}, q_{21}), (t_{22}, q_{22}), \ldots, (t_{2k_2}, q_{2k_2})]),\\ &\ldots,\\ &(c_n, [(t_{n1}, q_{n1}), (t_{n2}, q_{n2}), \ldots, (t_{nk_n}, q_{nk_n})])] \end{aligned}$$

in $(\mathbb{B}^* \times (\mathbb{B}^* \times \mathbb{Z})^*)^*$ with

- $n \geq 0$
- $c_1 < c_2 < \cdots < c_n$
- $k_i \geq 1$ for $1 \leq i \leq n$
- $t_{i1} < t_{i2} < \cdots < t_{ik_i}$ for $1 \leq i \leq n$
- $q_{ij} \in [-2^{127}, 2^{127} - 1] \setminus 0$ for all i and j

(we call a list satisfying these conditions a *well-formed* list) we can define a finitely supported map $l\uparrow \in \mathsf{FinSup}(\mathbb{B}^* \times \mathbb{B}^*, \mathbb{Z})$ by

$$l\uparrow(c,t) = \begin{cases} q_{ij} & \text{if } c = c_i \text{ for some } i \text{ and } t = t_{ij} \text{ for some } i \text{ and } j \\ 0 & \text{otherwise.} \end{cases}$$

This function is well defined because the well-formedness requirements ensure that a given pair (c,t) appears at most once in the nested list. If l is not well-formed then we put

$$l\uparrow = \times.$$

We have $v\downarrow\uparrow = v$ for all small values v and $l\uparrow\downarrow = l$ for all well-formed l, the latter because well-formed lists provide a canonical representation of values: for each such value v there is exactly one well-formed list l with $l\uparrow = v$. Without the restrictions we could permute the entries of the lists and add currencies with empty token lists or tokens with zero quantities to obtain different representations of the same value.

Concrete syntax. In the concrete syntax, constants of type `value` are represented by strings of the form of the form

$$\begin{aligned} &\texttt{[(}C_1\texttt{, [(}T_{11}\texttt{, }Q_{11}\texttt{), (}T_{12}\texttt{, }Q_{12}\texttt{), ..., (}T_{1k_1}\texttt{, }Q_{1k_1}\texttt{)]),}\\ &\texttt{ (}C_2\texttt{, [(}T_{21}\texttt{, }Q_{21}\texttt{), (}T_{22}\texttt{, }Q_{22}\texttt{), ..., (}T_{2k_2}\texttt{, }Q_{2k_2}\texttt{)]),}\\ &\texttt{ ...,}\\ &\texttt{ (}C_n\texttt{, [(}T_{n1}\texttt{, }Q_{n1}\texttt{), (}T_{n2}\texttt{, }Q_{n2}\texttt{), ..., (}T_{nk_n}\texttt{, }Q_{nk_n}\texttt{)])]} \end{aligned}$$

where each C_i and T_i are literal bytestrings described by the regular expression `#([0-9A-Fa-f][0-9A-Fa-f])*` with at most 32 pairs of hexadecimal digits and each Q_{ij} is a non-zero literal integer in the range $[-2^{127}, 2^{127}-1]$. The currency IDs and the token IDs for each currency must be in strictly ascending order.

The denotation of the associated `value` is

$$\begin{aligned} &[(\llbracket C_1 \rrbracket, [(\llbracket T_{11} \rrbracket, \llbracket Q_{11} \rrbracket), (\llbracket T_{12} \rrbracket, \llbracket Q_{12} \rrbracket), \ldots, (\llbracket T_{1k_1} \rrbracket, \llbracket Q_{1k_1} \rrbracket)]),\\ &\ (\llbracket C_2 \rrbracket, [(\llbracket T_{21} \rrbracket, \llbracket Q_{21} \rrbracket), (\llbracket T_{22} \rrbracket, \llbracket Q_{22} \rrbracket), \ldots, (\llbracket T_{2k_2} \rrbracket, \llbracket Q_{2k_2} \rrbracket)]),\\ &\ \ldots,\\ &\ (\llbracket C_n \rrbracket, [(\llbracket T_{n1} \rrbracket, \llbracket Q_{n1} \rrbracket), (\llbracket T_{n2} \rrbracket, \llbracket Q_{n2} \rrbracket), \ldots, (\llbracket T_{nk_n} \rrbracket, \llbracket Q_{nk_n} \rrbracket)])]\uparrow \in \mathsf{FinSup}(\mathbb{B}^* \times \mathbb{B}^*, \mathbb{Z}) \end{aligned}$$

More intuitively (but less precisely), this maps every pair (C_i, T_{ij}) occurring in the constant to Q_{ij} and everything else to 0.

Some examples of valid `value` constants are

```
(con value [])
(con value [(#1234AB, [(#, 2), (#01, 44), (#05, -3)])])
(con value [(#11, [(#, 2), (#01, 1), (#05, 3)]),
            (#3333, [(#0f,  4), (#1000, -1)])])
```

The expressions

```
(con value [(#33,[(#1234, 455)]), (#2222, [(#9a, -7)])])
```

and

```
(con value [(#2222, [(#9a, 44), (#F0, 0), (#FF, 22)])])
```

are not valid, the first because the currency IDs are not in ascending order and the second because the token with ID `#F0` appears with quantity 0.

4.3.6.4 Built-in functions operating on `values`

The built-in functions operating on `values` are defined in Table 4.20. The inputs to most of these are restricted in size and to make it easier to specify them we introduce the sets $\mathbb{B}^*_{32}$ of bytestrings of length at most 32 and Q of 128-bit integers:

$$\mathbb{B}^*_{32} = \{[c_0, \ldots, c_{n-1}] \in \mathbb{B}^* : n \leq 32\}$$

$$Q = \{n \in \mathbb{Z} : -2^{127} \leq n \leq 2^{127} - 1\}.$$

Function	Signature	Denotation	Can fail?	Note
`insertCoin`	[`bytestring`, `bytestring`, `integer`, `value`] → `value`	$(c,t,n,v) \mapsto \begin{cases} v[(c,t) \mapsto 0] & \text{if } n = 0 \\ v[(c,t) \mapsto n] & \text{if } c,t \in B^*_{32}, n \in Q \\ \times & \text{otherwise} \end{cases}$	Yes	4
`lookupCoin`	[`bytestring`, `bytestring`, `value`] → `integer`	$(c,t,v) \mapsto v(c,t)$	No	5
`unionValue`	[`value`, `value`] → `value`	unionValue	Yes	6
`valueContains`	[`value`, `value`] → `bool`	valueContains	Yes	7
`scaleValue`	[`integer`, `value`] → `value`	scaleValue	Yes	8
`valueData`	[`value`] → `data`	valueData	Yes	9
`unValueData`	[`data`] → `value`	unValueData	Yes	10

Table 4.20: Batch 6: built-in `value` functions

Note 4. Semantics of `insertCoin`. The `insertCoin` function updates the quantity q associated with a (currency identifier, token identifier) pair (c, t) in a value v. If $q = 0$ then the function returns v with the entry for (c, t) (if present) deleted; if there is no entry for (c, t) then the function succeeds and v is returned unaltered. When $q = 0$ there is no limit on the size of the identifiers and the function always succeeds. If q is nonzero and lies in $[-2^{127}, 2^{127} - 1]$ and both c and t are no longer than 32 bytes then the function returns v updated to map (c, t) to q; any previous quantity associated with (c, t) is discarded. If $q \neq 0$ and any of c, t, or q is out of bounds then the function fails.

Note 5. Semantics of `lookupCoin`. The `lookupCoin` function always accepts currency and token identifiers of any length; however there is no way to create a value where an identifier more than 32 bytes long is associated with a nonzero quantity, so if `lookupCoin` is called with such an identifier it will always succeed and return 0.

Note 6. Semantics of `unionValue`. The result of `unionValue` is obtained by adding together corresponding quantities in its inputs, provided that all of the added quantities are in the signed 128-bit range $[-2^{127}, 2^{127} - 1]$: its denotation unionValue is defined by

$$\mathsf{unionValue}(v, w) = \begin{cases} v + w & \text{if } v(c,t) + w(c,t) \in Q \text{ for all } c \text{ and } t \\ \times & \text{otherwise} \end{cases}$$

where $v + w \in \mathsf{FinSup}(\mathbb{B}^* \times \mathbb{B}^*, \mathbb{Z})$ is defined by $(v + w)(c, t) = v(c, t) + w(c, t)$ for all c and t.

Note 7. Semantics of `valueContains`. For `valueContains` v w, an error occurs if either v or w contains a negative quantity. If all quantities are non-negative then the function returns `true` if every quantity in v is greater than or equal to the corresponding quantity in w, otherwise it returns `false`: its denotation valueContains is defined by

$$\mathsf{valueContains}(v, w) = \begin{cases} \mathsf{true} & \text{if } v(c,t) \geq w(c,t) \geq 0 \text{ for all } c \text{ and } t \\ \mathsf{false} & \text{if } v(c_0,t_0) < w(c_0,t_0) \text{ for some } c_0 \text{ and } t_0 \text{ and } v(c,t), w(c,t) \geq 0 \text{ for all } c \text{ and } t \\ \times & \text{otherwise.} \end{cases}$$

Note 8. Semantics of `scaleValue`. The result of `scaleValue` n v is v with all quantities multiplied by n, provided that all of the scaled quantities are in the signed 128-bit range: its denotation scaleValue is defined by

$$\mathsf{scaleValue}(n, v) = \begin{cases} n * v & \text{if } (n * v)(c,t) \in Q \text{ for all } c \text{ and } t \\ \times & \text{otherwise} \end{cases}$$

where $n * v \in \mathsf{FinSup}(\mathbb{B}^* \times \mathbb{B}^*, \mathbb{Z})$ is defined by $(n * v)(c, t) = n * v(c, t)$ for all c and t.

Note 9. Semantics of `valueData`. The `valueData` function converts a `value` object to an object of type `data` (see Section 4.3.1). It returns a `Map` (the *outer map*) containing a list of pairs, the first element of each pair being a `B` object containing a currency identifier, and the second element (an *inner map*) containing a list of (token identifier, quantity) pairs. The elements of the outer and inner maps are

sorted lexicograpically by identifier, there are no repeated identifiers in any map, and the quantities are all nonzero. The function fails if there are more than 40,000 token IDs in total in the input `value`.

Formally, given a value v with

$$\begin{aligned} v\downarrow = [&(c_1, [(t_{11}, q_{11}), (t_{12}, q_{12}), \ldots, (t_{1k_1}, q_{1k_1})]),\\ &(c_2, [(t_{21}, q_{21}), (t_{22}, q_{22}), \ldots, (t_{2k_2}, q_{2k_2})]),\\ &\ldots,\\ &(c_n, [(t_{n1}, q_{n1}), (t_{n2}, q_{n2}), \ldots, (t_{nk_n}, q_{nk_n})])] \end{aligned}$$

we let

$$\mathsf{valueData}(v) = \begin{cases} m & \text{if } \Sigma_{i=1}^{n} k_i \leq 40000 \\ \times & \text{otherwise} \end{cases}$$

where

$$\begin{aligned} m = \mathtt{Map}\,[&(\mathtt{B}\ c_1, \mathtt{Map}\,[(\mathtt{B}\ t_{11}, \mathtt{I}\ q_{11}), (\mathtt{B}\ t_{12}, \mathtt{I}\ q_{12}), \ldots, (\mathtt{B}\ t_{1k_1}, \mathtt{I}\ q_{1k_1})]),\\ &(\mathtt{B}\ c_2, \mathtt{Map}\,[(\mathtt{B}\ t_{21}, \mathtt{I}\ q_{21}), (\mathtt{B}\ t_{22}, \mathtt{I}\ q_{22}), \ldots, (\mathtt{B}\ t_{2k_2}, \mathtt{I}\ q_{2k_2})]),\\ &\ldots,\\ &(\mathtt{B}\ c_n, \mathtt{Map}\,[(\mathtt{B}\ t_{n1}, \mathtt{I}\ q_{n1}), (\mathtt{B}\ t_{n2}, \mathtt{I}\ q_{n2}), \ldots, (\mathtt{B}\ t_{nk_n}, \mathtt{I}\ q_{nk_n})])]. \end{aligned}$$

Note 10. Semantics of `unValueData`. The `unValueData` function is the inverse of `valueData`. It takes a `data` object as input and returns a `value` object. The function requires its input to consist of a single outer `Map` whose entries are pairs of (`B`, `Map`) pairs, with each inner map containing a list of (`B`, `I`) pairs. The keys in the outer map and all of the inner maps must be at most 32 bytes long and they must be strictly ascending according to the lexicographic ordering on bytestrings, and all quantities (the `I` entries) must be nonzero and lie in the interval $[-2^{127}, 2^{127} - 1]$. The outer map is allowed to be empty (representing the empty value) but the inner maps must all be nonempty.

Formally, if d is a data object then if d is of the form

$$\begin{aligned} \mathtt{Map}\,[&(\mathtt{B}\ c_1, \mathtt{Map}\,[(\mathtt{B}\ t_{11}, \mathtt{I}\ q_{11}), (\mathtt{B}\ t_{12}, \mathtt{I}\ q_{12}), \ldots, (\mathtt{B}\ t_{1k_1}, \mathtt{I}\ q_{1k_1})]),\\ &(\mathtt{B}\ c_2, \mathtt{Map}\,[(\mathtt{B}\ t_{21}, \mathtt{I}\ q_{21}), (\mathtt{B}\ t_{22}, \mathtt{I}\ q_{22}), \ldots, (\mathtt{B}\ t_{2k_2}, \mathtt{I}\ q_{2k_2})]),\\ &\ldots,\\ &(\mathtt{B}\ c_n, \mathtt{Map}\,[(\mathtt{B}\ t_{n1}, \mathtt{I}\ q_{n1}), (\mathtt{B}\ t_{n2}, \mathtt{I}\ q_{n2}), \ldots, (\mathtt{B}\ t_{nk_n}, \mathtt{I}\ q_{nk_n})])], \end{aligned}$$

then let

$$\begin{aligned} l = [&(c_1, [(t_{11}, q_{11}), (t_{12}, q_{12}), \ldots, (t_{1k_1}, q_{1k_1})]),\\ &(c_2, [(t_{21}, q_{21}), (t_{22}, q_{22}), \ldots, (t_{2k_2}, q_{2k_2})]),\\ &\ldots,\\ &(c_n, [(t_{n1}, q_{n1}), (t_{n2}, q_{n2}), \ldots, (t_{nk_n}, q_{nk_n})])] \end{aligned}$$

and define

$$\mathsf{unValueData}(d) = l\!\uparrow,$$

remembering that $l\!\uparrow = \times$ if l is not well formed. If the `data` object d is not of the expected form (involving nested `Map`s) then also we let $\mathsf{unValueData}(d) = \times$.

Note. Later we expect to extend the `data` type to include a `Value` field containing a `value` object. This will enable us to introduce much simpler and faster versions of `valueData` and `unValueData`.

4.3.7 Batch 7

A seventh batch of built-in functions is expected to be enabled at Protocol Version 12. It contains an operation for looking up several array elements (CIP-0156 [26]) and four operations on the built-in `value` type (CIP-0168 [27]). The `array` and `value` types are described in Sections 4.3.6.1 and 4.3.6.3, respectively.

4.3.7.1 Built-in functions

The functions in Batch 7 are defined in Table 4.21.

Function	Signature	Denotation	Can fail?	Note
`multiIndexArray`	$[\forall a_{\#}, \texttt{array}\,(a_{\#}),$ $\texttt{list}\,(\texttt{integer})] \to \texttt{list}\,(a_{\#})$	multiIndexArray	Yes	1
`policies`	$[\texttt{value}] \to \texttt{list}\,(\texttt{bytestring})$	$v \mapsto [c_1, \ldots, c_n]$, where $C_v = \{c_1, \ldots, c_n\}$ with $c_1 < \cdots < c_n$	No	2
`assetCount`	$[\texttt{value}] \to \texttt{integer}$	$v \mapsto \lvert\{(c,t) : v(c,t) \neq 0\}\rvert$	No	3
`keepPolicies`	$[\texttt{list}\,(\texttt{bytestring}), \texttt{value}] \to \texttt{value}$	keepPolicies	Yes	4
`dropPolicies`	$[\texttt{list}\,(\texttt{bytestring}), \texttt{value}] \to \texttt{value}$	dropPolicies	Yes	4

Table 4.21: Built-in functions, Batch 7

Note 1. Semantics of `multiIndexArray`. For an array $[x_0, \ldots, x_{n-1}]$ and a list of integer indices $[i_1, \ldots, i_k]$, the denotation is

$$\mathsf{multiIndexArray}([x_0, \ldots, x_{n-1}], [i_1, \ldots, i_k]) = \begin{cases} [x_{i_1}, \ldots, x_{i_k}] & \text{if } k \leq 1024 \text{ and } 0 \leq i_j < n \text{ for all } 1 \leq j \leq k, \\ \times & \text{otherwise.} \end{cases}$$

The indices are zero-based; their order and repetitions are preserved in the result. An empty index list returns the empty list, including when the array is empty.

Note 2. Semantics of `policies`. The function returns the currency identifiers C_v defined in Section 4.3.6.3, once each in strictly ascending lexicographic order.

Note 3. Semantics of `assetCount`. The function returns the number of pairs (c,t) with $v(c,t) \neq 0$. Each distinct pair is counted once, regardless of the sign of its quantity.

Note 4. Semantics of `keepPolicies` and `dropPolicies`. The first argument of both functions is a list of currency identifiers, say $P = [c_1, \ldots, c_k]$. The results are the values defined by

$$\mathsf{keepPolicies}(P, v)(c,t) = \begin{cases} v(c,t) & \text{if } c \in P, \\ 0 & \text{otherwise,} \end{cases}$$

$$\mathsf{dropPolicies}(P, v)(c,t) = \begin{cases} 0 & \text{if } c \in P, \\ v(c,t) & \text{otherwise.} \end{cases}$$

Both functions fail when $k > 256$, even if v is empty. Every list entry counts towards the limit, including duplicates and identifiers longer than 32 bytes. The list need not be sorted. Duplicate identifiers have the same effect as a single occurrence; identifiers longer than 32 bytes have no effect. Quantities under retained currencies are unchanged.

For $k \leq 256$ we have

$$\mathsf{unionValue}(\mathsf{keepPolicies}(P, v), \mathsf{dropPolicies}(P, v)) = v.$$

Appendix A

Formally Verified Behaviours

To follow.

Appendix B

Serialising `data` Objects Using the CBOR Format

B.1 Introduction

In this section we define a CBOR encoding for the `data` type introduced in Section 4.3.1.1. For ease of reference we reproduce the definition of the Haskell `Data` type, which we may regard as the definition of the Plutus `data` type. Other representations are of course possible, but this is useful for the present discussion.

```
data Data =
   Constr Integer [Data]
   | Map [(Data, Data)]
   | List [Data]
   | I Integer
   | B ByteString
```

The CBOR encoding defined here uses basic CBOR encodings as defined in the CBOR standard [9], but with some refinements. Specifically

- We use a restricted encoding for bytestrings which requires that bytestrings are serialised as sequences of blocks, each block being at most 64 bytes long. Any encoding of a bytestring using our scheme is valid according to the CBOR specification, but the CBOR specification permits some encodings which we do not accept. The purpose of the size restriction is to prevent arbitrary data from being stored on the blockchain.
- Large integers (less than -2^{64} or greater than $2^{64} - 1$) are encoded via the restricted bytestring encoding; other integers are encoded as normal. Again, our restricted encodings are compatible with the CBOR specification.
- The `Constr` case of the `data` type is encoded using a scheme which is an early version of a proposed extension of the CBOR specification to include encodings for discriminated unions. See [30] and [8, Section 9.1].

B.2 Notation

We introduce some extra notation for use here and in Appendix C.

The notation $f : X \rightharpoonup Y$ indicates that f is a partial map from X to Y. We denote the empty bytestring by ϵ and (as in 1.2.2) use $\ell(s)$ to denote the length of a bytestring s and $\cdot$ to denote the concatenation of two bytestrings, and also the operation of prepending or appending a byte to a bytestring. We will also make use of the div and mod functions described in Note 1 in Section 4.3.1.

Encoders and decoders. Recall that $\mathbb{B} = \mathbb{N}_{[0,255]}$, the set of integral values that can be represented in a single byte, and that we identify bytestrings with elements of $\mathbb{B}^*$. We will describe the CBOR encoding of the `data` type by defining families of encoding functions (or *encoders*)

$$\mathcal{E}_X : X \to \mathbb{B}^*$$

and decoding functions (or *decoders*)

$$\mathcal{D}_X : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times X$$

for various sets X, such as the set $\mathbb{Z}$ of integers and the set of all `data` items. The encoding function $\mathcal{E}_X$ takes an element $x \in X$ and converts it to a bytestring, and the decoding function $\mathcal{D}_X$ takes a bytestring s, decodes some initial prefix of s to a value $x \in X$, and returns the remainder of s together with x. Decoders for complex types will often be built up from decoders for simpler types. Decoders are *partial* functions because they can fail, for instance, if there is insufficient input, or if the input is not well formed, or if a decoded value is outside some specified range.

Many of the decoders which we define below involve a number of cases for different forms of input, and we implicitly assume that the decoder fails if none of the cases applies. We also assume that if a decoder fails then so does any other decoder which invokes it, so any failure when attempting to decode a particular data item in a bytestring will cause the entire decoding process to fail (immediately).

B.3 The CBOR format

A CBOR-encoded item consists of a bytestring beginning with a *head* which occupies 1,2,3,5, or 9 bytes. Depending on the contents of the head, some sequence of bytes following it may also contribute to the encoded item. The first three bits of the head are interpreted as a natural number between 0 and 7 (the *major type*) which gives basic information about the type of the following data. The remainder of the head is called the *argument* of the head and is used to encode further information, such as the value of an encoded integer or the size of a list of encoded items. Encodings of complex objects may occupy the bytes following the head, and these will typically contain further encoded items.

B.4 Encoding and decoding the heads of CBOR items

For $i \in \mathbb{N}$ we define a function $\mathsf{b}_i : \mathbb{N} \to \mathbb{B}$ which returns the i-th byte of an integer, with the 0-th byte being the least significant:

$$\mathsf{b}_i(n) = \mathsf{mod}(\mathsf{div}(n, 256^i), 256).$$

We use this to define for each $k \geq 1$ a partial function $\mathsf{e}_k : \mathbb{N} \rightharpoonup \mathbb{B}^*$ which converts a sufficiently small integer to a bytestring of length k (possibly with leading zeros):

$$\mathsf{e}_k(n) = [\mathsf{b}_{k-1}(n), \ldots, \mathsf{b}_0(n)] \quad \text{if } n \leq 256^k - 1.$$

This function fails if the input is too large to fit into a k-byte bytestring.

We also define inverse functions $\mathsf{d}_k : \mathbb{B}^* \rightharpoonup \mathbb{N}$ which decode a k-byte natural number from the start of a bytestring, failing if there is insufficient input:

$$\mathsf{d}_k(s) = (s', \sum_{i=0}^{k-1} 256^i b_i) \qquad \text{if } s = [b_{k-1}, \ldots, b_0] \cdot s'.$$

We now define an encoder $\mathcal{E}_{\mathsf{head}} : \mathbb{N}_{[0,7]} \times \mathbb{N}_{[0,2^{64}-1]} \to \mathbb{B}^*$ which takes a major type and a natural number and encodes them as a CBOR head using the standard encoding:

$$\mathcal{E}_{\mathsf{head}}(m,n) = \begin{cases} [32m+n] & \text{if } n \leq 23 \\ (32m+24) \cdot \mathsf{e}_1(n) & \text{if } 24 \leq n \leq 255 \\ (32m+25) \cdot \mathsf{e}_2(n) & \text{if } 256 \leq n \leq 256^2 - 1 \\ (32m+26) \cdot \mathsf{e}_4(n) & \text{if } 256^2 \leq n \leq 256^4 - 1 \\ (32m+27) \cdot \mathsf{e}_8(n) & \text{if } 256^4 \leq n \leq 256^8 - 1. \end{cases}$$

The corresponding decoder $\mathcal{D}_{\mathsf{head}} : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times \mathbb{N}_{[0,7]} \times \mathbb{N}_{[0,2^{64}-1]}$ is given by

$$\mathcal{D}_{\mathsf{head}}(n \cdot s) = \begin{cases} (s, \mathsf{div}(n,32), \mathsf{mod}(n,32)) & \text{if } \mathsf{mod}(n,32) \leq 23 \\ (s', \mathsf{div}(n,32), k) & \text{if } \mathsf{mod}(n,32) = 24 \text{ and } \mathsf{d}_1(s) = (s',k) \\ (s', \mathsf{div}(n,32), k) & \text{if } \mathsf{mod}(n,32) = 25 \text{ and } \mathsf{d}_2(s) = (s',k) \\ (s', \mathsf{div}(n,32), k) & \text{if } \mathsf{mod}(n,32) = 26 \text{ and } \mathsf{d}_4(s) = (s',k) \\ (s', \mathsf{div}(n,32), k) & \text{if } \mathsf{mod}(n,32) = 27 \text{ and } \mathsf{d}_8(s) = (s',k). \end{cases}$$

This function is undefined if the input is the empty bytestring ϵ, if the input is too short, or if its initial byte is not of the expected form.

Heads for indefinite-length items. The functions $\mathcal{E}_{\mathsf{head}}$ and $\mathcal{D}_{\mathsf{head}}$ defined above are used for a number of purposes. One use is to encode integers less than 64 bits, where the argument of the head is the relevant integer. Another use is for "definite-length" encodings of items such as bytestrings and lists, where the head contains the length n of the object and is followed by some encoding of the object itself (for example a sequence of n bytes for a bytestring or a sequence of n encoded objects for the elements of a list). It is also possible to have "indefinite-length" encodings of objects such as lists and arrays, which do not specify the length of an object in advance: instead a special head with argument 31 is emitted, followed by the encodings of the individual items; the end of the sequence is marked by a "break" byte with value 255. We define an encoder $\mathcal{E}_{\mathsf{indef}} : \mathbb{N}_{[2,5]} \to \mathbb{B}^*$ and a decoder $\mathcal{D}_{\mathsf{indef}} : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times \mathbb{N}_{[2,5]}$ which deal with indefinite heads for a given major type:

$$\begin{aligned} \mathcal{E}_{\mathsf{indef}}(m) &= [32m+31] \\ \mathcal{D}_{\mathsf{indef}}(n \cdot s) &= (s,m) \qquad \text{if } n = 32m+31. \end{aligned}$$

Note that $\mathcal{E}_{\mathsf{indef}}$ and $\mathcal{D}_{\mathsf{indef}}$ are only defined for $m \in \{2,3,4,5\}$ (and we shall only use them in these cases). The case $m = 31$ corresponds to the break byte and for $m \in \{0,1,6\}$ the value is not well formed: see [9, 3.2.4].

B.5 Encoding and decoding bytestrings

The standard CBOR encoding of bytestrings encodes a bytestring as either a definite-length sequence of bytes (the length being given in the head) or as an indefinite-length sequence of definite-length "chunks" (see [9, §§3.1 and 3.4.2]). We use a similar scheme, but only allow chunks of length up to 64. To this end, suppose that $a = [a_1, \ldots, a_{64k+r}] \in \mathbb{B}^* \backslash \{\epsilon\}$ where $k \geq 0$ and $0 \leq r \leq 63$. We define the *canonical 64-byte decomposition* $\bar{a}$ of a to be

$$\bar{a} = [[a_1, \ldots, a_{64}], [a_{65}, \ldots, a_{128}], \ldots, [a_{64(k-1)+1}, \ldots, a_{64k}]] \in (\mathbb{B}^*)^*$$

if $r = 0$ and

$$\bar{a} = [[a_1, \ldots, a_{64}], [a_{65}, \ldots, a_{128}], \ldots, [a_{64(k-1)+1}, \ldots, a_{64k}], [a_{64k+1}, \ldots, a_{64k+r}]] \in (\mathbb{B}^*)^*$$

if $r > 0$. The canonical decomposition of the empty list is $\bar{\epsilon} = []$.

We define the encoder $\mathcal{E}_{\mathbb{B}^*} : \mathbb{B}^* \to \mathbb{B}^*$ for bytestrings by encoding bytestrings of size up to 64 using the standard CBOR encoding and encoding larger bytestrings by breaking them up into 64-byte chunks (with the final chunk possibly being less than 64 bytes long) and encoding them as an indefinite-length list (major type 2 indicates a bytestring):

$$\mathcal{E}_{\mathbb{B}^*}(s) = \begin{cases} \mathcal{E}_{\mathsf{head}}(2, \ell(s)) \cdot s & \text{if } \ell(s) \leq 64 \\ \mathcal{E}_{\mathsf{indef}}(2) \cdot \mathcal{E}_{\mathsf{head}}(2, \ell(c_1)) \cdot c_1 \cdot \mathcal{E}_{\mathsf{head}}(2, \ell(c_2)) \cdot \cdots & \\ \qquad \cdots \cdot c_{n-1} \cdot \mathcal{E}_{\mathsf{head}}(2, \ell(c_n)) \cdot c_n \cdot 255 & \text{if } \ell(s) > 64 \text{ and } \bar{s} = [c_1, \ldots, c_n]. \end{cases}$$

The decoder is slightly more complicated. Firstly, for every $n \geq 0$ we define a decoder $\mathcal{D}^{(n)}_{\mathsf{bytes}} : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times \mathbb{B}^*$ which extracts an n-byte prefix from its input (failing in the case of insufficient input):

$$\mathcal{D}^{(n)}_{\mathsf{bytes}}(s) = \begin{cases} (s, \epsilon) & \text{if } n = 0 \\ (s'', b \cdot t) & \text{if } s = b \cdot s' \text{ and } \mathcal{D}^{(n-1)}_{\mathsf{bytes}}(s') = (s'', t). \end{cases}$$

Secondly, we define a decoder $\mathcal{D}_{\mathsf{block}} : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times \mathbb{B}^*$ which attempts to extract a bytestring of length at most 64 from its input; $\mathcal{D}_{\mathsf{block}}$ (and any other function which calls it) will fail if it encounters a bytestring which is greater than 64 bytes.

$$\mathcal{D}_{\mathsf{block}}(s) = \mathcal{D}^{(n)}_{\mathsf{bytes}}(s') \quad \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 2, n) \text{ and } n \leq 64.$$

Thirdly, we define a decoder $\mathcal{D}_{\mathsf{blocks}} : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times \mathbb{B}^*$ which decodes a sequence of blocks and returns their concatenation.

$$\mathcal{D}_{\mathsf{blocks}}(s) = \begin{cases} (s', \epsilon) & \text{if } s = 255 \cdot s' \\ (s'', t \cdot t') & \text{if } \mathcal{D}_{\mathsf{block}}(s) = (s', t) \text{ and } \mathcal{D}_{\mathsf{blocks}}(s') = (s'', t'). \end{cases}$$

Finally we define the decoder $\mathcal{D}_{\mathbb{B}^*} : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times \mathbb{B}^*$ for bytestrings by

$$\mathcal{D}_{\mathbb{B}^*}(s) = \begin{cases} (s', t) & \text{if } \mathcal{D}_{\mathsf{block}}(s) = (s', t) \\ \mathcal{D}_{\mathsf{blocks}}(s') & \text{if } \mathcal{D}_{\mathsf{indef}}(s) = (s', 2). \end{cases}$$

This looks for either a single block or an indefinite-length list of blocks, in the latter case returning their concatenation. It will accept the output of $\mathcal{E}_{\mathbb{B}^*}$ but will reject bytestring encodings containing any blocks greater than 64 bytes long, even if they are valid bytestring encodings according to the CBOR specification.

B.6 Encoding and decoding integers

As with bytestrings we use a specialised encoding scheme for integers which prohibits encodings with overly-long sequences of arbitrary data. We encode integers in $\mathbb{N}_{[-2^{64},2^{64}-1]}$ as normal (see [9, §3.1]: the major type is 0 for positive integers and 1 for negative ones) and larger ones by emitting a CBOR tag (major type 6; argument 2 for positive numbers and 3 for negative numbers) to indicate the sign, then converting the integer to a bytestring and emitting that using the encoder defined above. This encoding scheme is the same as the standard one except for the size limitations.

We firstly define conversion functions $\mathsf{itos} : \mathbb{N} \to \mathbb{B}^*$ and $\mathsf{stoi} : \mathbb{B}^* \to \mathbb{N}$ by

$$\mathsf{itos}(n) = \begin{cases} \epsilon & \text{if } n = 0 \\ \mathsf{itos}(\mathsf{div}(n, 256)) \cdot \mathsf{mod}(n, 256) & \text{if } n > 0. \end{cases}$$

and

$$\mathsf{stoi}(l) = \begin{cases} 0 & \text{if } l = \epsilon \\ 256 \times \mathsf{stoi}(l') + n & \text{if } l = l' \cdot n \text{ with } n \in \mathbb{B}. \end{cases}$$

The encoder $\mathcal{E}_{\mathbb{Z}} : \mathbb{Z} \to \mathbb{B}^*$ for integers is now defined by

$$\mathcal{E}_{\mathbb{Z}}(n) = \begin{cases} \mathcal{E}_{\mathsf{head}}(0, n) & \text{if } 0 \le n \le 2^{64} - 1 \\ \mathcal{E}_{\mathsf{head}}(6, 2) \cdot \mathcal{E}_{\mathbb{B}^*}(\mathsf{itos}(n)) & \text{if } n \ge 2^{64} \\ \mathcal{E}_{\mathsf{head}}(1, -n - 1) & \text{if } -2^{64} \le n \le -1 \\ \mathcal{E}_{\mathsf{head}}(6, 3) \cdot \mathcal{E}_{\mathbb{B}^*}(\mathsf{itos}(-n - 1)) & \text{if } n \le -2^{64} - 1. \end{cases}$$

The decoder $\mathcal{D}_{\mathbb{Z}} : \mathbb{B}^* \rightharpoonup \mathbb{B}^* \times \mathbb{Z}$ inverts this process. The decoder is in fact slightly more permissive than the encoder because it also accepts small integers encoded using the scheme for larger ones. However, the CBOR standard permits integer encodings which contain bytestrings longer than 64 bytes and it will not accept those.

$$\mathcal{D}_{\mathbb{Z}}(s) = \begin{cases} (s', n) & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 0, n) \\ (s', -n - 1) & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 1, n) \\ (s'', \mathsf{stoi}(b)) & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 6, 2) \text{ and } \mathcal{D}_{\mathbb{B}^*}(s') = (s'', b) \\ (s'', -\mathsf{stoi}(b) - 1) & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 6, 3) \text{ and } \mathcal{D}_{\mathbb{B}^*}(s') = (s'', b). \end{cases}$$

B.7 Encoding and decoding `data`

It is now quite straightforward to encode most `data` values. The main complication is in the encoding of constructor tags (the number i in `Constr` i l).

The encoder. The encoder is given by

$$\begin{aligned} \mathcal{E}_{\mathtt{data}}(\mathtt{Map}\ l) &= \mathcal{E}_{\mathsf{head}}(5, \ell(l)) \cdot \mathcal{E}_{(\mathtt{data}^2)^*}(l) \\ \mathcal{E}_{\mathtt{data}}(\mathtt{List}\ l) &= \mathcal{E}_{\mathsf{indef}}(4) \cdot \mathcal{E}_{\mathtt{data}^*}(l) \cdot 255 \\ \mathcal{E}_{\mathtt{data}}(\mathtt{Constr}\ i\ l) &= \mathcal{E}_{\mathsf{ctag}}(i) \cdot \mathcal{E}_{\mathsf{indef}}(4) \cdot \mathcal{E}_{\mathtt{data}^*}(l) \cdot 255 \\ \mathcal{E}_{\mathtt{data}}(\mathtt{I}\ n) &= \mathcal{E}_{\mathbb{Z}}(n) \\ \mathcal{E}_{\mathtt{data}}(\mathtt{B}\ s) &= \mathcal{E}_{\mathbb{B}^*}(s). \end{aligned}$$

This definition uses encoders for lists of data items, lists of pairs of data items, and constructor tags as follows:

$$\mathcal{E}_{\mathtt{data}^*}([d_1, \ldots, d_n]) = \mathcal{E}_{\mathtt{data}}(d_1) \cdot \cdots \cdot \mathcal{E}_{\mathtt{data}}(d_n)$$
$$\mathcal{E}_{(\mathtt{data}^2)^*}([(k_1, d_1), \ldots, (k_n, d_n)]) = \mathcal{E}_{\mathtt{data}}(k_1) \cdot \mathcal{E}_{\mathtt{data}}(d_1) \cdot \cdots \cdot \mathcal{E}_{\mathtt{data}}(k_n) \cdot \mathcal{E}_{\mathtt{data}}(d_n)$$
$$\mathcal{E}_{\mathtt{ctag}}(i) = \begin{cases} \mathcal{E}_{\mathsf{head}}(6, 121 + i) & \text{if } 0 \le i \le 6 \\ \mathcal{E}_{\mathsf{head}}(6, 1280 + (i - 7)) & \text{if } 7 \le i \le 127 \\ \mathcal{E}_{\mathsf{head}}(6, 102) \cdot \mathcal{E}_{\mathsf{head}}(4, 2) \cdot \mathcal{E}_{\mathbb{Z}}(i) & \text{otherwise.} \end{cases}$$

In the final case of $\mathcal{E}_{\mathtt{ctag}}$ we emit a head with major type 4 and argument 2. This indicates that an encoding of a list of length 2 will follow: the first element of the list is the constructor number and the second is the argument list of the constructor, which is actually encoded in $\mathcal{E}_{\mathtt{data}}$. It might be conceptually more accurate to have a single encoder which would encode both the constructor tag and the argument list, but this would increase the complexity of the notation even further. Similar remarks apply to $\mathcal{D}_{\mathtt{ctag}}$ below.

The decoder. The decoder is given by

$$\mathcal{D}_{\mathtt{data}}(s) = \begin{cases} (s'', \mathtt{Map}\ l) & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 5, n) \text{ and } \mathcal{D}^{(n)}_{(\mathtt{data}^2)^*}(s') = (s'', l) \\ (s', \mathtt{List}\ l) & \text{if } \mathcal{D}_{\mathtt{data}^*}(s) = (s', l) \\ (s'', \mathtt{Constr}\ i\ l) & \text{if } \mathcal{D}_{\mathtt{ctag}}(s) = (s', i) \text{ and } \mathcal{D}_{\mathtt{data}^*}(s') = (s'', l) \\ (s', \mathtt{I}\ n) & \text{if } \mathcal{D}_{\mathbb{Z}}(s) = (s', n) \\ (s', \mathtt{B}\ b) & \text{if } \mathcal{D}_{\mathbb{B}^*}(s) = (s', b) \end{cases}$$

where

$$\mathcal{D}_{\mathtt{data}^*}(s) = \begin{cases} \mathcal{D}^{(n)}_{\mathtt{data}^*}(s') & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 4, n) \\ \mathcal{D}^{\mathsf{indef}}_{\mathtt{data}^*}(s') & \text{if } \mathcal{D}_{\mathsf{indef}}(s) = (s', 4) \end{cases}$$

$$\mathcal{D}^{(n)}_{\mathtt{data}^*}(s) = \begin{cases} (s, \epsilon) & \text{if } n = 0 \\ (s'', d \cdot l) & \text{if } \mathcal{D}_{\mathtt{data}}(s) = (s', d) \text{ and } \mathcal{D}^{(n-1)}_{\mathtt{data}^*}(s') = (s'', l) \end{cases}$$

$$\mathcal{D}^{\mathsf{indef}}_{\mathtt{data}^*}(s) = \begin{cases} (s', \epsilon) & \text{if } s = 255 \cdot s' \\ (s'', d \cdot l) & \text{if } \mathcal{D}_{\mathtt{data}}(s) = (s', d) \text{ and } \mathcal{D}^{\mathsf{indef}}_{\mathtt{data}^*}(s') = (s'', l) \end{cases}$$

$$\mathcal{D}^{(n)}_{(\mathtt{data}^2)^*}(s) = \begin{cases} (s, \epsilon) & \text{if } n = 0 \\ (s''', (k, d) \cdot l) & \begin{cases} \text{if } n > 0 \\ \text{and } \mathcal{D}_{\mathtt{data}}(s) = (s', k) \\ \text{and } \mathcal{D}_{\mathtt{data}}(s') = (s'', d) \\ \text{and } \mathcal{D}^{(n-1)}_{(\mathtt{data}^2)^*}(s'') = (s''', l) \end{cases} \end{cases}$$

$$\mathcal{D}_{\mathtt{ctag}}(s) = \begin{cases} (s', i - 121) & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 6, i) \text{ and } 121 \le i \le 127 \\ (s', (i - 1280) + 7) & \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 6, i) \text{ and } 1280 \le i \le 1400 \\ (s''', i) & \begin{cases} \text{if } \mathcal{D}_{\mathsf{head}}(s) = (s', 6, 102) \\ \text{and } \mathcal{D}_{\mathsf{head}}(s') = (s'', 4, 2) \\ \text{and } \mathcal{D}_{\mathbb{Z}}(s'') = (s''', i) \\ \text{and } 0 \le i \le 2^{64} - 1. \end{cases} \end{cases}$$

Note that the decoders for `List` and `Constr` accept both definite-length and indefinite-length lists of encoded `data` values, but the decoder for `Map` only accepts definite-length lists (and the length is the number of *pairs* in the map). This is consistent with CBOR's standard encoding of arrays and lists (major type 4) and maps (major type 5).

Note also that the encoder $\mathcal{E}_{\texttt{ctag}}$ accepts arbitrary integer values for `Constr` tags, but (for compatibility with [30]) the decoder $\mathcal{D}_{\texttt{ctag}}$ only accepts tags in $\mathbb{N}_{[0,2^{64}-1]}$. This means that some valid Plutus Core programs can be serialised but not deserialised, and is the reason for the recommendation in Section 4.3.1.1 that only constructor tags between 0 and $2^{64} - 1$ should be used.

Appendix C

Serialising Plutus Core Terms and Programs Using the `flat` Format

We use the `flat` format [3] to serialise Plutus Core terms, and we regard this format as being the definitive concrete representation of Plutus Core programs. For compactness we generally (and *always* for scripts on the blockchain) replace names with de Bruijn indices (see Section 2.1.3) in serialised programs.

We use bytestrings for serialisation, but it is convenient to define the serialisation and deserialisation process in terms of strings of bits. Some extra bits of padding are added at the end of the encoding of a program to ensure that the number of bits in the output is a multiple of 8, and this allows us to regard serialised programs as bytestrings in the obvious way.

See Section C.4 for some restrictions on serialisation specific to the Cardano blockchain.

Note: `flat` versus CBOR. Much of the Cardano codebase uses the CBOR format for serialisation; however, it is important that serialised scripts not be too large. CBOR pays a price for being a self-describing format. The size of the serialised terms is consistently larger than a format that is not self-describing: benchmarks show that `flat` encodings of Plutus Core scripts are smaller than CBOR encodings by about 35% (without using compression).

C.1 Encoding and decoding

Firstly recall some notation from Section 1.2. The set of all finite sequences of bits is denoted by $\mathsf{b}^* = \{0,1\}^*$. For brevity we write a sequence of bits in the form $b_{n-1}\cdots b_0$ instead of $[b_{n-1},\ldots,b_0]$: thus `011001` instead of $[0,1,1,0,0,1]$). We denote the empty sequence by ϵ, and use $\ell(s)$ to denote the length of a sequence of bits, and $\cdot$ to denote concatenation (or prepending or appending a single bit to a sequence of bits).

Similarly to the CBOR encoding for `data` described in Appendix B, we will describe the flat encoding by defining families of encoding functions (or *encoders*)

$$\mathsf{E}_X : \mathsf{b}^* \times X \to \mathsf{b}^*$$

and (partial) decoding functions (or *decoders*)

$$\mathsf{D}_X : \mathsf{b}^* \rightharpoonup \mathsf{b}^* \times X$$

for various sets X, such as the set $\mathbb{Z}$ of integers and the set of all Plutus Core terms. The encoding function E_X takes a sequence $s \in \mathsf{b}^*$ and an element $x \in X$ and produces a new sequence of bits by appending the encoding of x to s, and the decoding function D_X takes a sequence of bits, decodes some initial prefix of s to a value $x \in X$, and returns the remainder of s together with x.

Encoding functions basically operate by decomposing an object into subobjects and concatenating the encodings of the subobject; however it is sometimes necessary to add some padding between subobjects in order to make sure that parts of the output are aligned on byte boundaries, and for this reason (unlike the CBOR encoding for `data`) all of our encoding functions have a first argument containing all of the previous output, so that it can be examined to determine how much alignment is required.

As in the case of CBOR, decoding functions are partial: they can fail if, for instance, there is insufficient input, or if a decoded value is outside some specified range. To simplify notation we will mention any preconditions separately, with the assumption that the decoder will fail if the preconditions are not met; we also make a blanket assumption that all decoders fail if there is not enough input for them to proceed. Many of the definitions of decoders construct objects by calling other decoders to obtain subobjects which are then composed, and these are often introduced by a condition of the form "if $\mathsf{D}_X(s) = x$". Conditions like this should be read as implicitly saying that if the decoder D_X fails then the whole decoding process fails.

C.1.1 Padding

The encoding functions mentioned above produce sequences of *bits*, but we sometimes need sequences of *bytes*. To this end we introduce a functions $\mathsf{pad} : \mathsf{b}^* \to \mathsf{b}^*$ which adds a sequence of 0s followed by a 1 to a sequence s to get a sequence whose length is a multiple of 8; if s is a sequence such that $\ell(s)$ is already a multiple of 8 then pad still adds an extra byte of padding; pad is used both for internal alignment (for example, to make sure that the contents of a bytestring are aligned on byte boundaries) and at the end of a complete encoding of a Plutus Core program to make the length a multiple of 8 bits. Symbolically,

$$\mathsf{pad}(s) = s \cdot \mathsf{p}_k \quad \text{if } \ell(s) = 8n + k \text{ with } n, k \in \mathbb{N} \text{ and } 0 \leq k \leq 7$$

where

$$\begin{aligned}
\mathsf{p}_0 &= \mathtt{00000001} \\
\mathsf{p}_1 &= \mathtt{0000001} \\
\mathsf{p}_2 &= \mathtt{000001} \\
\mathsf{p}_3 &= \mathtt{00001} \\
\mathsf{p}_4 &= \mathtt{0001} \\
\mathsf{p}_5 &= \mathtt{001} \\
\mathsf{p}_6 &= \mathtt{01} \\
\mathsf{p}_7 &= \mathtt{1}.
\end{aligned}$$

We also define a (partial) inverse function $\mathsf{unpad} : \mathsf{b}^* \rightharpoonup \mathsf{b}^*$ which discards padding:

$$\mathsf{unpad}(q \cdot s) = s \quad \text{if } q = \mathsf{p}_i \text{ for some } i \in \{0, 1, 2, 3, 4, 5, 6, 7\}.$$

This can fail if the padding is not of the expected form or if the input is the empty sequence ϵ.

C.2 Basic `flat` encodings

C.2.1 Fixed-width natural numbers

We often wish to encode and decode natural numbers which fit into some fixed number of bits, and we do this simply by encoding them as their binary expansion (most significant bit first), adding leading zeros if necessary. More precisely for $n \geq 1$ we define an encoder

$$\mathsf{E}_n : \mathsf{b}^* \times \mathbb{N}_{[0,2^{n-1}-1]} \to \mathsf{b}^*$$

by

$$\mathsf{E}_n(s, \sum_{i=0}^{n-1} b_i 2^i) = s \cdot b_{n-1} \cdots b_0 \quad (b_i \in \{0,1\})$$

and a decoder

$$\mathsf{D}_n : \mathsf{b}^* \rightharpoonup \mathsf{b}^* \times \mathbb{N}_{[0,2^{n-1}-1]}$$

by

$$\mathsf{D}_n(b_{n-1} \cdots b_0 \cdot s) = (s, \sum_{i=0}^{n-1} b_i 2^i).$$

As in Appendix B, $\mathbb{N}_{[a,b]}$ denotes the closed interval of integers $\{n \in \mathbb{Z} : a \leq n \leq b\}$. Note that n here is a variable (not a fixed label) so we are defining whole families of encoders $\mathsf{E}_1, \mathsf{E}_2, \mathsf{E}_3, \ldots$ and decoders $\mathsf{D}_1, \mathsf{D}_2, \mathsf{D}_3 \ldots$.

C.2.2 Pairs

Given sets X and Y for which we have defined encoders E_X and E_Y, we define an encoder for elements of $X \times Y$ by composing the individual encoders:

$$\mathsf{E}_{X\times Y}(s, (x, y)) = \mathsf{E}_Y(\mathsf{E}_X(s, x), y)$$

Similarly, if we have decoders D_X and D_Y for X and Y we define

$$\mathsf{D}_{X\times Y}(s) = (s'', (x, y)) \quad \text{if } \mathsf{D}_X(s) = (s', x) \text{ and } \mathsf{D}_Y(s') = (s'', y).$$

C.2.3 Lists

Suppose that we have a set X for which we have defined an encoder E_X and a decoder D_X; we define an encoder $\vec{\mathsf{E}}_X$ which encodes lists of elements of X by emitting the encodings of the elements of the list, each preceded by a `1` bit, then emitting a `0` bit to mark the end of the list.

$$\vec{\mathsf{E}}_X(s, []) = s \cdot 0$$
$$\vec{\mathsf{E}}_X(s, [x_1, \ldots, x_n]) = \vec{\mathsf{E}}_X(s \cdot 1 \cdot \mathsf{E}_X(x_1), [x_2, \ldots, x_n]).$$

The corresponding decoder is given by

$$\vec{\mathsf{D}}_X(0 \cdot s) = (s, [])$$
$$\vec{\mathsf{D}}_X(1 \cdot s) = (s'', x \cdot l) \quad \text{if } D_X(s) = (s', x) \text{ and } \vec{\mathsf{D}}_X(s') = (s'', l).$$

C.2.4 Natural numbers

Every natural number m can be uniquely written in the form

$$m = \sum_{i=0}^{n} k_i 2^{7i}$$

with $n \geq 0$, $0 \leq k_i \leq 127$ for $0 \leq i \leq n$, and $k_n \neq 0$. Each k_i is 7 bit binary block, k_0 being the least significant block.

With this representation, the encoder for natural numbers is

$$\mathsf{E}_{\mathbb{N}}(s, \sum_{i=0}^{n} k_i 2^{7i}) = \begin{cases} \mathsf{E}_7(\vec{\mathsf{E}}_7(s, []), k_0) & \text{if } n = 0 \\ \mathsf{E}_7(\vec{\mathsf{E}}_7(s, [k_0, \ldots, k_{n-1}]), k_n) & \text{if } n \neq 0. \end{cases}$$

The decoder is

$$\mathsf{D}_{\mathbb{N}}(s) = \begin{cases} D_7(s') & \text{if } \vec{\mathsf{D}}_7(s) = (s', []) \\ (s'', \sum_{i=0}^{n} k_i 2^{7i}) & \text{if } \vec{\mathsf{D}}_7(s) = (s', [k_0, \ldots, k_{n-1}]) \text{ and } \mathsf{D}_7(s') = (s'', k_n) \text{ with } n \geq 1. \end{cases}$$

Intuitively, every 7 bit block except for the last one is prefixed by 1 and the last block is prefixed by 0.

C.2.5 Integers

Signed integers are encoded by converting them to natural numbers using the zigzag encoding ($0 \mapsto 0, -1 \mapsto 1, 1 \mapsto 2, -2 \mapsto 3, 2 \mapsto 4, \ldots$) and then encoding the result using $\mathsf{E}_{\mathbb{N}}$:

$$\mathsf{E}_{\mathbb{Z}}(s, n) = \begin{cases} \mathsf{E}_{\mathbb{N}}(s, 2n) & \text{if } n \geq 0 \\ \mathsf{E}_{\mathbb{N}}(s, -2n - 1) & \text{if } n < 0. \end{cases}$$

The decoder is

$$\mathsf{D}_{\mathbb{Z}}(s) = \begin{cases} (s', \frac{n}{2}) & \text{if } n \equiv 0 \pmod 2 \\ (s', -\frac{n+1}{2}) & \text{if } n \equiv 1 \pmod 2 \end{cases} \quad \text{if } \mathsf{D}_{\mathbb{N}}(s) = (s', n).$$

C.2.6 Bytestrings

Bytestrings are encoded by dividing them into nonempty blocks of up to 255 bytes and emitting each block in sequence. Each block is preceded by a single unsigned byte containing its length, and the end of the encoding is marked by a zero-length block (so the empty bytestring is encoded just as a zero-length block). Before emitting a bytestring, the preceding output is padded so that its length (in bits) is a multiple of 8; if this is already the case a single padding byte is still added; this ensures that contents of the bytestring are aligned to byte boundaries in the output.

Recall that $\mathbb{B}$ denotes the set of 8-bit bytes, $\{0, 1, \ldots, 255\}$. For specification purposes we may identify the set of bytestrings with the set $\mathbb{B}^*$ of (possibly empty) lists of elements of $\mathbb{B}$. We denote by C the set of *bytestring chunks* of **nonempty** bytestrings of length at most 255: $C = \{[b_1, \ldots, b_n] : b_i \in \mathbb{B}, 1 \leq n \leq 255\}$, and define a function $E_C : C \to \mathsf{b}^*$ by

$$E_C([b_1, \ldots, b_n]) = \mathsf{E}_8(n) \cdot \mathsf{E}_8(b_1) \cdot \cdots \cdot \mathsf{E}_8(b_n).$$

We define an encoder E_{C^*} for lists of chunks by

$$\mathsf{E}_{C^*}(s, [c_1, \ldots, c_n]) = s \cdot E_C(c_1) \cdot \cdots \cdot E_C(c_n) \cdot \mathtt{00000000}.$$

Note that each c_i is required to be nonempty but that we allow the case $n = 0$, so that an empty list of chunks encodes as 00000000.

To encode a bytestring we decompose it into a list L of chunks and then apply E_{C^*} to L. However, there will usually be many ways to decompose a given bytestring a into chunks. For definiteness we recommend (but do not demand) that a is decomposed into a sequence of chunks of length 255 possibly followed by a smaller chunk. Formally, suppose that $a = [a_1, \ldots, a_{255k+r}] \in \mathbb{B}^*\backslash\{\epsilon\}$ where $k \geq 0$ and $0 \leq r \leq 254$. We define the *canonical 256-byte decomposition* $\tilde{a}$ of a to be

$$\tilde{a} = [[a_1, \ldots, a_{255}], [a_{256}, \ldots, a_{510}], \ldots [a_{255(k-1)+1}, \ldots, a_{255k}]] \in C^*$$

if $r = 0$ and

$$\tilde{a} = [[a_1, \ldots, a_{255}], [a_{256}, \ldots, a_{510}], \ldots [a_{255(k-1)+1}, \ldots, a_{255k}], [a_{255k+1}, \ldots, a_{255k+r}]] \in C^*$$

if $r > 0$.

For the empty bytestring we define

$$\tilde{\epsilon} = [].$$

Given all of the above, we define the canonical encoding function $\mathsf{E}_{\mathbb{B}^*}$ for bytestrings to be

$$\mathsf{E}_{\mathbb{B}^*}(s, a) = E_{C^*}(\mathsf{pad}(s), \tilde{a}).$$

Non-canonical encodings can be obtained by replacing $\tilde{a}$ with any other decomposition of a into nonempty chunks, and the decoder below will accept these as well.

To define a decoder for bytestrings we first define a decoder D_C for bytestring chunks:

$$\mathsf{D}_C(s) = \mathsf{D}_C^{(n)}(s', []) \quad \text{if } \mathsf{D}_8(s) = (s', n)$$

where

$$\mathsf{D}_C^{(n)}(s, l) = \begin{cases} (s, l) & \text{if } n = 0 \\ \mathsf{D}_C^{(n-1)}(s', l \cdot x) & \text{if } n > 0 \text{ and } \mathsf{D}_8(s) = (s', x). \end{cases}$$

Now we define

$$\mathsf{D}_{C^*}(s) = \begin{cases} (s', []) & \text{if } D_C(s) = (s', []) \\ (s'', x \cdot l) & \text{if } \mathsf{D}_C(s) = (s', x) \text{ with } x \neq [] \text{ and } \mathsf{D}_{C^*}(s') = (s'', l). \end{cases}$$

The notation is slightly misleading here: D_{C^*} does not decode to a list of bytestring chunks, but to a single bytestring. We could alternatively decode to a list of bytestrings and then concatenate them later, but this would have the same overall effect.

Finally, we define the decoder for bytestrings by

$$\mathsf{D}_{\mathbb{B}^*}(s) = \mathsf{D}_{C^*}(\mathsf{unpad}(s)).$$

C.2.7 Strings

We have defined values of the `string` type to be sequences of Unicode characters. As mentioned earlier we do not specify any particular internal representation of Unicode characters, but for serialisation we use the UTF-8 representation to convert between strings and bytestrings and then use the bytestring encoder and decoder:

$$\mathsf{E}_{\mathbb{U}^*}(s, u) = \mathsf{E}_{\mathbb{B}^*}(s, \mathsf{utf8}(u))$$

$$\mathsf{D}_{\mathbb{U}^*}(s) = (s', \mathsf{utf8}^{-1}(a)) \quad \text{if } \mathsf{D}_{\mathbb{B}^*}(s) = (s', a)$$

where $\mathsf{utf8}$ and $\mathsf{utf8}^{-1}$ are the UTF8 encoding and decoding functions mentioned in Section 4.3.1. Recall that $\mathsf{utf8}^{-1}$ is partial (not all bytestrings represent valid Unicode sequences), so $\mathsf{D}_{\mathbb{U}^*}$ may fail if the input is invalid.

C.3 Encoding and decoding Plutus Core

C.3.1 Programs

A program is encoded by encoding the three components of the version number in sequence then encoding the body, and possibly adding some padding to ensure that the total number of bits in the output is a multiple of 8 (and hence the output can be viewed as a bytestring).

$$\mathsf{E}_{\mathsf{program}}((\texttt{program}\ a.b.c\ t)) = \mathsf{pad}(\mathsf{E}_{\mathsf{term}}(\mathsf{E}_{\mathbb{N}}(\mathsf{E}_{\mathbb{N}}(\mathsf{E}_{\mathbb{N}}(\epsilon, a), b), c), t)).$$

The decoding process is the inverse of the encoding process: three natural numbers are read to obtain the version number and then the body is decoded. After this we discard any padding in the remaining input and check that all of the input has been consumed.

$$\mathsf{D}_{\mathsf{program}}(s) = (\texttt{program}\ a.b.c\ t) \quad \begin{cases} \text{if} & \mathsf{D}_{\mathbb{N}}(s) = (s', a) \\ \text{and} & \mathsf{D}_{\mathbb{N}}(s') = (s'', b) \\ \text{and} & \mathsf{D}_{\mathbb{N}}(s'') = (s''', c) \\ \text{and} & \mathsf{D}_{\mathsf{term}}(s''') = (r, t) \\ \text{and} & \mathsf{unpad}(r) = \epsilon. \end{cases}$$

C.3.2 Terms

Plutus Core terms are encoded by emitting a 4-bit tag identifying the type of the term (see Table C.1; recall that `[]` denotes application) then emitting the encodings for any subterms. We currently only use ten of the sixteen available tags: the remainder are reserved for potential future expansion.

Term type	Binary	Decimal
Variable	0000	0
`delay`	0001	1
`lam`	0010	2
`[]`	0011	3
`const`	0100	4
`force`	0101	5
`error`	0110	6
`builtin`	0111	7
`constr`	1000	8
`case`	1001	9

Table C.1: Term tags

The encoder for terms is given below: it refers to other encoders (for names, types, and constants) which will be defined later.

$$
\begin{aligned}
\mathsf{E}_{\mathsf{term}}(s, x) &= \mathsf{E}_{\mathsf{name}}(s \cdot 0000, x) \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{delay}\ t)) &= \mathsf{E}_{\mathsf{term}}(s \cdot 0001, t) \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{lam}\ x\ t)) &= \mathsf{E}_{\mathsf{term}}(\mathsf{E}_{\lambda\mathsf{var}}(s \cdot 0010, x), t) \\
\mathsf{E}_{\mathsf{term}}(s, [t_1\ t_2]) &= \mathsf{E}_{\mathsf{term}}(\mathsf{E}_{\mathsf{term}}(s \cdot 0011, t_1), t_2) \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{const}\ tn\ c)) &= \mathsf{E}^{tn}_{\mathsf{constant}}(\mathsf{E}_{\mathsf{type}}(s \cdot 0100, T), c) \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{force}\ t)) &= \mathsf{E}_{\mathsf{term}}(s \cdot 0101, t) \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{error})) &= s \cdot 0110 \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{builtin}\ b)) &= \mathsf{E}_{\mathsf{builtin}}(s \cdot 0111, b) \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{constr}\ i\ l)) &= \vec{\mathsf{E}}_{\mathsf{term}}(\mathsf{E}_{\mathbb{N}}(s \cdot 1000, i), l) \\
\mathsf{E}_{\mathsf{term}}(s, (\texttt{case}\ u\ l)) &= \vec{\mathsf{E}}_{\mathsf{term}}(\mathsf{E}_{\mathsf{term}}(s \cdot 1001, u), l)
\end{aligned}
$$

The decoder for terms is given below. To simplify the definition we use some pattern-matching syntax for inputs to decoders: for example the argument $0101 \cdot s$ indicates that when the input is a string beginning with 0101 the definition after the = sign should be used (and the remainder of the input is available in s there). If the input is not long enough to permit the indicated decomposition then the decoder fails. The decoder also fails if the input begins with a prefix which is not listed; that does not happen here, but does in some later decoders.

$$
\begin{array}{lll}
\mathsf{D}_{\mathsf{term}}(0000 \cdot s) = (s', x) & \text{if } \mathsf{D}_{\mathsf{name}}(s) = (s', x) & \\
\mathsf{D}_{\mathsf{term}}(0001 \cdot s) = (s', (\texttt{delay}\ t)) & \text{if } \mathsf{D}_{\mathsf{term}}(s) = (s', t) & \\
\mathsf{D}_{\mathsf{term}}(0010 \cdot s) = (s'', (\texttt{lam}\ x\ t)) & \text{if } \mathsf{D}_{\lambda\mathsf{var}}(s) = (s', x) & \text{and } \mathsf{D}_{\mathsf{term}}(s') = (s'', t) \\
\mathsf{D}_{\mathsf{term}}(0011 \cdot s) = (s'', [t_1\ t_2]) & \text{if } \mathsf{D}_{\mathsf{term}}(s) = (s', t_1) & \text{and } \mathsf{D}_{\mathsf{term}}(s') = (s'', t_2) \\
\mathsf{D}_{\mathsf{term}}(0100 \cdot s) = (s'', (\texttt{const}\ tn\ c)) & \text{if } \mathsf{D}_{\mathsf{type}}(s) = (s', T) & \text{and } \mathsf{D}^{T}_{\mathsf{constant}}(s') = (s'', c) \\
\mathsf{D}_{\mathsf{term}}(0101 \cdot s) = (s', (\texttt{force}\ t)) & \text{if } \mathsf{D}_{\mathsf{term}}(s) = (s', t) & \\
\mathsf{D}_{\mathsf{term}}(0110 \cdot s) = (s, (\texttt{error})) & & \\
\mathsf{D}_{\mathsf{term}}(0111 \cdot s) = (s', (\texttt{builtin}\ b)) & \text{if } \mathsf{D}_{\mathsf{builtin}}(s) = (s', b) & \\
\mathsf{D}_{\mathsf{term}}(1000 \cdot s) = (s'', (\texttt{constr}\ i\ l)) & \text{if } \mathsf{D}_{\mathbb{N}}(s) = (s', i) \text{ and } i < 2^{64} & \text{and } \overrightarrow{\mathsf{D}}_{\mathsf{term}}(s') = (s'', l) \\
\mathsf{D}_{\mathsf{term}}(1001 \cdot s) = (s'', (\texttt{case}\ u\ l)) & \text{if } \mathsf{D}_{\mathsf{term}}(s) = (s', u) & \text{and } \overrightarrow{\mathsf{D}}_{\mathsf{term}}(s') = (s'', l)
\end{array}
$$

NOTE. The decoder $\mathsf{D}_{\mathsf{term}}$ should fail if we are decoding a program with a version less than 1.1.0 and an input of the form $1000 \cdot s$ or $1001 \cdot s$ is encountered. It should also fail when decoding a `constr` term if a tag is encountered which is greater than or equal to 2^{64} (this enforces the 64-bit limitation mentioned in the paragraph headed **Constructor tags** in Section 2.1.3).

C.3.3 Built-in types

Constants from built-in types are essentially encoded by emitting a sequence of 4-bit tags representing the constant's type and then emitting the encoding of the constant itself. However the encoding of types is somewhat complex because it has to be able to deal with type operators such as `list` and `pair`. The tags are given in Table C.2: they include tags for the basic types together with a tag for a type application operator.

Type	Binary	Decimal
`integer`	0000	0
`bytestring`	0001	1
`string`	0010	2
`unit`	0011	3
`bool`	0100	4
`list`	0101	5
`pair`	0110	6
(type application)	0111	7
`data`	1000	8
`bls12_381_G1_element`	1001	9
`bls12_381_G2_element`	1010	10
`bls12_381_MlResult`	1011	11
`array`	1100	12
`value`	1101	13

Table C.2: Type tags

We define auxiliary functions $\mathsf{e}_{\mathsf{type}} : \mathcal{U} \rightharpoonup \mathbb{N}^*$ and $\mathsf{d}_{\mathsf{type}} : \mathbb{N}^* \rightharpoonup \mathbb{N}^* \times \mathcal{U}$. Both $\mathsf{e}_{\mathsf{type}}$ and $\mathsf{d}_{\mathsf{type}}$ are partial and $\mathcal{U}$ denotes the universe of types defined in Sections 4.3.1, 4.3.2, and 4.3.3.

$$
\begin{aligned}
\mathsf{e}_{\mathsf{type}}(\texttt{integer}) &= [0] \\
\mathsf{e}_{\mathsf{type}}(\texttt{bytestring}) &= [1] \\
\mathsf{e}_{\mathsf{type}}(\texttt{string}) &= [2] \\
\mathsf{e}_{\mathsf{type}}(\texttt{unit}) &= [3] \\
\mathsf{e}_{\mathsf{type}}(\texttt{bool}) &= [4] \\
\mathsf{e}_{\mathsf{type}}(\texttt{list}\,(t)) &= [7,5] \cdot \mathsf{e}_{\mathsf{type}}(t) \\
\mathsf{e}_{\mathsf{type}}(\texttt{array}\,(t)) &= [7,12] \cdot \mathsf{e}_{\mathsf{type}}(t) \\
\mathsf{e}_{\mathsf{type}}(\texttt{pair}\,(t_1,t_2)) &= [7,7,6] \cdot \mathsf{e}_{\mathsf{type}}(t_1) \cdot \mathsf{e}_{\mathsf{type}}(t_2) \\
\mathsf{e}_{\mathsf{type}}(\texttt{data}) &= [8] \\
\mathsf{e}_{\mathsf{type}}(\texttt{value}) &= [13].
\end{aligned}
$$

$$
\begin{aligned}
\mathsf{d}_{\mathsf{type}}(0 \cdot l) &= (l, \texttt{integer}) \\
\mathsf{d}_{\mathsf{type}}(1 \cdot l) &= (l, \texttt{bytestring}) \\
\mathsf{d}_{\mathsf{type}}(2 \cdot l) &= (l, \texttt{string}) \\
\mathsf{d}_{\mathsf{type}}(3 \cdot l) &= (l, \texttt{unit}) \\
\mathsf{d}_{\mathsf{type}}(4 \cdot l) &= (l, \texttt{bool}) \\
\mathsf{d}_{\mathsf{type}}([7,5] \cdot l) &= (l', \texttt{list}\,(t)) && \text{if } \mathsf{d}_{\mathsf{type}}(l) = (l', t) \\
\mathsf{d}_{\mathsf{type}}([7,12] \cdot l) &= (l', \texttt{array}\,(t)) && \text{if } \mathsf{d}_{\mathsf{type}}(l) = (l', t) \\
\mathsf{d}_{\mathsf{type}}([7,7,6] \cdot l) &= (l'', \texttt{pair}\,(t_1,t_2)) && \begin{cases} \text{if} & \mathsf{d}_{\mathsf{type}}(l) = (l', t_1) \\ \text{and} & \mathsf{d}_{\mathsf{type}}(l') = (l'', t_2) \end{cases} \\
\mathsf{d}_{\mathsf{type}}(8 \cdot l) &= (l, \texttt{data}) \\
\mathsf{d}_{\mathsf{type}}(13 \cdot l) &= (l, \texttt{value}).
\end{aligned}
$$

The encoder and decoder for types is obtained by combining $\mathsf{e}_{\mathsf{type}}$ and $\mathsf{d}_{\mathsf{type}}$ with $\vec{\mathsf{E}}_4$ and $\vec{\mathsf{D}}_4$, the encoder and decoder for lists of four-bit integers (see Section C.2).

$$\mathsf{E}_{\mathsf{type}}(s,t) = \vec{\mathsf{E}}_4(s, \mathsf{e}_{\mathsf{type}}(t))$$

$$\mathsf{D}_{\mathsf{type}}(s) = (s',t) \quad \text{if } \vec{\mathsf{D}}_4(s) = (s', l) \text{ and } \mathsf{d}_{\mathsf{type}}(l) = ([], t).$$

C.3.4 Constants

Values of built-in types can mostly be encoded quite simply by using encoders already defined:

$$
\begin{aligned}
\mathsf{E}^{\texttt{integer}}_{\mathsf{constant}}(s, n) &= \mathsf{E}_{\mathbb{Z}}(s, n) \\
\mathsf{E}^{\texttt{bytestring}}_{\mathsf{constant}}(s, a) &= \mathsf{E}_{\mathbb{B}^*}(s, a) \\
\mathsf{E}^{\texttt{string}}_{\mathsf{constant}}(s, t) &= \mathsf{E}_{\mathbb{U}^*}(s, t) \\
\mathsf{E}^{\texttt{unit}}_{\mathsf{constant}}(s, c) &= s \\
\mathsf{E}^{\texttt{bool}}_{\mathsf{constant}}(s, \texttt{False}) &= s \cdot 0 \\
\mathsf{E}^{\texttt{bool}}_{\mathsf{constant}}(s, \texttt{True}) &= s \cdot 1 \\
\mathsf{E}^{\texttt{list}\,(T)}_{\mathsf{constant}}(s, l) &= \vec{\mathsf{E}}^{T}_{\mathsf{constant}}(s, l) \\
\mathsf{E}^{\texttt{array}\,(T)}_{\mathsf{constant}}(s, a) &= \vec{\mathsf{E}}^{T}_{\mathsf{constant}}(s, a) \\
\mathsf{E}^{\texttt{pair}\,(T_1, T_2)}_{\mathsf{constant}}(s, (c_1, c_2)) &= \mathsf{E}^{T_2}_{\mathsf{constant}}(\mathsf{E}^{T_1}_{\mathsf{constant}}(s, c_1), c_2) \\
\mathsf{E}^{\texttt{data}}_{\mathsf{constant}}(s, d) &= \mathsf{E}_{\mathbb{B}^*}(s, \mathcal{E}_{\texttt{data}}(d)) \\
\mathsf{E}^{\texttt{value}}_{\mathsf{constant}}(s, v) &= \vec{\mathsf{E}}_{\mathbb{B}^* \times I^*}(s, v{\downarrow}) \quad \text{where } I = \mathbb{B}^* \times \mathbb{Z} \text{ and } \mathsf{E}_{I^*} = \vec{\mathsf{E}}_I
\end{aligned}
$$

$$
\begin{aligned}
\mathsf{D}^{\texttt{integer}}_{\mathsf{constant}}(s) &= \mathsf{D}_{\mathbb{Z}}(s) \\
\mathsf{D}^{\texttt{bytestring}}_{\mathsf{constant}}(s) &= \mathsf{D}_{\mathbb{B}^*}(s) \\
\mathsf{D}^{\texttt{string}}_{\mathsf{constant}}(s) &= \mathsf{D}_{\mathbb{U}^*}(s) \\
\mathsf{D}^{\texttt{unit}}_{\mathsf{constant}}(s) &= s \\
\mathsf{D}^{\texttt{bool}}_{\mathsf{constant}}(0 \cdot s) &= (s, \texttt{False}) \\
\mathsf{D}^{\texttt{bool}}_{\mathsf{constant}}(1 \cdot s) &= (s, \texttt{True}) \\
\mathsf{D}^{\texttt{list}\,(T)}_{\mathsf{constant}}(s) &= \vec{\mathsf{D}}^{T}_{\mathsf{constant}}(s) \\
\mathsf{D}^{\texttt{array}\,(T)}_{\mathsf{constant}}(s) &= \vec{\mathsf{D}}^{T}_{\mathsf{constant}}(s) \\
\mathsf{D}^{\texttt{pair}\,(T_1, T_2)}_{\mathsf{constant}}(s) &= (s'', (c_1, c_2)) \begin{cases} \text{if} & \mathsf{D}^{T_1}_{\mathsf{constant}}(s) = (s', c_1) \\ \text{and} & \mathsf{D}^{T_2}_{\mathsf{constant}}(s') = (s'', c_2) \end{cases} \\
\mathsf{D}^{\texttt{data}}_{\mathsf{constant}}(s) &= (s', d) \qquad \text{if } \mathsf{D}_{\mathbb{B}*}(s) = (s', t) \text{ and } \mathcal{D}_{\texttt{data}}(t) = (t', d) \text{ for some } t' \\
\mathsf{D}^{\texttt{value}}_{\mathsf{constant}}(s) &= (s', x{\uparrow}) \qquad \text{if } \vec{\mathsf{D}}_{\mathbb{B}^* \times I^*} = (s', l) \text{ where } I = \mathbb{B}^* \times \mathbb{Z} \text{ and } \mathsf{D}_{I^*} = \vec{\mathsf{D}}_I.
\end{aligned}
$$

Units. The unit value `(con unit ())` does not have an explicit encoding: the type has only one possible value, so there is no need to use any space to serialise it.

Data. The `data` type is encoded by converting to a bytestring using the CBOR encoder $\mathcal{E}_{\texttt{data}}$ described in Appendix B and then using $\mathsf{E}_{\mathbb{B}^*}$. The decoding process is the opposite of this: a bytestring is obtained using $\mathsf{D}_{\mathbb{B}^*}$ and this is then decoded from CBOR using $\mathcal{D}_{\texttt{data}}$ to obtain a `data` object.

Arrays. Arrays use the same encoders and decoders as lists (see Section C.2.3): given a set X for which we have defined an encoder E_X and a decoder D_X, arrays of elements of X are encoded using $\vec{\mathsf{E}}_X$ and decoded using $\vec{\mathsf{D}}_X$. In practice the run-time implementations of lists and arrays may differ and some extra work may be required to convert arrays to lists before encoding and lists to arrays after decoding.

BLS12-381 elements. We do not provide serialisation and deserialisation methods for constants of type `bls12_381_G1_element`, `bls12_381_G2_element`, or `bls12_381_mlresult`. We have specified tags for these types, but if one of these tags is encountered during deserialisation then deserialisation fails and any subsequent input is ignored. Note however that constants of the first two types can be serialised by using the compression functions defined in Section 4.3.4.3 and serialising the resulting bytestrings. Decoding can similarly be performed indirectly by using `bls12_381_G1_uncompress` and `bls12_381_G2_uncompress` on bytestring constants during program execution.

The `value` type. The encoder and decoder for the `value` type use the concrete representation of a `value` as an element of $(\mathbb{B}^* \times (\mathbb{B}^* \times \mathbb{Z})^*)^*$ described in Section 4.3.6.3, where a `value` is represented by an outer list of pairs of bytestrings and inner lists, an inner list being a list of pairs of bytestrings and integers, ie a list of elements of $I = \mathbb{B}^* \times \mathbb{Z}$. To encode a `value` v we convert it to the concrete representation $v\downarrow$ and then encode that: inner lists are encoded using the standard list encoding (see Section C.2.3) for lists with elements in $\mathbb{B}^* \times \mathbb{Z}$ and outer lists are encoded using the standard list encoding for lists with elements in $\mathbb{B}^* \times I$. The decoder reverses this process, but note that we use the $\uparrow$ operation, which fails if the currency identifiers or the token identifiers for a given currency are not in strictly ascending order; it also fails if it encounters a bytestring with length greater than 32 or an integer value which is zero or lies outside the range $[-2^{127}, 2^{127} - 1]$.

C.3.5 Built-in functions

Built-in functions are represented by seven-bit integer tags and encoded and decoded using E_7 and D_7. The tags are specified in Tables C.3–C.9. We assume that there are (partial) functions tag and tag^{-1} which convert back and forth between builtin names and their tags.

$$\begin{aligned}
\mathsf{E}_{\mathsf{builtin}}(s, b) &= \mathsf{E}_7(s, \mathsf{tag}(b)) \\
\mathsf{D}_{\mathsf{builtin}}(s) \quad &= (s', \mathsf{tag}^{-1}(n)) \quad \text{if } \mathsf{D}_7(s) = (s', n).
\end{aligned}$$

Builtin	Binary	Decimal	Builtin	Binary	Decimal
addInteger	0000000	0	ifThenElse	0011010	26
subtractInteger	0000001	1	chooseUnit	0011011	27
multiplyInteger	0000010	2	trace	0011100	28
divideInteger	0000011	3	fstPair	0011101	29
quotientInteger	0000100	4	sndPair	0011110	30
remainderInteger	0000101	5	chooseList	0011111	31
modInteger	0000110	6	mkCons	0100000	32
equalsInteger	0000111	7	headList	0100001	33
lessThanInteger	0001000	8	tailList	0100010	34
lessThanEqualsInteger	0001001	9	nullList	0100011	35
appendByteString	0001010	10	chooseData	0100100	36
consByteString	0001011	11	constrData	0100101	37
sliceByteString	0001100	12	mapData	0100110	38
lengthOfByteString	0001101	13	listData	0100111	39
indexByteString	0001110	14	iData	0101000	40
equalsByteString	0001111	15	bData	0101001	41
lessThanByteString	0010000	16	unConstrData	0101010	42
lessThanEqualsByteString	0010001	17	unMapData	0101011	43
sha2_256	0010010	18	unListData	0101100	44
sha3_256	0010011	19	unIData	0101101	45
blake2b_256	0010100	20	unBData	0101110	46
verifyEd25519Signature	0010101	21	equalsData	0101111	47
appendString	0010110	22	mkPairData	0110000	48
equalsString	0010111	23	mkNilData	0110001	49
encodeUtf8	0011000	24	mkNilPairData	0110010	50
decodeUtf8	0011001	25			

Table C.3: Tags for built-in functions (Batch 1)

Builtin	Binary	Decimal
serialiseData	0110011	51

Table C.4: Tags for built-in functions (Batch 2)

Builtin	Binary	Decimal
`verifyEcdsaSecp256k1Signature`	0110100	52
`verifySchnorrSecp256k1Signature`	0110101	53

Table C.5: Tags for built-in functions (Batch 3)

Builtin	Binary	Decimal
`bls12_381_G1_add`	0110110	54
`bls12_381_G1_neg`	0110111	55
`bls12_381_G1_scalarMul`	0111000	56
`bls12_381_G1_equal`	0111001	57
`bls12_381_G1_hashToGroup`	0111010	58
`bls12_381_G1_compress`	0111011	59
`bls12_381_G1_uncompress`	0111100	60
`bls12_381_G2_add`	0111101	61
`bls12_381_G2_neg`	0111110	62
`bls12_381_G2_scalarMul`	0111111	63
`bls12_381_G2_equal`	1000000	64
`bls12_381_G2_hashToGroup`	1000001	65
`bls12_381_G2_compress`	1000010	66
`bls12_381_G2_uncompress`	1000011	67
`bls12_381_millerLoop`	1000100	68
`bls12_381_mulMlResult`	1000101	69
`bls12_381_finalVerify`	1000110	70
`keccak_256`	1000111	71
`blake2b_224`	1001000	72
`integerToByteString`	1001001	73
`byteStringToInteger`	1001010	74

Table C.6: Tags for built-in functions (Batch 4)

Builtin	Binary	Decimal
andByteString	1001011	75
orByteString	1001100	76
xorByteString	1001101	77
complementByteString	1001110	78
readBit	1001111	79
writeBits	1010000	80
replicateByte	1010001	81
shiftByteString	1010010	82
rotateByteString	1010011	83
countSetBits	1010100	84
findFirstSetBit	1010101	85
ripemd_160	1010110	86

Table C.7: Tags for built-in functions (Batch 5)

Builtin	Binary	Decimal
expModInteger	1010111	87
dropList	1011000	88
lengthOfArray	1011001	89
listToArray	1011010	90
indexArray	1011011	91
bls12_381_G1_multiScalarMul	1011100	92
bls12_381_G2_multiScalarMul	1011101	93
insertCoin	1011110	94
lookupCoin	1011111	95
unionValue	1100000	96
valueContains	1100001	97
valueData	1100010	98
unValueData	1100011	99
scaleValue	1100100	100

Table C.8: Tags for built-in functions (Batch 6)

Builtin	Binary	Decimal
`multiIndexArray`	1100101	101
`policies`	1100110	102
`assetCount`	1100111	103
`keepPolicies`	1101000	104
`dropPolicies`	1101001	105

Table C.9: Tags for built-in functions (Batch 7)

C.3.6 Variable names

Variable names are encoded and decoded using the $\mathsf{E}_{\mathsf{name}}$ and $\mathsf{D}_{\mathsf{name}}$ functions, and variables bound in `lam` expressions are encoded and decoded by the $\mathsf{E}_{\lambda\mathsf{var}}$ and $\mathsf{D}_{\lambda\mathsf{var}}$ functions.

De Bruijn indices. We use serialised de Bruijn-indexed terms for script transmission because this makes serialised scripts significantly smaller. Recall from Section 2.1.3 that when we want to use our syntax with de Bruijn indices we replace names with natural numbers and the bound variable in a `lam` expression with 0. During serialisation the zero is ignored, and during deserialisation no input is consumed and the index 0 is always returned:

$$\mathsf{E}_{\lambda\mathsf{var}}(s, n) = s$$
$$\mathsf{D}_{\lambda\mathsf{var}}(s) = 0.$$

For variables we always use indices which are greater than zero, and our encoder and decoder for names are given by

$$\mathsf{E}_{\mathsf{name}} = \mathsf{E}_{\mathbb{N}}$$

and

$$\mathsf{D}_{\mathsf{name}}(s) = (s', n) \quad \text{if } \mathsf{D}_{\mathbb{N}} = (s', n) \text{ and } n > 0.$$

Other types of name. One can serialise code involving other types of name by providing suitable encoders and decoders for name. For example, for textual names one could use $\mathsf{E}_{\lambda\mathsf{var}} = \mathsf{E}_{\mathsf{name}} = \mathsf{E}_{\mathbb{U}^*}$ and $\mathsf{D}_{\lambda\mathsf{var}} = \mathsf{D}_{\mathsf{name}} = \mathsf{D}_{\mathbb{U}^*}$. Depending on the method used to represent variable names it may also be necessary to check during deserialisation the more general requirement that variables are well-scoped, but this problem will not arise if de Bruijn indices are used.

C.4 Cardano-specific serialisation issues

C.4.1 Scope checking

To execute a Plutus Core program on the blockchain it will be necessary to deserialise it to some in-memory representation, and during or immediately after deserialisation it should be checked that the body of the program is a closed term (see the requirement in Section 2.1.3); if this is not the case then evaluation should fail immediately.

C.4.2 CBOR wrapping

Plutus Core programs are not stored on the Cardano chain directly as `flat` bytestrings; for consistency with other objects used on the chain, the `flat` bytestrings are in fact wrapped in a CBOR encoding. This should not concern most users, but we mention it here to avoid possible confusion.

C.5 Example

Consider the program

```
(program 5.0.2
 [
  [(builtin indexByteString)(con bytestring #1a5f783625ee8c)]
  (con integer 54321)
 ])
```

Suppose this is stored in `index.uplc`. We can convert it to `flat` by running

```
$ cabal run exec uplc convert -- -i index.uplc --of flat -o index.flat
```

The serialised program looks like this:

```
$ xxd -b index.flat
00000000: 00000101 00000000 00000010 00110011 01110001 11001001  ...3q.
00000006: 00010001 00000111 00011010 01011111 01111000 00110110  ..._x6
0000000c: 00100101 11101110 10001100 00000000 01001000 00111000  %...H8
00000012: 10110100 00000001 10000001
```

Figure C.1 shows how this encodes the original program. Sequences of bits are followed by explanatory comments and lines beginning with # provide further commentary on preceding bit sequences.

```
00000101 : Final integer chunk: 0000101 → 5
00000000 : Final integer chunk: 0000000 → 0
00000010 : Final integer chunk: 0000000 → 2
         # Version: 5.0.2
0011     : Term tag 3: apply
0011     : Term tag 3: apply
0111     : Term tag 7: builtin
0001110  : Builtin tag 14
         # builtin indexByteString
0100     : Term tag 4: constant
1        : Start of type tag list
0001     : Type tag 1
0        : End of list
         # Type tags: [1] → bytestring
001      : Padding before bytestring
00000111 : Bytestring chunk size: 7
00011010 : 0x1a
01011111 : 0x5f
01111000 : 0x78
00110110 : 0x36
00100101 : 0x25
11101110 : 0xee
10001100 : 0x8c
00000000 : Bytestring chunk size: 0 (end of list of chunks)
         # con bytestring #1a5f783625ee8c
0100     : Term tag 4: constant
1        : Start of type tag list
0000     : Type tag 0
0        : End of list
         # Type tags: [0] → integer
11100010 : Integer chunk 1100010 (least significant)
11010000 : Integer chunk 1010000
00000110 : Final integer chunk 0000110 (most significant)
         # 0000110 · 1010000 · 1100010 → 108642 decimal
         # Zigzag encoding: 108642/2 → +54321
         # con integer 54321
000001   : Padding
```

Figure C.1: `flat` encoding of `index.uplc`

Bibliography

[1] ANSI. X9.62: Public Key Cryptography for the Financial Services Industry: the Elliptic Curve Digital Signature Algorithm (ECDSA), 2005.

[2] ANSI. X9.142: Public Key Cryptography for the Financial Services Industry: the Elliptic Curve Digital Signature Algorithm (ECDSA), 2020.

[3] Pasqualino 'Titto' Assini. Flat format specification. http://quid2.org/docs/Flat.pdf.

[4] Hendrik Pieter Barendregt. *The Lambda Calculus - its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, 1985.

[5] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. In *CHES*, volume 6917 of *Lecture Notes in Computer Science*, pages 124–142. Springer, 2011.

[6] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. The Keccak Reference. Submission to NIST (Round 3), January 2011.

[7] Bitcoin Wiki. Elliptic Curve Digital Signature Algorithm, 2022.

[8] Carsten Bormann. Notable CBOR Tags. https://www.ietf.org/archive/id/draft-bormann-cbor-notable-tags-06.html.

[9] Carsten Bormann and Paul E. Hoffman. RFC 8949: Concise Binary Object Representation (CBOR). https://datatracker.ietf.org/doc/html/rfc8949, December 2020.

[10] Antoon Bosselaers. The hash function RIPEMD-160. https://homes.esat.kuleuven.be/~bosselae/ripemd160.html, 2012.

[11] Sean Bowe. BLS12-381: New zk-SNARK Elliptic Curve Construction. https://electriccoin.co/blog/new-snark-curve/, March 2017.

[12] Certicom Research. Standards for Efficient Cryptography 2 (SEC 2). https://www.secg.org/SEC2-Ver-2.0.pdf, 2010.

[13] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Jann Müller, Michael Peyton Jones, Polina Vinogradova, and Philip Wadler. Native custom tokens in the extended utxo model. In *Leveraging Applications of Formal Methods, Verification and Validation: Applications: 9th International Symposium on Leveraging Applications of Formal Methods, ISoLA 2020, Rhodes, Greece, October 20–30, 2020, Proceedings, Part III*, page 89–111, Berlin, Heidelberg, 2020. Springer-Verlag.

[14] Cardano Improvement Proposal 0042 (CIP 0042) – New Plutus Builtin serialiseData. https://cips.cardano.org/cip/CIP-0042/.

[15] Cardano Improvement Proposal 0049 (CIP 0049) – ECDSA and Schnorr signatures in Plutus Core. https://cips.cardano.org/cip/CIP-0049/.

[16] Cardano Improvement Proposal 0059 (CIP 0059) – Terminology Surrounding Core Features. https://cips.cardano.org/cip/CIP-0059/.

[17] Cardano Improvement Proposal 0101 (CIP 0101) – Integration of `keccak256` into Plutus. https://cips.cardano.org/cip/CIP-0101/.

[18] Cardano Improvement Proposal 0109 (CIP-0109) – Modular Exponentiation Built-in for Plutus Core. https://cips.cardano.org/cip/CIP-0109.

[19] Draft Cardano Improvement Proposal 0121 (CIP 0121) – Integer-Bytestring conversions. https://github.com/cardano-foundation/CIPs/tree/master/CIP-0121.

[20] Draft Cardano Improvement Proposal 0122 (CIP 0122) – Logical operations over BuiltinByteString. https://github.com/cardano-foundation/CIPs/tree/master/CIP-0122.

[21] Draft Cardano Improvement Proposal 0123 (CIP 0123) – Bitwise operations over BuiltinByteString. https://github.com/cardano-foundation/CIPs/tree/master/CIP-0123.

[22] Cardano improvement proposal 0132 (CIP-0132) – New Plutus builtin `dropList`. https://cips.cardano.org/cip/CIP-0132.

[23] Cardano Improvement Proposal 0138 (CIP-0138) – Plutus Core Builtin Type - `Array`. https://cips.cardano.org/cip/CIP-0138.

[24] Cardano Improvement Proposal 0133 (CIP-0133) – Plutus support for multi-scalar multiplication over BLS12-381. https://cips.cardano.org/cips/cip0133/.

[25] Cardano Improvement Proposal 0153 (CIP 0153) – Plutus Core Builtin Type - MaryEraValue. https://cips.cardano.org/cips/cip0153/.

[26] Cardano Improvement Proposal 0156 (CIP-0156) – Plutus Core Builtin Function — `multiIndexArray`. https://cips.cardano.org/cip/CIP-0156.

[27] Cardano Improvement Proposal 0168 (CIP-0168) – More `BuiltinValue` Functions. https://cips.cardano.org/cip/CIP-0168.

[28] Cardano Improvement Proposal 0381 (CIP 0381) – Plutus support for pairings over BLS12_381. https://cips.cardano.org/cips/cip0381/.

[29] Craig Costello. Pairings for beginners. https://static1.squarespace.com/static/5fdbb09f31d71c1227082339/t/5ff394720493bd28278889c6/1609798774687/PairingsForBeginners.pdf.

[30] Duncan Coutts, Michael Peyton Jones, and Carsten Bormann. CBOR Tags for Discriminated Unions. https://github.com/cabo/cbor-discriminated-unions/.

[31] Quynh Dang. Secure Hash Standard (SHS). FIPS PUB 180-4, https://www.nist.gov/publications/secure-hash-standard, March 2012.

[32] N.G de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972.

[33] Hans Dobbertin, Antoon Bosselaers, and Bart Preneel. RIPEMD-160: A strengthened version of RIPEMD. In *Fast Software Encryption, Third International Workshop, Cambridge, UK, February 21-23, 1996, Proceedings*, volume 1039 of *Lecture Notes in Computer Science*, pages 71–82. Springer, 1996.

[34] Morris Dworkin. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. FIPS PUB 202, https://www.nist.gov/publications/sha-3-standard-permutation-based-hash-and-extendable-output-functions, August 2015.

[35] A. Faz-Hernandez, S. Scott, N. Sullivan, and R.S. Wahby. Hashing to Elliptic Curves (Version 16). https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-hash-to-curve, June 2022.

[36] Matthias Felleisen. Programming languages and lambda calculi, 2007.

[37] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. *Semantics Engineering with PLT Redex.* MIT Press, 2009.

[38] Matthias Felleisen and Daniel P. Friedman. Control operators, the SECD-machine, and the lambda-calculus. In *3rd Working Conference on the Formal Description of Programming Concepts*, August 1986.

[39] Matthias Felleisen and Robert Hieb. The revised report on the syntactic theories of sequential control and state. *Theor. Comput. Sci.*, 103(2):235–271, September 1992.

[40] Robert Harper. *Practical Foundations for Programming Languages*. Cambridge University Press, New York, NY, USA, 2012.

[41] Don Johnson, Alfred Menezes, and Scott A. Vanstone. The elliptic curve digital signature algorithm (ECDSA). *Int. J. Inf. Sec.*, 1(1):36–63, 2001.

[42] Simon Josefsson and Ilari Liusvaara. RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA). https://datatracker.ietf.org/doc/html/rfc8032, January 2017.

[43] Neal Koblitz. *A Course in Number Theory and Cryptography*, volume 114 of *Graduate Texts in Mathematics*. Springer-Verlag, 2nd edition, 1994.

[44] Johnson Lau, Jonas Nick, and Tim Ruffing. Bitcoin Improvement Proposal 340: Schnorr Signatures for secp256k1. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki, 2020.

[45] Johnson Lau and Pieter Wuille. Bitcoin Improvement Proposal 146: Dealing with signature encoding malleability. https://github.com/bitcoin/bips/blob/master/bip-0146.mediawiki, 2016.

[46] Supranational LLC. `blst library`. https://github.com/supranational/blst.

[47] Plinth User Guide. Working with scripts: Plutus Ledger Language Version. https://plutus.cardano.intersectmbo.org/docs/working-with-scripts/ledger-language-version.

[48] Gordon D. Plotkin. Call-by-name, call-by-value and the lambda-calculus. *Theor. Comput. Sci.*, 1(2):125–159, 1975.

[49] John C. Reynolds. Types, abstraction and parametric polymorphism. In R. E. A. Mason, editor, *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress, Paris, France, September 19-23, 1983*, pages 513–523. North-Holland/IFIP, 1983.

[50] Markku-Juhani O. Saarinen and Jean-Philippe Aumasson. The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC). RFC 7693, https://datatracker.ietf.org/doc/html/rfc7693, November 2015.

[51] Y. Sakemi, T. Kobayashi, T. Saito, and R. Wahby. Pairing-Friendly Curves (Version 11). https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-pairing-friendly-curves-11, November 2022.

[52] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *CRYPTO*, volume 435 of *Lecture Notes in Computer Science*, pages 239–252. Springer, 1989.

[53] Joseph H Silverman. *The Arithmetic of Elliptic Curves*, volume 106 of *Graduate Texts in Mathematics*. Springer-Verlag, 2009.

[54] The Unicode Consortium. The Unicode Standard. https://www.unicode.org/versions/latest/.

[55] F. Vercauteren. Optimal pairings. Cryptology ePrint Archive, Paper 2008/096, 2008. https://eprint.iacr.org/2008/096.

[56] Philip Wadler. Theorems for free! In *FPCA '89: Proceedings of the fourth international conference on Functional programming languages and computer architecture*, pages 347–359, New York, NY, USA, 1989. ACM.

[57] Zcash. Zcash BLS12-381 serialization format. https://github.com/zcash/librustzcash/blob/6e0364cd42a2b3d2b958a54771ef51a8db79dd29/pairing/src/bls12_381/README.md#serialization.

Index of Notation

Sets

$\uplus$	Disjoint union of sets, 4
$\mathsf{FinSup}(X, Y)$	Finitely supported functions from X to Y, 4
$f[a \mapsto b]$	f updated to map a to b, 5
X^*	The set of all finite sequences of elements of a set X, 5
X^+	The set of all nonempty finite sequences of elements of a set X, 5
$()$	The empty sequence/tuple, 5
$\mathbb{N}$	$\{0, 1, 2, 3, \dots\}$, 4
$\mathbb{N}_+$	$\{1, 2, 3, \dots\}$, 4
$\mathbb{N}_{[a,b]}$	$\{n \in \mathbb{N} : a \leq n \leq b\}$, 5
$\mathbb{B}$	$\{n \in \mathbb{Z} : 0 \leq n \leq 255\}$, 5
$\mathbb{B}^*$	The set of all bytestrings, 5
$\mathbb{b}$	$\{0, 1\}$, 5
$\mathbb{b}^*$	The set of all bitstrings, 5
$\mathbb{F}_q$	The finite field with q elements, 5
$\mathbb{F}_q^\times$	The multiplicative group of $\mathbb{F}_q$, 5
$\mathbb{U}$	The set of Unicode scalar values, 5
$\mathbb{U}^*$	The set of Unicode strings, 5
$\mathbb{Z}$	$\{\dots, -2, -1, 0, 1, 2, \dots\}$, 5
S_x	$S \uplus \{\mathsf{x}\}$ (S a set), 5
div, mod	Integer division operations, 36
quot, rem	Integer division operations, 36

Lists

$[]$	The empty list, 5
$L \cdot L'$	Concatenation of lists L and L', 5
$x \cdot L$	$[x] \cdot L$, 5
$L \cdot x$	$L \cdot [x]$, 5
$\overline{V}$	A sequence $[V_1, \dots, V_n]$, 5
$\ell(\cdot)$	Length of a list or bytestring, 5
ϵ	The empty bytestring, 64

Plutus Core grammar

bn, b	The name of a built-in function, 7
c	A literal constant, 7

L, M, N A term, 7
n A name, 7
n A natural number, 7
P A Plutus Core program, 7
v Plutus Core version, 7
x A variable name, 7

Built-in types

$\mathcal{C}_T$ Constants of built-in type T, 13
$\mathcal{O}$ The set of built-in type operator names, 10
$\mathcal{U}$ The universe of built-in types, 10
$\mathcal{U}_0$ The set of atomic type names, 10
$\mathcal{U}_\#$ The set of builtin-polymorphic types, 11
$\mathcal{V}$ The set of all type variables, $\mathcal{V}_\# \cup \{*\}$, 11
$\mathcal{V}_\#$ The set of builtin-polymorphic type variables, 11
op A built-in type operator (an element of $\mathcal{O}$), 10
$*$ The unique fully-polymorphic type variable, 11
$v_\#$ A builtin-polymorphic type variable, 11
T A built-in type (an element of $\mathcal{U}$), 10
$[\![T]\!]$ The denotation of a type name T, 10
$[\![\mathcal{U}]\!]$ $\biguplus \{[\![T]\!] : T \in \mathcal{U}\}$, 10
S A type assignment, 11
$\hat{S}$ The extension of a type assignment S to $\mathcal{U}_\# \cup \{*\}$, 12
P A builtin-polymorphic type, 11
$\mathsf{FV}_\#(P)$ Free #-variables of a polymorphic type $P \in \mathcal{U}_\#$, 11
$T \preceq_S P$ T is an instance of P via S and $\mathrm{dom}\, S = \mathsf{FV}_\#(P)$, 12
$[\![\cdot]\!]_T$ Denotation of constants of type T, 13
$\{\!|\cdot|\!\}_T$ Reification of constants of type T, 13
$\{\!|\cdot|\!\}$ Reification of the result of a built-in function application, 16
match_T Matching function for a built-in type T, 12
$\mathsf{maxBranch}(T)$ Maximum number of branches in a `case` expression on a built-in type T, 12

Built-in functions

$\mathcal{B}$ The set of built-in functions, 9
$\mathcal{I}$ The set of inputs to built-in functions, 13
$\mathcal{Q}$ The set of all type quantifications, 14
$\mathcal{R}$ The set of single values returned by built-in functions, 13
α Arity of built-in function, 14
$\bar{\alpha}$ Reduced arity, 15
χ Number of arguments of built-in function, 14
ι Signature item, 14
ω Return type of built-in function, 14
σ Signature of built-in function, 14
$\bar{\sigma}$ Reduced signature, 14
τ A member of $\mathcal{U}_\# \uplus \{*\}$, ie a type or type variable, 14

$\approx_S$	Compatibility of built-in function arguments with function arity via S, 17
$\forall v$	Type quantification, 14
$[\![b]\!]_S$	The denotation of the built-in function b at the type assignment S, 15
EvalBuiltin	Evaluation of built-in functions, 17

Term reduction

A	Well-formed partial built-in function application, 18
B	Partial built-in function application (possibly ill-formed), 18
B	Set of all partial built-in function applications, 18
V	Plutus Core value, 18
$\beta(B)$	Function in partial builtin application B, 18
$\|\|B\|\|$	Size of partial builtin application B, 18
next(A)	Next argument type (or `force`) required by a partial builtin application A, 19
args(A)	Term arguments received so far by partial builtin application A, 19
$[V/x]M$	Capture-avoiding substitution of value V for variable x in term M, 19
f	Reduction frame for contextual semantics: `[_ M]`, `[V _]`, `(force _)`, 20

CEK machine

$\triangleright$	CEK compute phase, 21
$\triangleleft$	CEK return phase, 21
$\blacklozenge$	CEK error state, 21
$\square$	CEK halting state, 21
Σ	CEK machine state, 22
s	CEK machine stack, 22
f	CEK stack frame: `(force _)`, `[_ (M, ρ)]`, `[V _]`, `(constr` $i\ M_1 \dots M_{j-1}\ _\ V_{j+1} \dots V_n$`)`, `(case _` $M_1 \dots M_n$`)`, 22
V	CEK value: ⟨`con` $T\ c$⟩, ⟨`delay` $M\ \rho$⟩, ⟨`lam` $x\ M\ \rho$⟩, ⟨`builtin` $b\ \overline{V}\ \eta$⟩, 22
ρ	CEK environment, 22
$\rho[x]$	Value bound to variable x in environment ρ, 22
η	Arguments expected by partial builtin application, 22
$\mathcal{U}(V)$	Discharge a CEK value V to obtain a Plutus Core term, 25
$M\,@\,\rho$	Discharge all variables bound by ρ in the term M, 25

Concrete syntax

$\mathbf{C}_T$	Set of strings used for the concrete syntax of constants of built-in type T, 30

Serialisation and deserialisation

$\mathcal{D}_X$	CBOR decoder for `data`, 64
$\mathcal{E}_X$	CBOR encoder for `data`, 64
D_X	Flat decoder, 70
E_X	Flat encoder, 70

Miscellaneous

$v\downarrow$	Concrete representation of a `value` v using nested association lists, 54
$l\uparrow$	A nested list l converted into a `value`, 55